



**UNIVERSIDAD
DE GRANADA**

TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA

Dego

Aplicación de Decisiones con Amigos

Autor

Natalia Serrano Cerceda

Director

Luis López Escudero



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍAS INFORMÁTICA Y DE
TELECOMUNICACIÓN

—
Granada, 31 de agosto de 2026

Dego: Aplicación de Decisiones con Amigos

Natalia Serrano Cerceda

Palabras clave: toma de decisiones, sistemas de votación, Flutter, Supabase, aplicaciones móviles, decisiones en grupo, recomendaciones audiovisuales

Resumen

En este Trabajo Fin de Grado se ha desarrollado una aplicación móvil orientada a facilitar la toma de decisiones, tanto individuales como grupales. La aplicación permite crear distintos tipos de decisiones, como votaciones, rankings y ruletas, e incorpora un sistema opcional para la resolución de empates mediante una votación adicional o una ruleta aleatoria. Asimismo, proporciona funcionalidades para la gestión de grupos de usuarios, notificaciones en tiempo real y la consulta del historial de decisiones finalizadas. Como funcionalidad adicional, integra información sobre películas y series, permitiendo obtener recomendaciones basadas en diferentes filtros y consultar información detallada de un título concreto. El desarrollo se ha realizado utilizando Flutter para la aplicación móvil y Supabase como plataforma backend.

Dego: Decisions with Friends Application

Natalia Serrano Cerceda

Keywords: decision-making, voting systems, Flutter, Supabase, mobile applications, group decisions, audiovisual recommendations

Abstract

In this Bachelor's Thesis, a mobile application has been developed to facilitate both individual and group decision-making. The application allows users to create different types of decisions, such as voting, rankings, and wheels, and incorporates an optional system for tie-breaking through an additional vote or a random wheel. Furthermore, it provides features for managing user groups, real-time notifications, and viewing the history of finalized decisions. As an additional feature, it integrates information about movies and series, allowing users to get recommendations based on different filters and consult detailed information about a specific title. The development was carried out using Flutter for the mobile application and Supabase as the backend platform.

Yo, **Natalia Serrano Cerceda**, alumno de la titulación Grado en Ingeniería Informática de la **Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación de la Universidad de Granada**, con DNI 71371023E, autorizo la ubicación de la siguiente copia de mi Trabajo Fin de Grado en la biblioteca del centro para que pueda ser consultada por las personas que lo deseen.

Fdo: Natalia Serrano Cerceda

Firmado por SERRANO
CERCEDA NATALIA -
***7102* el día
31/08/2026
certificado emitido por
AC FNMT Usuarios

Granada a 31 de agosto de 2026.

D. **Luis López Escudero**, Profesor del Departamento de Lenguajes y Sistemas Informáticos de la Universidad de Granada.

Informa:

Que el presente trabajo, titulado ***Dego, Aplicación de Decisiones con Amigos***, ha sido realizado bajo su supervisión por **Natalia Serrano Cerceda**, y autoriza la defensa de dicho trabajo ante el tribunal que corresponda.

Y para que conste, expide y firma el presente informe en Granada a 31 de agosto de 2026.

El director:

Firmado por LOPEZ ESCUDERO LUIS - ***5390**
el día 31/08/2026 con un certificado
emitido por AC FNMT Usuarios

Luis López Escudero

Agradecimientos

En primer lugar, quiero agradecer a mi familia por estar siempre ahí. Su guía, sus ánimos y los valores que me han inculcado a lo largo de mi vida me han ayudado a convertirme en la persona que soy hoy.

A mis amigos, por acompañarme durante todos estos años y, especialmente, por escuchar pacientemente mis avances en el monotema de todo este curso, aportando valiosas sugerencias de usabilidad y diseño siempre que lo necesité.

Por último, deseo expresar mi gratitud a los docentes que me han acompañado a lo largo de toda mi etapa educativa, con una mención especial a mi tutor, Luis, por estar siempre dispuesto a aclararme cualquier duda que me pudiera surgir.

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
2. Estado del arte	3
2.1. Aplicaciones	3
2.2. Posibles tecnologías a usar	6
2.2.1. Front-End	6
2.2.2. Back-End	9
2.2.3. Base de Datos	12
3. Especificación de Requisitos	17
3.1. Introducción	17
3.1.1. Propósito	17
3.1.2. Alcance	17
3.1.3. Definiciones	17
3.2. Descripción general	18
3.2.1. Perspectiva del producto	18
3.2.2. Funciones del Producto	19
3.2.3. Características de los usuarios	19
3.2.4. Restricciones	19
3.2.5. Suposiciones y dependencias	20
3.3. Requerimientos específicos	20
3.3.1. Requisitos funcionales	20
3.3.2. Requisitos no funcionales	22
3.3.3. Requisitos de interfaces externas	23
3.3.4. Interfaces externas	23
4. Planificación	25
4.1. Introducción	25
4.2. Cálculo de la velocidad	25
4.3. Historias de usuario	26
4.4. Planificación temporal	28

4.5. Diagrama de Gantt	29
4.6. Estimación de costes	29
4.6.1. Honorarios	31
4.6.2. Entidades y personal subcontratados	32
4.6.3. Adquisición de equipamiento para las actividades previstas en el proyecto	32
4.6.4. Promoción y difusión del proyecto	33
4.6.5. Costes indirectos	33
4.6.6. Total	33
5. Análisis	35
5.1. Product Backlog	35
5.2. Tarjetas de usuario	37
5.3. Diagrama de clases de análisis	59
6. Diseño	61
6.1. Arquitectura del sistema	61
6.2. API de la aplicación	63
6.3. Diagrama de base de datos	65
6.4. Diagrama de clases	72
6.5. Diagramas de secuencia	72
6.6. Diseño de interfaces de usuario	74
6.7. Diagrama de navegabilidad	79
7. Implementación	97
7.1. Herramientas	97
7.2. Explicación de la implementación	99
7.2.1. Decisiones	99
7.2.2. Decisiones de películas y series (<code>watch_decision</code>)	103
7.2.3. Seguridad y Control de Acceso	106
7.2.4. Gestión de grupos y usuarios	110
7.2.5. Notificaciones	113
8. Pruebas	117
8.1. Pruebas unitarias	117
8.2. Pruebas de widget	118
8.3. Pruebas de sistema	121
9. Conclusiones	125
9.1. Líneas de trabajo futuro	126
Bibliografía	132

Índice de figuras

2.1. Diferentes vistas de la aplicación Handshake	4
2.2. Fever página principal	5
2.3. TheFork resultado de una búsqueda	6
2.4. Diferentes vistas de la aplicación 2NITE	7
2.5. Diferentes vistas de la aplicación Daccord	7
2.6. Diferentes vistas de la aplicación TinyDecisions	8
2.7. ChoicePro resultado de manera científica	9
2.8. Diferentes vistas de la aplicación Rank it!	10
2.9. Interfaz de Android Studio	11
2.10. Interfaz de Xcode	12
2.11. Interfaz de Unity. Fuente: Elaboración propia	13
2.12. Logos de las tecnologías frontend	13
2.13. Interfaz de Django	14
2.14. Interfaz de Swagger. Fuente: Elaboración propia	14
2.15. Interfaz de Laravel	15
2.16. Logos de las tecnologías backend	15
2.17. Interfaz de MongoDB Compass	16
2.18. Logos de las tecnologías de base de datos	16
3.1. Diagrama de interacción del sistema [1] [2] [3] [4] [5] [6]	18
3.2. Bocetos de la aplicación	24
4.1. Diagrama de Gantt de las iteraciones	30
5.1. Diagrama de clases de análisis	60
6.1. Diagrama de la arquitectura del sistema [7] [8]	62
6.2. Diagrama de la base de datos	66
6.3. Diagrama de clases	81
6.4. Diagramas de secuencia de sesión	82
6.5. Diagrama de secuencia de registro	82
6.6. Diagramas de secuencia de creación	83
6.7. Diagramas de secuencia de creación de decisión	84
6.8. Diagramas de secuencia de visualización de una decision watch	85

6.9. Diagramas de secuencia de voto	85
6.10. Bocetos de las pantallas iniciales	86
6.11. Bocetos de las pantallas de grupos	87
6.12. Bocetos del historial de un grupo	88
6.13. Boceto de la pantalla inicial para crear una decisión	89
6.14. Bocetos del modo individual	89
6.15. Bocetos de las notificaciones	90
6.16. Bocetos del perfil	90
6.17. Creación y consulta de una decisión	91
6.18. Creación y consulta de una opción	91
6.19. Votar	92
6.20. Bocetos de decisión predeterminada	93
6.21. Bocetos de las pantallas de administración	94
6.22. Diagrama de la navegabilidad del sistema	95
7.1. Estructura de las tablas de base de datos con Supabase	98
7.2. Contenido de las tablas de base de datos con Supabase	98
7.3. Creación de la base de datos con Supabase	99
7.4. Gestión de los sprints con Jira	100
7.5. Gestión de las historias de usuario con Jira	101
7.6. Ciclo de vida de una decisión	102
7.7. Arquitectura reactiva de la aplicación	103
7.8. Respuesta de la API TMDB	104
7.9. Respuesta de la API TMDB	106
7.10. Proceso de verificación del correo electrónico	108
7.11. Privilegios de gestión	110
7.12. Dependencia entre las notificaciones	114
7.13. Interfaz de las notificaciones	115
8.1. Resultado de los tests unitarios	119
8.2. Resultado de los tests de widgets	120
8.3. Aspecto de las pantallas usadas en las pruebas de widget	122
8.4. Flujo de las pruebas de sistemas	123
8.5. Resultado de los tests de sistema	123

Índice de cuadros

4.1. Historias de usuario	26
4.2. Estimación de honorarios	32
4.3. Estimación de costes del proyecto	33
5.1. Orden del product backlog	35
6.1. API de usuarios	63
6.2. API de notificaciones	63
6.3. API de grupos	63
6.4. API de miembros de grupo	64
6.5. API de decisiones	64
6.6. API de opciones	64
6.7. API de watch_decision	64
6.8. API de watch_position	65
6.9. API de votos	65
6.10. Definición de la tabla <i>auth.users</i>	66
6.11. Definición de la tabla <i>usuario</i>	67
6.12. Definición de la tabla <i>grupo</i>	67
6.13. Definición de la tabla <i>group_members</i>	67
6.14. Definición de la tabla <i>notifications</i>	68
6.15. Definición de la tabla <i>decision</i>	68
6.16. Definición de la tabla <i>decision_watch</i>	69
6.17. Definición de la tabla <i>opcion</i>	69
6.18. Definición de la tabla <i>simple_vote</i>	70
6.19. Definición de la tabla <i>ranking_vote</i>	70
6.20. Definición de la tabla <i>watch_vote</i>	71
6.21. Definición de la tabla <i>watch_decision_position</i>	71

Capítulo 1

Introducción

En la actualidad, la toma de decisiones en grupo puede verse dificultada debido a la indecisión y la diversidad de opiniones. Situaciones cotidianas como elegir una película, planificar una salida cultural o decidir dónde cenar suelen requerir un tiempo significativo, llegando incluso a generar desacuerdos entre los participantes.

Este proyecto nace de la necesidad de optimizar los procesos de elección en entornos sociales. El objetivo principal es el desarrollo de una aplicación móvil orientada a la toma de decisiones en grupo de manera ágil e intuitiva. Mediante perfiles personalizables, los usuarios podrán crear núcleos sociales y proponer diversas alternativas que serán escogidas mediante distintos algoritmos de resolución, tales como la votación única, de forma aleatoria o por ranking.

1.1. Motivación

Este proyecto surge con el objetivo de reducir el tiempo invertido en decidir y mitigar los conflictos derivados de la indecisión y diversidad de opiniones. Abarcando desde elecciones triviales entre dos personas, como quién hace la comida, hasta decisiones más complejas con múltiples alternativas y un mayor número de participantes, como decidir qué película visualizar en común. Desde el punto de vista académico y técnico, esta aplicación constituye un reto significativo, al implicar la implementación completa de un sistema software de manera individual. Mientras que a lo largo del grado los proyectos de gran envergadura los hemos realizado en grupo, este proyecto me exige liderar todas las fases del ciclo de vida del software. Es especialmente relevante el reto técnico que supone el diseño y despliegue del back-end,

expandiendo mis conocimientos más allá del front-end, mi área de especialización en el grado. Esta visión integral del desarrollo me ayudará a definir hacia dónde quiero orientar mi trayectoria profesional. Asimismo, la complejidad técnica y la aplicabilidad real del producto final constituyen un valor diferencial para mi currículum, alineándose con las competencias de autonomía y resolución de problemas complejos que demanda el sector tecnológico actual.

1.2. Objetivos

Se pretende desarrollar una aplicación multiplataforma con la siguiente funcionalidad:

- Poder crear, editar y eliminar perfiles, grupos y decisiones
- Integrar múltiples metodologías de toma de decisiones
- Permitir consultar el historial de decisiones
- Disponer de opciones de personalización de la vista
- Garantizar la seguridad y privacidad de los datos personales
- Suministrar categorías de decisión por defecto
- Incorporar herramientas de filtrado
- Optimizar la toma de decisiones, reduciendo el tiempo y mitigando las discusiones

Capítulo 2

Estado del arte

2.1. Aplicaciones

En el mercado, existen ya aplicaciones que cuentan con una temática parecida. Vamos a comentarlas a continuación.

- **Handshake:** Es una aplicación disponible para dispositivos Android e iOS. Como podemos ver en la Figura 2.1a permite tomar decisiones en grupo sobre qué película o serie ver, dónde cenar, qué receta hacer, qué vino tomar, decidir el nombre de un bebé o crear tu elección personalizada. Para tomar decisiones se crea una sala con un código a la cual se pueden unir el resto de compañeros, tal como se ve en la Figura 2.1b. Entre sus carencias, encontramos que al elegir una película, no permite ver la descripción y se queda en un proceso de elección infinito que es necesario parar manualmente. Asimismo, dispone de anuncios y algunas opciones de pago, como puede observarse en la Figura 2.1c.
- **Fever [9]:** Está disponible en Android e iOS. Según se ilustra en la Figura 2.2 recomienda de manera individual distintas actividades culturales con coste en una ciudad. Permite elegir entre un listado o en un mapa.
- **TheFork [10]:** Funciona tanto en Android como en iOS. Su funcionamiento es encontrar restaurantes en tu ciudad, pudiendo visualizarlos en un mapa o un listado, tal como vemos en la Figura 2.3. Además, permite añadir ejemplos a la búsqueda y reservar en el que finalmente decidas.
- **2NITE [11]:** Se trata de una solución multiplataforma con soporte nativo para los sistemas operativos iOS y Android. Aporta información

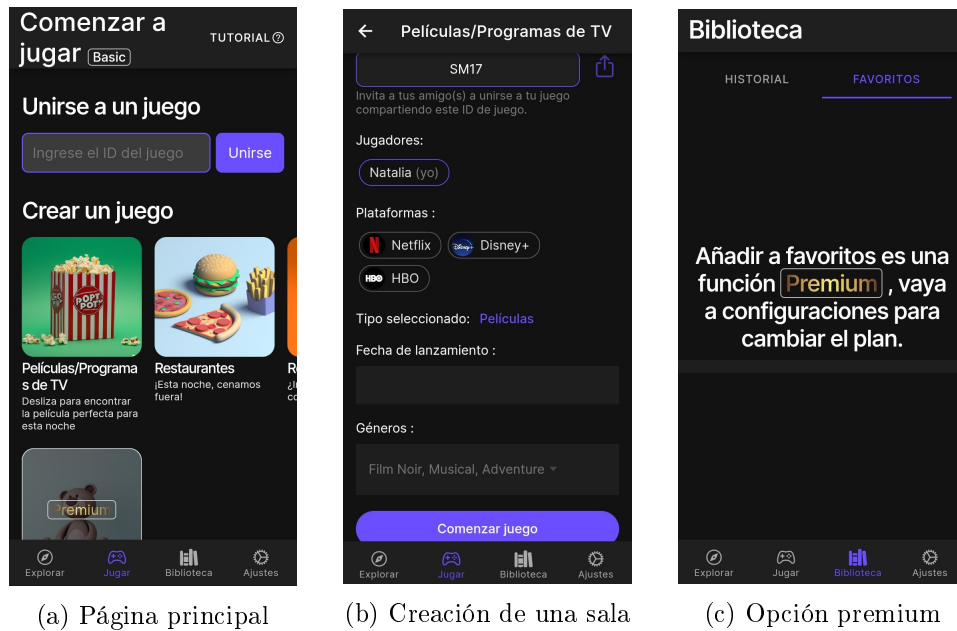


Figura 2.1: Diferentes vistas de la aplicación Handshake

sobre eventos musicales en localizaciones diversas como conciertos, bares, clubs, etc. Como podemos apreciar en la Figura 2.4a permite filtrar la información con diversos filtros tales como por ciudad o género musical. Asimismo, facilita información exclusiva de preventa, da recomendaciones personalizadas y permite comparar el precio de distintos proveedores de entradas como se muestra en la Figura 2.4b.

- **Daccord [12]:** Es una herramienta desarrollada para su ejecución en dispositivos móviles con arquitectura Android. Su función es permitir tomar decisiones en grupo, pudiendo crear o unirse a una sala mediante un código de tres palabras. Permite elegir tu nombre de usuario y foto de perfil eligiendo entre unos iconos que tienen de modelo. Además, de poder ponerlo en modo claro u oscuro. En este momento, solo permite crear una lista de opciones con descripción y opcionalmente incluirle una foto a cada una de ellas. No obstante, tanto insertar una imagen como crear un grupo de decisión generan una excepción como podemos observar en la Figura 2.5a. Al tratarse de una herramienta reciente, diversas funcionalidades se encuentran aún en fase de desarrollo como ilustra la Figura 2.5b, tales como temáticas predefinidas (películas, restaurantes, juegos, deportes y viajes) y utilidades como botones para poner un tiempo límite para decidir o permitir elegir una opción como no negociable. En estos momentos, solo está disponible en inglés.
- **Tiny Decisions [13]:** La plataforma se encuentra accesible para usua-

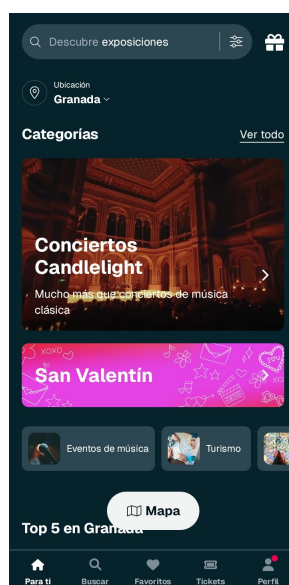


Figura 2.2: Fever página principal

rios tanto de terminales Android como iPhone. Su función es de toma de decisiones individuales o en grupo desde un mismo dispositivo pudiendo elegirse tal como podemos ver en la Figura 2.6a mediante una ruleta, poner un dedo por persona en la pantalla y seleccionar aleatoriamente uno, sacar un número o lanzar una moneda. Como se observa en la Figura 2.6b tiene muchas opciones de ruletas por defecto y permite crear las tuyas propias pudiendo personalizar el color de las opciones y organizar las ruletas por etiquetas. Asimismo, dispone de modo claro y oscuro y está traducida en distintos idiomas. Adicionalmente, tiene disponibles opciones premium como eliminar los anuncios, ponerle distinto peso a las opciones de la ruleta, elegir más de un dedo, etc.

- **ChoicePro:** Es una herramienta desarrollada para dispositivos Android. Permite tomar decisiones de manera individualizada mediante una ruleta o de manera científica. En la segunda modalidad, el usuario puntúa las distintas alternativas mediante unos criterios personalizados y una prioridad ajustable, aportando al final las puntuaciones de las distintas opciones e indicando la óptima, como se puede apreciar en la Figura 2.7.
- **Rank it!:** Es una aplicación disponible para Android. Ayuda a la toma de decisiones de manera individual o en grupo desde un mismo dispositivo. Permite que haya hasta doce opciones y hasta diez jugadores. La decisión como podemos observar en la Figura 2.8a se toma de manera aleatoria con una ruleta o mediante un ranking que hacen los jugado-

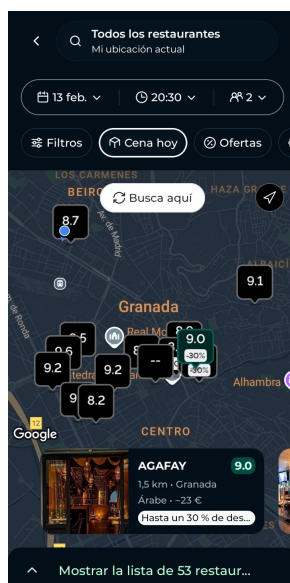


Figura 2.3: TheFork resultado de una búsqueda

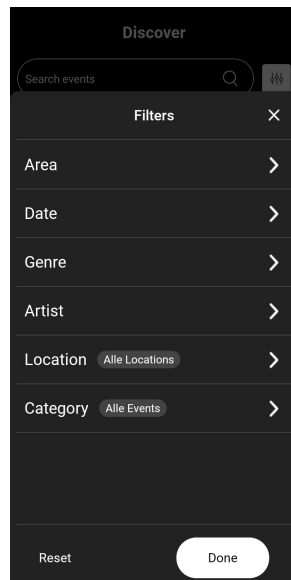
res por turnos. Si la decisión final es un empate tal como muestra la Figura 2.8b, se elije entre las opciones empatadas de manera aleatoria o elije el dueño del dispositivo. Al hacer una nueva partida permite conservar las opciones y/o los jugadores.

2.2. Posibles tecnologías a usar

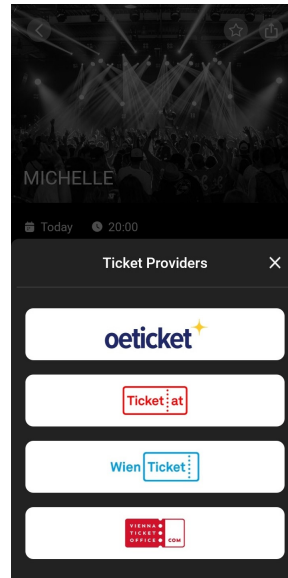
2.2.1. Front-End

Algunas de las posibles tecnologías front-end que más se alinean con los objetivos de esta aplicación son [14]:

- **Flutter [15] (2.12a):** Framework de Google que permite el desarrollo multiplataforma en iOS, Android, Web y Desktop con un único código fuente. Usa su propio motor gráfico, lo que le proporciona un buen rendimiento y una interfaz consistente. Tiene un desarrollo rápido y permite crear interfaces visuales complejas. Posee una amplia biblioteca de widgets y elementos de interfaz de usuario prediseñados.
- **React Native [16]:** Framework basado en React que permite el desarrollo multiplataforma usando tecnologías web. El código se traduce a componentes nativos, logrando un rendimiento cercano al nativo. Consta de una gran comunidad, amplias bibliotecas y plugins de terceros. Sin embargo, algunas funcionalidades nativas pueden requerir módulos

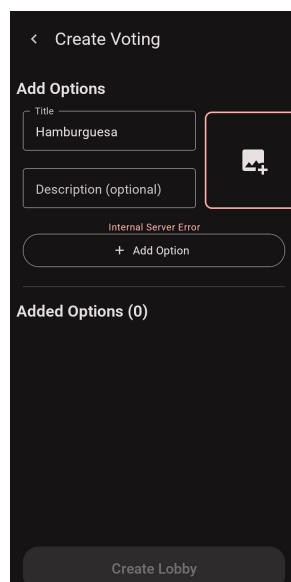


(a) Opciones de filtrado

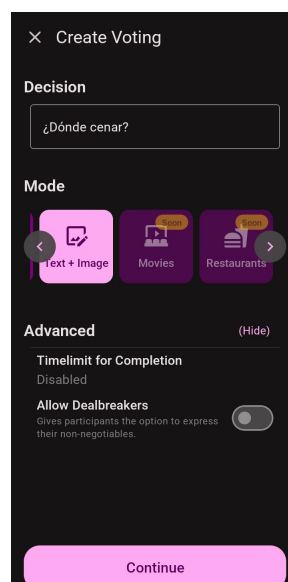


(b) Opciones de compra de entradas

Figura 2.4: Diferentes vistas de la aplicación 2NITE



(a) Error al insertar una imagen



(b) Temáticas en desarrollo

Figura 2.5: Diferentes vistas de la aplicación Daccord



Figura 2.6: Diferentes vistas de la aplicación TinyDecisions

personalizados. Además, el rendimiento se puede ver afectado en tareas complejas.

- **Android SDK [17]:** Proporciona las herramientas necesarias para el desarrollo nativo en Android. Aporta el máximo rendimiento y acceso completo al hardware del dispositivo. El uso de lenguajes como Kotlin permite usar librerías y APIs nativas mejorando el rendimiento. Para utilizarlo podemos usar Android Studio como se ve en la Figura 2.9
- **iOS SDK [18]:** Tecnología oficial para el desarrollo nativo en iOS. Permite crear aplicaciones estables y altamente integradas, ofreciendo un alto rendimiento y un diseño acorde a Apple. Se puede programar con la interfaz que proporciona Xcode como podemos observar en la Figura 2.10
- **Ionic [19] (2.12c):** Framework híbrido de desarrollo multiplataforma usando tecnologías web. Se ejecuta dentro de un contenedor web facilitando el desarrollo, pero reduciendo el rendimiento. Es adecuado para proyectos sencillos o prototipos.
- **.NET MAUI [20] (2.12d):** Framework multiplataforma de Microsoft basado en el ecosistema .NET. Permite compartir gran parte del código entre Android e iOS, permitiendo acceder a funcionalidades nativas.

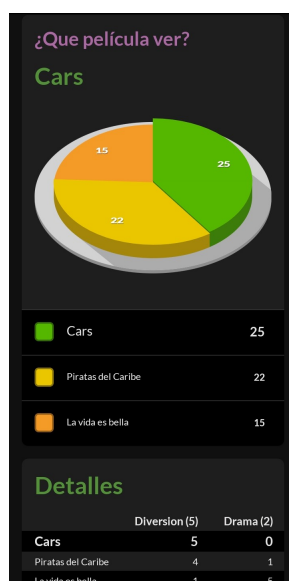


Figura 2.7: ChoicePro resultado de manera científica

- **Unity [21]:** Motor de desarrollo multiplataforma que posee una gran interfaz gráfica y funciones de arrastre. Está escrito en C# y posee un uso sencillo. Puede crear aplicaciones en 2D y 3D. Sin embargo, puede requerir plugins adicionales, ser excesivo para aplicaciones sencillas y tener un alto coste de licencias. Podemos usar su interfaz que presenta el aspecto que vemos en la Figura 2.11
- **NativeScript [22] (2.12e):** Framework de desarrollo multiplataforma usando tecnologías web. Utiliza APIs y componentes nativos, dando un gran rendimiento. Posee un amplio conjunto de plugins, sin embargo, la depuración puede ser compleja.

2.2.2. Back-End

- **Django [23]:** Marco de desarrollo de Python ideal para aplicaciones grandes, CMS y proyectos que tienen que salir a producción rápidamente integrando seguridad. Incluye un panel de administración automático y un mapeo objeto-relacional para base de datos. Sin embargo, puede resultar pesado y tiene una estructura muy rígida. Como podemos apreciar en la Figura 2.13 cuenta con un panel de administración propio.
- **Flask [24] (2.16a):** Microframework del lenguaje Python enfocado en la simplicidad. Es usado en proyectos pequeños, prototipos rápidos

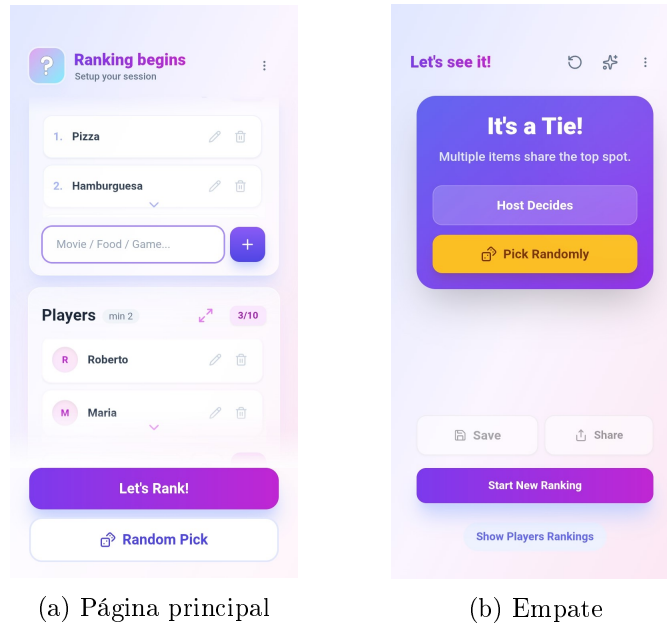


Figura 2.8: Diferentes vistas de la aplicación Rank it!

y para tener control de cada componente. Destaca por ser muy ligero, flexible y de rápido aprendizaje.

- **FastAPI [25]:** Plataforma de Python increíblemente rápido. Es muy utilizado para crear APIs de alto rendimiento, aplicaciones de Machine Learning y sistemas asíncronos. Además, genera documentación interactiva automáticamente. Cuenta con la interfaz Swagger que podemos visualizar en la Figura 2.14
- **Express.js [26] (2.16b):** Framework minimalista de JavaScript. Es adecuado para una amplia gama de proyectos y cuenta con una de las comunidades más grandes. No obstante, no impone una estructura, pudiendo generar problemas de mantenimiento en proyectos grandes
- **NestJS [27] (2.16c):** Plataforma usada en JavaScript y TypeScript que aporta una estructura profesional y organizada basada en módulos y decoradores. Establece una arquitectura altamente robusta, facilitando el testing y el trabajo en equipos grandes. Como punto negativo, presenta una curva de aprendizaje elevada.
- **Spring Boot [28] (2.16d):** Framework robusto y escalable orientado a aplicaciones empresariales. Spring Boot simplifica la configuración y el despliegue de servicios web. Aporta seguridad, alta fiabilidad y mantenimiento a largo plazo.

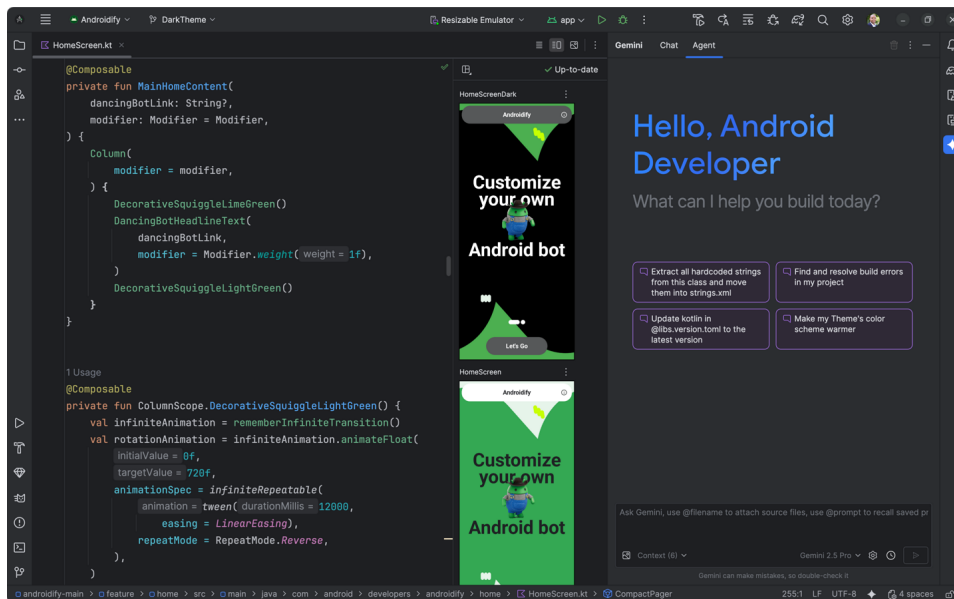


Figura 2.9: Interfaz de Android Studio

- **Laravel [29]:** Facilita el desarrollo rápido de aplicaciones webs y APIs. Contiene herramientas que conceden autenticación, validación y acceso a bases de datos. Es sencillo y eficaz para proyectos medianos. Utiliza el panel de administrador propio que podemos ver en la Figura 2.15
- **ASP.NET Core [30]:** Plataforma de alto rendimiento desarrollada por Microsoft y perteneciente al ecosistema .NET. Permite crear APIs seguras y escalables, con buena integración en entornos cloud. Se usa en proyectos empresariales y profesionales. Asimismo, hace uso de la interfaz de Swagger que podemos ver en la Figura 2.14
- **FireBase [30] (2.16e):** Plataforma de Google que ofrece servicios listos para usar como autenticación, base de datos y notificaciones. Reduce la necesidad de crear y mantener un servidor propio. Se utiliza en proyectos pequeños o con tiempo de desarrollo limitados.
- **Ruby on Rails [31] (2.16f):** Framework de desarrollo web basado en Ruby. Sigue el patrón modelo-vista-controlador. Es de fácil uso y flexible. Posee sintaxis sencilla, es orientado a objetos y permite cambios de manera sencilla en el código.
- **Supabase: [32] (2.16g):** Plataforma Backend-as-a-Service de código abierto basada en PostgreSQL. Aporta una API REST, servicios de autenticación, almacenamiento de archivos y control de acceso. Especialmente adecuada para aplicaciones con múltiples usuarios y necesidad de sincronización en tiempo real

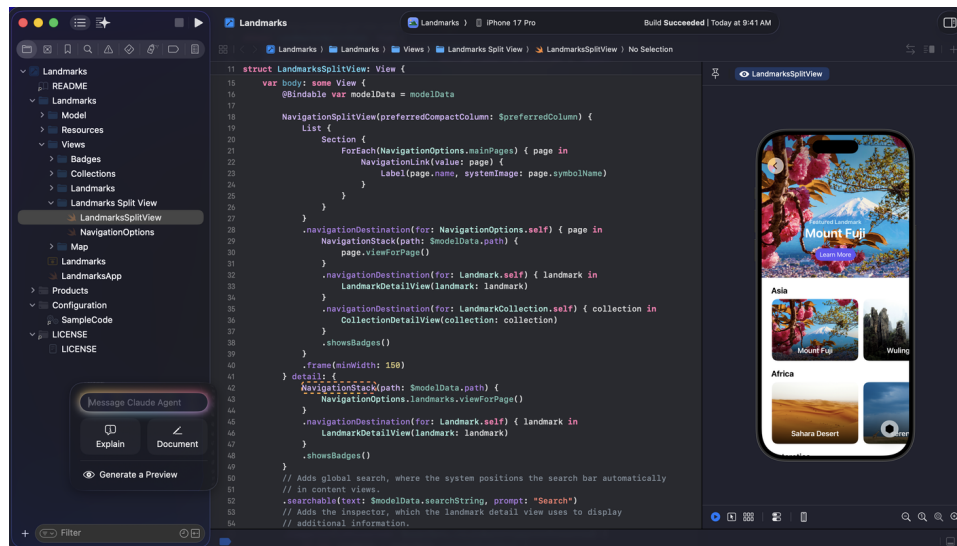


Figura 2.10: Interfaz de Xcode

2.2.3. Base de Datos

Relaciones:

- **PostgreSQL [33] (2.18a):** Consta de soporte para consultas complejas y alto rendimiento. Es muy robusta y escalable, siendo ideal para manejar grandes volúmenes de datos o lógica compleja.
- **MySQL [34] (2.18b) o MariaDB [35] (2.18c):** Es de fácil uso. Utiliza tablas y relaciones garantizando la integridad de los datos. Es adecuado para la mayoría de aplicaciones móviles.
- **SQLite [36] (2.18d):** Base de datos ligera que no necesita servidor, ya que se ejecuta localmente. Es usada para almacenamiento local, caché o funcionamiento sin conexión a internet.

No relacionales:

- **MongoDB [37]:** Almacena la información en formato JSON. Es flexible, escalable e ideal para aplicaciones con estructuras de datos cambiantes. Para utilizarlo podemos hacer uso de la interfaz MongoDB Compass que podemos observar en la Figura 2.17
- **Firebase Firestore [38] o Realtime Database [39] (2.16e):** Es almacenada en la nube y sincronizada en tiempo real. Es muy útil para aplicaciones colaborativas o con notificaciones instantáneas.

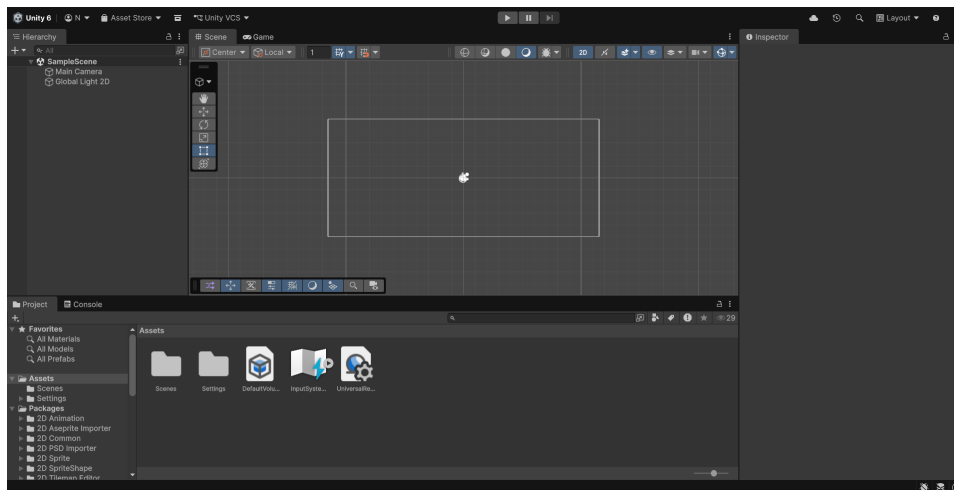


Figura 2.11: Interfaz de Unity. Fuente: Elaboración propia

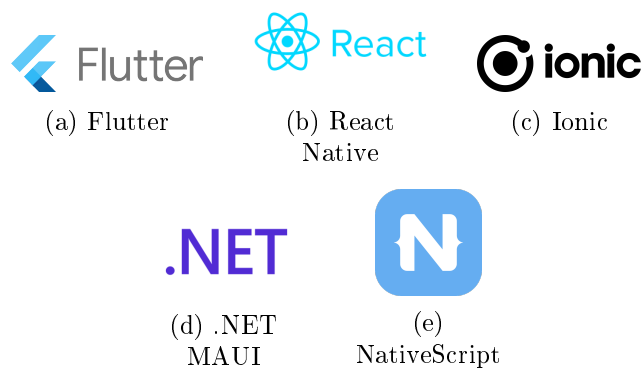


Figura 2.12: Logos de las tecnologías frontend

- **Redis [40] (2.18e):** Su almacenamiento es guardado en memoria y es extremadamente rápida mejorando el rendimiento. Se usa principalmente para caché, gestión de sesiones o colas de tareas.

Add flat page

URL:
Example: "/about/contact/". Make sure to have leading and trailing slashes.

Title:

Content:

Sites:

example.com +

Hold down "Control", or "Command" on a Mac, to select more than one.

► Advanced options

SAVE **Save and add another** **Save and continue editing**

Figura 2.13: Interfaz de Django

FastAPI 0.1.0 OAS 3.1
openapi.json

Authorize

User ^

POST	/user/register	Register	▼
POST	/user/login	Login For Access Token	▼
GET	/user/id/{user_id}	Get User	🔒 ▼
GET	/user/current	Get Current User	🔒 ▼
GET	/user/tutor/{tutor_id}/students	Get All Students	▼
GET	/user/students/{student_id}/tutor	Get Student Tutor	🔒 ▼
POST	/user/	Create User	🔒 ▼
POST	/user/assign_student/{student_id}/tutor	Assign Student	🔒 ▼
PUT	/user/{user_id}/update	Update User	🔒 ▼

Figura 2.14: Interfaz de Swagger. Fuente: Elaboración propia

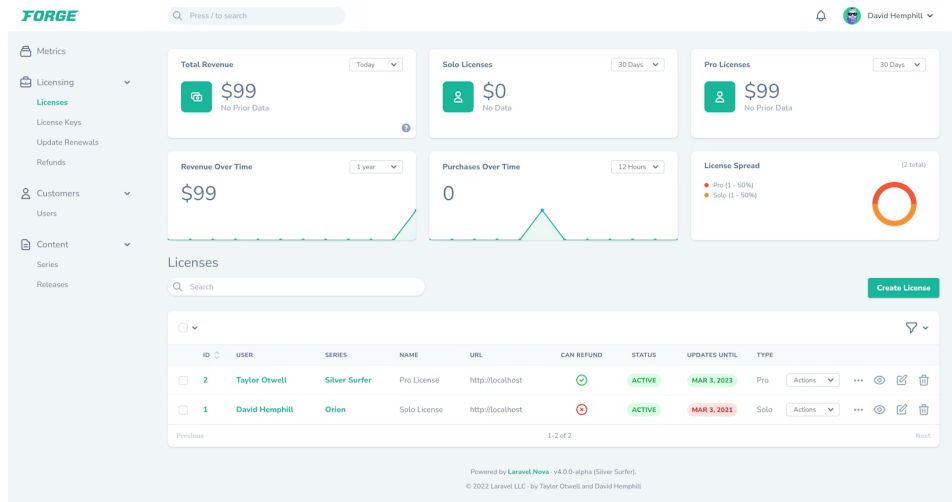


Figura 2.15: Interfaz de Laravel



Figura 2.16: Logos de las tecnologías backend

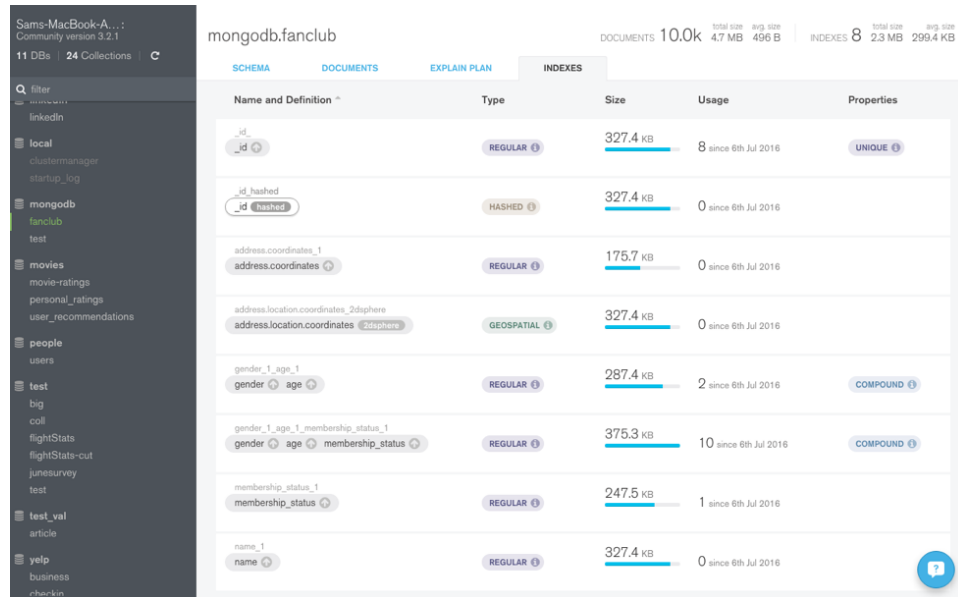


Figura 2.17: Interfaz de MongoDB Compass



Figura 2.18: Logos de las tecnologías de base de datos

Capítulo 3

Especificación de Requisitos

3.1. Introducción

3.1.1. Propósito

El propósito de esta especificación de requisitos es definir de manera sencilla, breve y clara las funciones y restricciones que deberá poseer el producto final que se va a desarrollar.

Para elaborarlo, se tendrán en cuenta las necesidades de los usuarios finales. El resultado final servirá como base para la construcción de la aplicación y se tendrá en cuenta a lo largo de todo su desarrollo.

3.1.2. Alcance

La aplicación facilitará la toma de decisiones dentro de un grupo social, pudiendo usar opciones por defecto o personalizadas y eligiendo finalmente mediante diversos métodos de elección. A su vez, irá guardando y tomando los datos de la base de datos del sistema.

3.1.3. Definiciones

- *Administrador*: Persona con acceso completo al sistema software y responsable del mantenimiento de la base de datos. De igual modo, podrá realizar las funciones de los *clientes*.
- *Cliente*: Persona interesada en la toma de decisiones gamificada que ofrece la aplicación.
- *Servidor*: Equipo de cómputo en el que se implementará el sistema.

- Grupo: Conjunto de clientes que pueden tomar decisiones en común.

3.2. Descripción general

En este apartado, vamos a proporcionar una visión general del sistema. Con el objetivo, de determinar las funciones principales, datos necesarios y restricciones que afecten al desarrollo.

3.2.1. Perspectiva del producto

En la Figura 3.1 se muestra la interacción entre los distintos componentes de la aplicación. Como podemos observar, la arquitectura se compone de tres elementos principales: el front-end desarrollado en Flutter, el back-end proporcionado por Supabase y el sistema de base de datos PostgreSQL.

El front-end se encarga de la interfaz de usuario con la que interactúan los usuarios a través de una pantalla táctil. Para ello, envía peticiones al back-end mediante solicitudes HTTPS en formato JSON.

El back-end gestiona la autenticación, la lógica de acceso a los datos y las funcionalidades de actualización en tiempo real.

Finalmente, los datos se almacenan en una base de datos PostgreSQL, donde se aplican políticas de seguridad a nivel de fila RLS (Row Level Security) para garantizar el acceso controlado a la información.

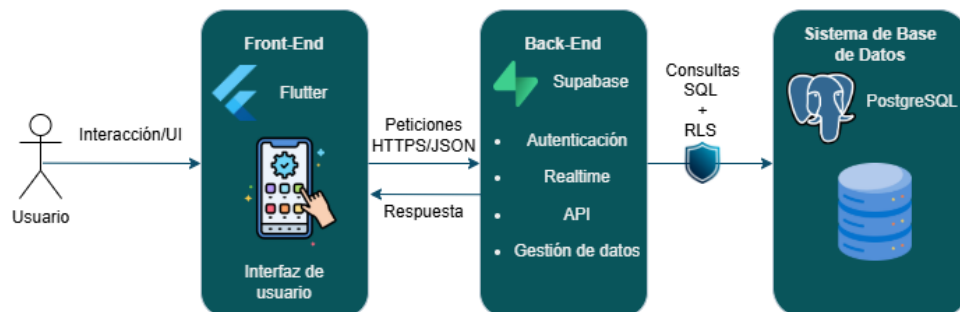


Figura 3.1: Diagrama de interacción del sistema [1] [2] [3] [4] [5] [6]

3.2.2. Funciones del Producto

Las funciones principales del sistema son la F1 y F2, las cuales vamos a comentar a continuación:

- Código F1: Gestión de una base de datos (permitiendo insertar, actualizar y eliminar) en la que se almacenan los miembros de cada grupo, las opciones de cada decisión por defecto y la información de los usuarios y su rol en la aplicación.
- Código F2: Devolver información filtrada sobre ocio a través de APIs y/o *scraping*.

Asimismo, la aplicación contará con las siguientes funciones:

- Creación-Edición-Eliminación: Se usará tanto para usuarios como para grupos, decisiones y opciones de las decisiones.
- Añadir-Eliminar: Podrá utilizarse sobre los miembros de un grupo.
- Historial: Se podrán consultar las votaciones anteriores.

3.2.3. Características de los usuarios

Es recomendable que los usuarios tengan conocimientos básicos de informática y estén familiarizados con el uso de aplicaciones en dispositivos móviles.

3.2.4. Restricciones

Las principales restricciones que debe cumplir el sistema son RES1 y RES2, las cuales vamos a comentar a continuación:

- Código RES1: La aplicación estará disponible para dispositivos móviles,
- Código RES2: El sistema será multiplataforma, pudiendo ejecutarse en sistemas operativos Android e iOS.
- Código RES3: Siempre debe haber al menos un administrador creado en el sistema.
- Código RES4: Un administrador no podrá cambiar su propio rol o eliminarse.

- Código RES5: Será necesario disponer de conexión a Internet para poder utilizar la aplicación.
- Código RES6: No será necesario hardware adicional aparte del propio dispositivo móvil para utilizar la aplicación.

3.2.5. Suposiciones y dependencias

Se asume que el sistema operativo de destino es Android o iOS y que estará disponible y correctamente configurado en los equipos donde se instalará la aplicación.

3.3. Requerimientos específicos

3.3.1. Requisitos funcionales

- Gestión de usuario:
 - Código REQ1: Registro de usuario → El sistema permitirá al usuario crearse una cuenta personal asociada a él.
 - Código REQ2: Modificación de usuario → El usuario podrá modificar los datos de su cuenta.
 - Código REQ3: Eliminación de usuario → Un usuario podrá eliminar su cuenta personal.
 - Código REQ4: Subida de imágenes → Los usuarios podrán adjuntar imágenes en el sistema.
 - Código REQ5: Historial → El sistema permitirá a los usuarios consultar un histórico de las votaciones pasadas.
- Gestión de notificaciones:
 - Código REQ6: Notificaciones → El usuario será informado de los cambios en los grupos a los que pertenezca.
- Gestión de administrador:
 - Código REQ7: Creación de administrador → Para crear un administrador otro administrador deberá cambiar el rol de un usuario de cliente a administrador.
 - Código REQ8: Eliminación de administrador → Para eliminar un administrador otro administrador deberá cambiarle el rol a cliente.

- Código REQ9: Gestión de datos → El administrador podrá modificar o eliminar la información almacenada en el sistema.
- Gestión de los grupos:
 - Código REQ10: Creación de grupos → Los usuarios podrán crear grupos asignándoles un nombre y opcionalmente una imagen y lista de miembros.
 - Código REQ11: Añadir usuarios → Los miembros de un grupo podrán invitar a otros miembros.
 - Código REQ12: Eliminar usuarios → Los miembros de un grupo podrán eliminar a otros miembros.
 - Código REQ13: Salir de un grupo → Los usuarios que formen parte de un grupo podrán abandonarlo cuando deseen.
 - Código REQ14: Permiso de voto → El sistema permitirá a los usuarios pertenecientes a un grupo participar en las votaciones activas del mismo.
- Gestión de las votaciones:
 - Código REQ15: Creación de la votación → El sistema permitirá crear una votación especificando título, tipo de votación y al menos dos opciones.
 - Código REQ16: Eliminación de la votación → El sistema permitirá eliminar una votación activa.
 - Código REQ17: Creación de opciones → El sistema permitirá que los usuarios añadan opciones a las decisiones, contando cada una con título y opcionalmente descripción e imagen.
 - Código REQ18: Límite de tiempo → El creador de la votación podrá fijar un tiempo máximo para que el resto de miembros ejerza su derecho al voto.
 - Código REQ19: Mecanismo de votación → El sistema permitirá al creador de la votación elegir el tipo de votación, pudiendo elegir entre votación única, votación múltiple, ranking o ruleta.
 - Código REQ20: Ganador de la votación → Una vez acabado el tiempo destinado a la votación o habiendo votado todos los miembros, el sistema calculará automáticamente el ganador según los votos y el método de votación elegido.
 - Código REQ21: Mostrar los resultados → Al finalizar una votación los miembros podrán visualizar los resultados.
 - Código REQ22: Desempatar → Se permitirá desempatar una votación finalizada mediante un voto extra del creador de la decisión o una ruleta.

- Gestión de la aplicación:

- Código REQ23: Filtrado → El sistema contará con sistemas de filtrado y búsqueda mediante APIs o scraping para aportar opciones en la votación.
- Código REQ24: Inicio de sesión → Todos los usuarios entrarán al sistema mediante un inicio de sesión compuesto por nombre de usuario o correo electrónico y contraseña.
- Código REQ25: Cerrado de sesión → Todos los usuarios podrán cerrar su sesión del dispositivo que estén usando.

3.3.2. Requisitos no funcionales

- Código REQ27: Mantenibilidad → El código será estructurado y comentado para facilitar futuras modificaciones o ampliaciones.
- Código REQ28: Fiabilidad → El sistema deberá tener un funcionamiento estable, minimizando la aparición de errores durante su uso.
- Código REQ29: Robustez → El sistema deberá responder adecuadamente ante situaciones inesperadas o entradas incorrectas sin generar fallos en la aplicación.
- Código REQ30: Facilidad de uso → La interfaz será intuitiva, permitiendo a los usuarios usarla sin formación previa.
- Código REQ31: Protección de datos → Los datos de los usuarios serán protegidos y tratados siguiendo lo establecido en la normativa vigente de protección de datos (Reglamento General de Protección de Datos (RGPD) [41] y Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales (LOPDGDD) [42]).
- Código REQ32: Autenticación de usuarios → Únicamente podrán acceder al sistema los usuarios autenticados.
- Código REQ33: Compatibilidad multiplataforma → La aplicación deberá funcionar en sistemas operativos Android e iOS.
- Código REQ34: Integridad de la información → Los datos guardados no deberán perderse ni corromperse ante fallos del sistema.
- Código REQ35: Escalabilidad → El sistema deberá soportar el crecimiento del número de usuarios y grupos.
- Código REQ36: Concurrencia → El sistema deberá gestionar los votos emitidos simultáneamente por múltiples usuarios sin producir conflictos o pérdida de información.

- Código REQ37: Sincronización → Los cambios producidos en una votación deberán ser transmitidos a todos los usuarios del grupo en un tiempo mínimo.
- Código REQ38: Consistencia de la información → Todos los usuarios deberán visualizar el mismo estado y resultado en las votaciones, evitando inconsistencias entre dispositivos.

3.3.3. Requisitos de interfaces externas

- Código REQ39: Interfaces de usuario → Se podrá comunicar con el usuario para aprovechar los requisitos del sistema, el usuario indicará al sistema las operaciones que desea realizar e introducirá los datos que el sistema le pida.
- Código REQ40: Interfaces del software → La comunicación entre los módulos del sistema se realizará mediante bases de datos.

3.3.4. Interfaces externas

- Interfaz de usuario: Será de fácil manejo, contando con una interfaz intuitiva. Tendrá una apariencia similar a los bocetos mostrados en la Figura 3.2. Como podemos apreciar, la página principal consta de un menú superior de búsqueda, un menú de navegación inferior y una lista central con los distintos grupos a los que se pertenece. Al acceder a un grupo específico, la vista se organiza verticalmente mostrando primero los datos del grupo junto con su navegación, seguidos de la sección de decisiones y finalmente el menú de navegación global.
- Interfaz hardware: Será desarrollado para poder usarse con una pantalla táctil. Asimismo, no hará falta una interfaz hardware específica para ejecutarlo.
- Interfaz software: La interfaz software utilizada será la proporcionada por los Sistemas Operativos que vamos a utilizar, en este caso Android e iOS.
- Comunicación entre interfaces: La comunicación con funciones del sistema se realizará mediante *Platform Channels*, ya sea a través de *plugins* de la comunidad o mediante implementaciones personalizadas.

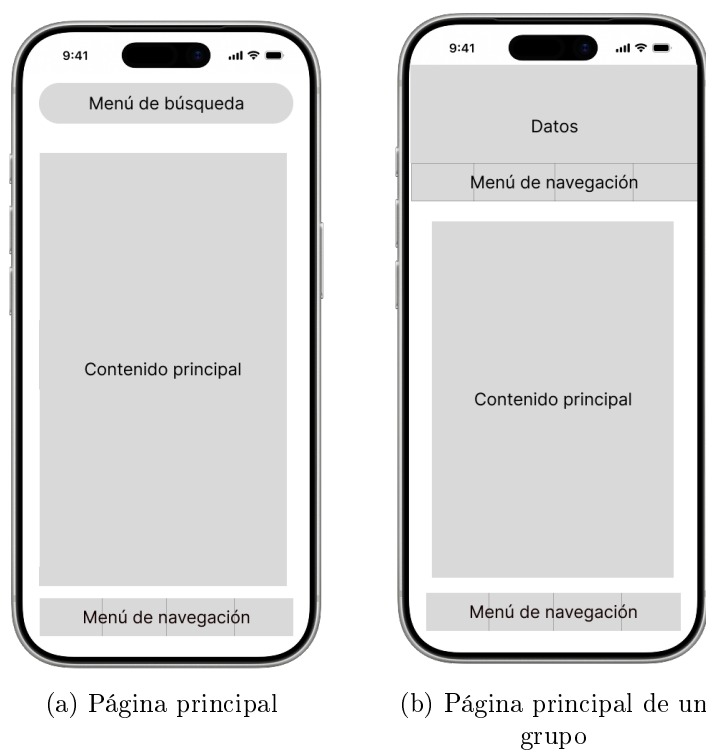


Figura 3.2: Bocetos de la aplicación

Capítulo 4

Planificación

4.1. Introducción

En este capítulo se explica la metodología y ritmo de trabajo que se seguirá durante el desarrollo del proyecto. Con respecto a la metodología, se empleará un modelo personalizado basado en distintos aspectos de la metodología ágil Scrum. Cabe destacar que, pese a que se usarán elementos de Scrum, no se seguirá esta metodología, debido a que está orientada al trabajo en equipo y, por tanto, no se adapta completamente a un proyecto individual como este.

Las tareas se organizarán en historias de usuario que se desarrollarán en sprints de dos semanas de duración. Al final de cada sprint, se tendrá una reunión con el tutor del TFG en la que se revisará el trabajo realizado, se identificarán posibles mejoras y se marcarán los siguientes objetivos a abordar.

4.2. Cálculo de la velocidad

La velocidad representa la cantidad de puntos de historia que se pueden desarrollar en cada iteración. Las iteraciones que vamos a marcar tendrán una duración de dos semanas, lo que equivale a diez días laborales.

Dado que el proyecto será desarrollado por un único programador y suponiendo que un punto de historia es un día de trabajo ideal, se introduce además un factor de dedicación del 80 %, que tiene en cuenta posibles interrupciones o tareas no directamente relacionadas con el desarrollo.

Teniendo todos esos factores en cuenta, la velocidad estimada del proyecto es de 8 puntos de historia por iteración.

$$\begin{aligned}
 \text{Velocidad} &= \text{N}^{\circ} \text{ de programadores} \times \text{Días} \times \text{Factor de dedicación} \\
 &= 1 \times 10 \times 80 \% \\
 &= 8 \text{ PH}
 \end{aligned}$$

4.3. Historias de usuario

A continuación se expone en la Tabla 4.1 un listado con todas las historias de usuario y sus correspondientes puntos de historia que contiene la aplicación:

Cuadro 4.1: Historias de usuario

Código	Título	PH
HU1	Como cliente quiero poder crearme un perfil para poder usar la aplicación	2
HU2	Como usuario quiero poder editar mi perfil para modificar mis datos	1
HU3	Como cliente quiero poder eliminar mi perfil para eliminar mis datos de la aplicación	1
HU4	Como usuario quiero poder iniciar sesión en la aplicación para acceder a sus funcionalidades	1
HU5	Como usuario quiero poder cerrar sesión en la aplicación para que nadie más pueda acceder a mi cuenta	1
HU6	Como cliente quiero poder crear un grupo para tomar decisiones con mis amigos	4
HU7	Como cliente quiero poder abandonar un grupo para dejar de formar parte de él	1
HU8	Como cliente quiero poder añadir miembros a un grupo para añadirlos en la toma de decisiones	1
HU9	Como cliente quiero poder eliminar miembros de un grupo para que dejen de formar parte del grupo	1
HU10	Como cliente quiero poder recibir notificaciones respectivas a los grupos a los que pertenezca	3
HU11	Como cliente quiero poder iniciar una votación para tomar una decisión	5

HU12	Como cliente quiero asignarle un tiempo máximo a una votación para recibir los resultados en un tiempo limitado	2
HU13	Como cliente quiero poder editar una votación que haya creado para editar la decisión	1
HU14	Como cliente quiero poder eliminar una votación para cancelar una decisión	1
HU15	Como cliente quiero poder ver las votaciones activas en los grupos a los que pertenezco para poder votar en ellas	2
HU16	Como cliente quiero poder votar en las votaciones para tener derecho de elección	2
HU17	Como cliente quiero poder añadir opciones a una votación para poder votarlas posteriormente	3
HU18	Como cliente quiero poder elegir el tipo de votación para adaptarlo a mis necesidades	5
HU19	Como cliente quiero ver las decisiones pasadas para recordar las elecciones tomadas	2
HU20	Como cliente quiero ver la opción ganadora en una votación para conocer la decisión final	2
HU21	Como cliente quiero poder desempatar decisiones finalizadas	1
HU22	Como cliente quiero poder seleccionar decisiones predeterminadas para facilitar mi toma de decisiones	4
HU23	Como cliente quiero poder subir imágenes a la aplicación para usarlas en las opciones o como foto de perfil o grupo	1
HU24	Como administrador quiero poder crear otro administrador para que me ayude en las tareas de gestión	2
HU25	Como administrador quiero poder eliminar a otro administrador para revocar sus permisos de administración	1
HU26	Como administrador quiero poder modificar los datos de un usuario para solucionar problemas de gestión	1
HU27	Como administrador quiero poder modificar datos de un grupo para solucionar problemas de gestión	1
HU28	Como administrador quiero poder modificar datos de una votación para solucionar problemas de gestión	1

HU29	Como administrador quiero poder eliminar un usuario para solucionar problemas de gestión	1
HU30	Como administrador quiero poder eliminar un grupo para solucionar problemas de gestión	1
HU31	Como administrador quiero poder eliminar una votación para solucionar problemas de gestión	1

4.4. Planificación temporal

Este trabajo se inició el día 2 de febrero y será finalizada el día 3 de julio, por lo que tendrá una duración total de 21 semanas. Tal como se ha explicado anteriormente, las iteraciones tendrán una longitud de dos semanas, lo que da lugar a un total de 11 iteraciones a lo largo del proyecto.

Las cuatro primeras iteraciones estuvieron dedicadas exclusivamente a la elaboración de la documentación. No obstante, la documentación se continuará desarrollando de forma paralela a la implementación durante todo el proyecto.

A continuación, se describen las iteraciones:

- 1ª iteración (2 febrero - 13 febrero): Introducción → Se elaborarán la introducción y el estado del arte correspondientes al primer y segundo capítulo respectivamente de la documentación.
- 2ª iteración (16 febrero - 27 febrero): Requisitos → Se realizarán la especificación de requisitos correspondiente al tercer capítulo de la documentación.
- 3ª iteración (2 marzo - 13 marzo): Planificación → Se especificarán la planificación correspondiente al cuarto capítulo de la documentación.
- 4ª iteración (16 marzo - 27 marzo): Análisis → Se realizará la documentación relativa al quinto capítulo de la documentación.
- 5ª iteración (30 marzo - 10 abril): Creación de usuarios → Se desarrollará la fase de diseño y se implementará la creación de los perfiles de cliente y administrador, permitiendo editar y eliminar el perfil e iniciar y cerrar sesión en el sistema. Se completarán las historias de usuario HU1, HU2, HU3, HU4, HU5 y HU24.
- 6ª iteración (13 abril - 24 abril): Creación de grupos → Se implementará la creación de grupos, permitiendo añadir miembros y recibir notificaciones. Se desarrollarán las historias de usuario HU6, HU8 y HU10.

- 7ª iteración (27 abril - 8 mayo): Eliminación de miembros y creación de votaciones → Se podrá abandonar un grupo, eliminar un administrador y crear de votaciones. Se implemetarán las historias de usuario HU7, HU9, HU25 y HU11.
- 8ª iteración (11 mayo - 22 mayo): Subida de imágenes, opciones de administración y edición de votaciones → Se implementarán opciones de edición y eliminación de las votaciones, subida de imágenes y modificación y eliminación de usuarios y grupos por parte de los administradores. Se completarán las historias de usuario HU23, HU26, HU27, HU29, HU12, HU13, HU14 y HU17.
- 9ª iteración (25 mayo - 5 junio): Modificación de votaciones → Se habilitarán los tipos de votaciones y visualización y modificación de las mismas. Se desarrollarán las historias de usuario HU18, HU15 y HU28.
- 10ª iteración (8 junio - 19 junio): Votación, opciones, historial y administración → Se habilitará el voto, la creación de opciones y se permitirá consultar el historial. Además, se añadirán nuevas opciones de administración. Se llevarán a cabo las historias de usuario HU16, HU17, HU30 y HU19.
- 11ª iteración (22 junio - 3 julio): Historial, decisiones predeterminadas, desempate → Se permitirá consultar y desempatar la opción ganadora de las decisiones finalizadas, crear una decisión predeterminada y se añadirán opciones de administración. Se completarán las historias de usuario HU20, HU22, HU31 y HU21.

4.5. Diagrama de Gantt

A continuación, en la Figura 4.1 exponemos un diagrama de Gantt donde se puede observar la correspondencia entre las iteraciones y las historias de usuario.

4.6. Estimación de costes

Se va a calcular la estimación de costes, siguiendo un modelo de presupuesto desglosado en coceptos.

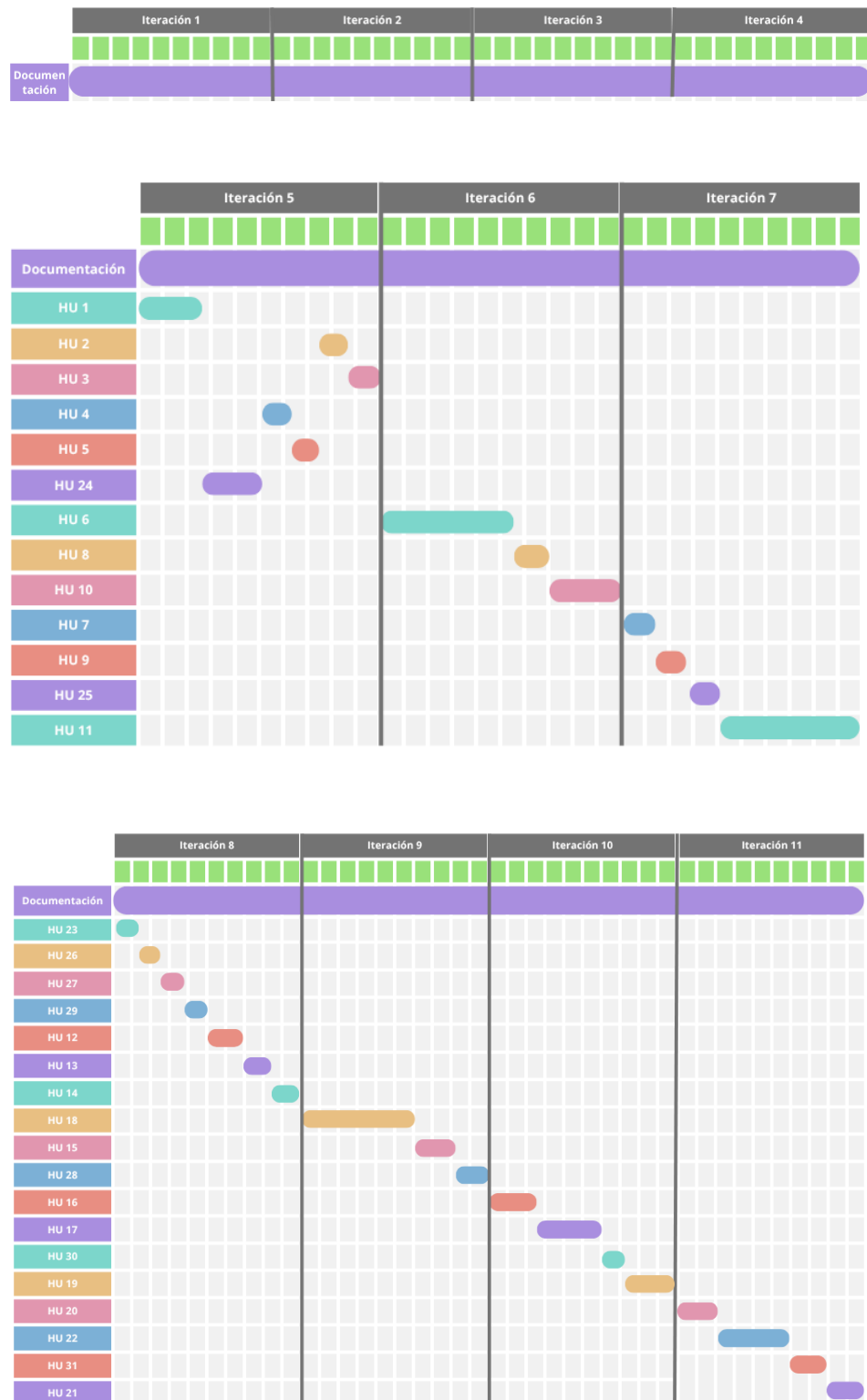


Figura 4.1: Diagrama de Gantt de las iteraciones

4.6.1. Honorarios

Para la estimación de este apartado, tal como se puede observar en la Tabla 4.2, se han tenido en cuenta distintos aspectos, que se desglosan a continuación.

Para el cálculo de la nómina, se ha tenido en cuenta el salario de un programador junior a jornada completa en España, que varía entre 15.000 € y 30.000 € anuales. En nuestros cálculos se utiliza el valor medio de 24.000 €, lo que corresponde a un salario mensual de 1.714,28 € para una jornada completa.

En este caso, se va a realizar un trabajo a media jornada, por lo tanto, se destinará la mitad del dinero de jornada completa. Dando en total un salario bruto mensual de 857,14 €.

Los empleados no dispondrán de ningún plus ni complementos salariales.

En lo respectivo a las deducciones salariales de un empleado en España, el porcentaje de cotización a la Seguridad social ronda el 6,45 %. Por otra parte, la cuota patronal para un contrato temporal se sitúa aproximadamente en el 32,6 %, mientras que el IRPF para personas solteras, sin hijos y con salario bruto medio-bajo se ha estimado en un 9 %.

En cuanto al seguro de vida, se ha comprobado que no es obligatorio; sin embargo, la mayoría de las empresas lo incluyen como beneficio adicional. Se ha comprobado que el rango mensual se sitúa entre 4 y 20 €, por lo que se utilizará un valor intermedio de 10 €.

La retención en especie es calculada mediante el coste de las distintas retenciones en especie, en este caso únicamente el seguro de vida, multiplicado por el porcentaje de IRPF. Dicho porcentaje se obtiene dividiendo el total del IRPF entre la base bruta sujeta a IRPF, que en este caso corresponde al salario base más el valor del seguro de vida.

Sobre el salario bruto mensual de 857,14 €, tras aplicar las deducciones correspondientes a la Seguridad Social y al IRPF, se obtiene una nómina mensual de 719,54 €. Sumando la cuota patronal, el coste total para la empresa asciende a 997,11 € mensuales.

Teniendo en cuenta que la empresa está compuesta por un único trabajador con un contrato de cinco meses de duración, el coste destinado a

honorarios asciende a 4.985,55 €.

Honorarios	Coste (€)
Salario Base	851,14
Complemento Puesto de Trabajo	0
Antigüedad	0
Plus Transporte	0
Plus Exclusividad	0
Salario Base Vac.	0
Vaca Puesto de Trabajo	0
Antigüedad Vac.	0
Vacac Plus Exclusi	0
Seguridad Social	54,90
R.E. Seguro Vida C	10
Cuota patronal	277,47
IRPF	77
Retención Especie	0,89
Neto	719,64
Total mensual	997,11
Total	4.985,55

Cuadro 4.2: Estimación de honorarios

4.6.2. Entidades y personal subcontratados

Este apartado no cuenta con presupuesto asignado, ya que todo el trabajo será realizado por el personal del propio proyecto.

4.6.3. Adquisición de equipamiento para las actividades previstas en el proyecto

Para la elaboración de bocetos, diagramas de Gantt, planificaciones o uso de entornos de desarrollo como Figma, Visual Studio Code o Jira, se emplearán herramientas gratuitas o licencias de estudiante, por lo cual no será necesario presupuesto en este apartado.

Asimismo, dadas las características del proyecto, el plan gratuito de Supabase se adapta completamente a las necesidades del sistema, por lo que no será necesario contratar un servidor de pago.

En cuanto al coste de los dispositivos electrónicos destinados al desarrollo

del proyecto, se ha estimado que el ordenador utilizado consta de un valor aproximado de 500 € y dispone de una vida útil de cuatro años. Considerando que el proyecto tendrá una duración de cinco meses, el coste que supondrá es de 52,08 €.

En total, el presupuesto para adquisición de equipamiento para las actividades previstas en el proyecto es de 52,08 €.

4.6.4. Promoción y difusión del proyecto

Dado que se trata de un proyecto académico que no cuenta con un ámbito comercial posterior, no se destinará presupuesto a actividades de promoción o difusión.

4.6.5. Costes indirectos

Puesto que el proyecto será desarrollado mediante teletrabajo, se incluye un apartado de consumibles, para suplir los gastos relativos a aspectos como la luz, conexión a internet o climatización. Tras consultar el precio medio de estos servicios en España, se ha estimado un gasto mensual de 30 €. Por tanto, para los cinco meses de duración del proyecto se asigna un total de 150 €.

4.6.6. Total

Teniendo en cuenta todos los apartados descritos anteriormente, el presupuesto total estimado para la realización del proyecto asciende a 5.187,63 €, tal como se aprecia en la Tabla 4.3.

Concepto	Coste (€)
Honorarios	4.985,55
Entidades y personal subcontratados	0
Equipamiento	52,08
Promoción y difusión	0
Costes indirectos	150
Total	5.187,63

Cuadro 4.3: Estimación de costes del proyecto

Capítulo 5

Análisis

5.1. Product Backlog

En esta sección, en la Tabla 5.1, se ordenan las historias de usuario por prioridad, teniendo en cuenta las dependencias existentes entre las mismas. Se ha optado por una ordenación numérica, de mayor a menor prioridad, con el objetivo de evitar cuellos de botella.

Cuadro 5.1: Orden del product backlog

Código	Título	Prioridad
HU1	Como cliente quiero poder crearme un perfil para poder usar la aplicación	1
HU24	Como administrador quiero poder crear otro administrador para que me ayude en las tareas de gestión	2
HU4	Como usuario quiero poder iniciar sesión en la aplicación para acceder a sus funcionalidades	3
HU5	Como usuario quiero poder cerrar sesión en la aplicación para que nadie más pueda acceder a mi cuenta	4
HU2	Como usuario quiero poder editar mi perfil para modificar mis datos	5
HU3	Como cliente quiero poder eliminar mi perfil para eliminar mis datos de la aplicación	6
HU6	Como cliente quiero poder crear un grupo para tomar decisiones con mis amigos	7
HU8	Como cliente quiero poder añadir miembros a un grupo para añadirlos en la toma de decisiones	8

HU10	Como cliente quiero poder recibir notificaciones respectivas a los grupos a los que pertenezca	9
HU25	Como administrador quiero poder eliminar a otro administrador para revocar sus permisos de administración	10
HU7	Como cliente quiero poder abandonar un grupo para dejar de formar parte de él	11
HU9	Como cliente quiero poder eliminar miembros de un grupo para que dejen de formar parte del grupo	12
HU11	Como cliente quiero poder iniciar una votación para tomar una decisión	13
HU23	Como cliente quiero poder subir imágenes a la aplicación para usarlas en las opciones o como foto de perfil o grupo	14
HU26	Como administrador quiero poder modificar los datos de un usuario para solucionar problemas de gestión	15
HU27	Como administrador quiero poder modificar datos de un grupo para solucionar problemas de gestión	16
HU29	Como administrador quiero poder eliminar un usuario para solucionar problemas de gestión	17
HU12	Como cliente quiero asignarle un tiempo máximo a una votación para recibir los resultados en un tiempo limitado	18
HU13	Como cliente quiero poder editar una votación que haya creado para editar la decisión	19
HU14	Como cliente quiero poder eliminar una votación para cancelar una decisión	20
HU18	Como cliente quiero poder elegir el tipo de votación para adaptarlo a mis necesidades	21
HU15	Como cliente quiero poder ver las votaciones activas en los grupos a los que pertenezco para poder votar en ellas	22
HU28	Como administrador quiero poder modificar datos de una votación para solucionar problemas de gestión	23
HU17	Como cliente quiero poder añadir opciones a una votación para poder votarlas posteriormente	24
HU16	Como cliente quiero poder votar en las votaciones para tener derecho de elección	25
HU30	Como administrador quiero poder eliminar un grupo para solucionar problemas de gestión	26

HU19	Como cliente quiero ver las decisiones pasadas para recordar las elecciones tomadas	27
HU20	Como cliente quiero ver la opción ganadora en una votación para conocer la decisión final	28
HU22	Como cliente quiero poder seleccionar decisiones predeterminadas para facilitar mi toma de decisiones	29
HU31	Como administrador quiero poder eliminar una votación para solucionar problemas de gestión	30
HU21	Como cliente quiero poder desempatar decisiones finalizadas	31

5.2. Tarjetas de usuario

A continuación, exponemos las distintas historias de usuario desglosadas en tarjetas de usuario.

HU1: Como cliente quiero poder crearme un perfil para poder usar la aplicación

Prioridad:	1
Estimación:	2 PH
RF Relacionados:	REQ1
Categoría:	Gestión de cuenta

Descripción Detallada:

Permitirá crear un nuevo cliente en el sistema.

Tareas:

- Guardar el nuevo usuario en el sistema.
- Validar los datos introducidos.
- Crear una interfaz de creación de usuario.

Pruebas de Validación:

1. El sistema no permitirá el registro de usuarios con un correo electrónico ya existente en la base de datos.
2. El sistema validará que el correo electrónico introducido cumple el formato estándar. Siendo el formato estándar:
 - Contiene un único arroba (@), el cual no podrá ser ni el primer ni el último carácter.
 - Presenta un punto después del arroba (@).
 - Entre el arroba (@) y el punto debe aparecer al menos un carácter.
 - Después del último punto hay al menos dos caracteres.
 - No contiene espacios en blanco.
3. El nombre de usuario no podrá estar repetido en el sistema y no podrá contener el carácter arroba (@).
4. La contraseña introducida deberá contener al menos 8 caracteres, una letra mayúscula, una letra minúscula, un número y un carácter especial.
5. El campo de contraseña debe poder estar oculto mientras el usuario escribe.
6. El campo de contraseña y confirmación deben coincidir.
7. El usuario deberá confirmar su cuenta mediante el correo electrónico.
8. La contraseña debe guardarse en el sistema encriptada.
9. El sistema mostrará mensajes de error claros indicando el motivo por el cual no puede crearse el usuario.
10. Todos los campos obligatorios deberán estar rellenos.

HU2: Como usuario quiero poder editar mi perfil para modificar mis datos

Prioridad:	5
Estimación:	1 PH
RF Relacionados:	REQ2
Categoría:	Gestión de cuenta

Descripción Detallada:

El cliente podrá modificar todos sus datos personales introducidos en el sistema, teniendo estos nuevos datos que cumplir las mismas restricciones que se comprueban en el registro.

Tareas:

- Crear la interfaz de modificación.
- Validar los datos introducidos.
- Modificar los datos guardados en la base de datos.

Pruebas de Validación:

1. El sistema no permitirá cambiar el correo electrónico a uno ya existente en la base de datos.
2. El sistema validará que el correo electrónico introducido cumple el formato estándar. Siendo el formato estándar:
 - Contiene un único arroba (@), el cual no podrá ser ni el primer ni el último carácter.
 - Presenta un punto después del arroba (@).
 - Entre el arroba (@) y el punto debe aparecer al menos un carácter.
 - Después del último punto hay al menos dos caracteres.
 - No contiene espacios en blanco.
3. El nombre de usuario no podrá estar repetido en el sistema y no podrá contener el carácter arroba (@).
4. La contraseña introducida deberá contener al menos 8 caracteres, una letra mayúscula, una letra minúscula, un número y un carácter especial.
5. El campo de contraseña debe poder estar oculto mientras el usuario escribe.
6. El campo de contraseña y confirmación deben coincidir.
7. El usuario deberá confirmar su nuevo correo electrónico.
8. La contraseña debe guardarse en el sistema encriptada.
9. El sistema mostrará mensajes de error claros indicando que campos del usuario no han podido modificarse.

HU3: Como cliente quiero poder eliminar mi perfil para eliminar mis datos de la aplicación

Prioridad:	6
Estimación:	1 PH
RF Relacionados:	REQ3
Categoría:	Gestión de cuenta

Descripción Detallada:

Permitirá que un usuario borre su cuenta del sistema.

Tareas:

- Crear la interfaz con botón de eliminación.
- Eliminar la cuenta y los datos del usuario de los grupos y votaciones de la base de datos.

Pruebas de Validación:

1. La cuenta se borra del sistema.
2. Se elimina la cuenta de los grupos a los que pertenezca y las decisiones que haya creado.
3. Al borrar la cuenta se redirige a la pantalla de inicio de sesión.

HU4: Como usuario quiero poder iniciar sesión en la aplicación para acceder a sus funcionalidades

Prioridad:	3
Estimación:	1 PH
RF Relacionados:	REQ24
Categoría:	Gestión de cuenta

Descripción Detallada:

Permitirá a un usuario entrar en la aplicación con los datos de la cuenta que tiene asociada en el sistema.

Tareas:

- Crear la interfaz de inicio de sesión.
- Validar credenciales con la base de datos.

Pruebas de Validación:

1. El sistema permite el acceso con credenciales válidas.
2. El sistema mostrará un error si los datos introducidos son incorrectos.
3. Se comprobará que todos los datos están rellenos.
4. Se podrá ocultar el campo de contraseña mientras se escribe.
5. Se mantendrá la sesión activa mientras se usa la aplicación.

HU5: Como usuario quiero poder cerrar sesión en la aplicación para que nadie más pueda acceder a mi cuenta

Prioridad:	4
Estimación:	1 PH
RF Relacionados:	REQ25
Categoría:	Gestión de cuenta

Descripción Detallada:

El cliente podrá cerrar en un dispositivo la sesión asociada a su cuenta del sistema.

Tareas:

- Crear la interfaz con botón de cierre de sesión.
- Borrar el token de sesión, el historial, los listeners y el estado.

Pruebas de Validación:

1. Se cierra la sesión del dispositivo, redirigiendo a la pantalla de inicio de sesión.
2. Se borran los datos de sesión guardados en el dispositivo.

HU6: Como cliente quiero poder crear un grupo para tomar decisiones con mis amigos

Prioridad:	7
Estimación:	4 PH
RF Relacionados:	REQ10
Categoría:	Gestión de grupos

Descripción Detallada:

Permitirá a un cliente crear un grupo con el objetivo de poder hacer votaciones en común con otros clientes.

Tareas:

- Implementar la interfaz de creación de grupo.
- Guardar el nuevo grupo en el sistema.
- Añadir automáticamente en la base de datos al cliente que ha creado el grupo como miembro.
- Crear la interfaz con la lista de grupos a los que perteneces.

Pruebas de Validación:

1. El grupo se ha guardado en la base de datos.
2. El creador está registrado como miembro del grupo.

HU7: Como cliente quiero poder abandonar un grupo para dejar de formar parte de él

Prioridad:	11
Estimación:	1 PH
RF Relacionados:	REQ13
Categoría:	Gestión de grupos

Descripción Detallada:

Un cliente podrá abandonar cualquiera de los grupos de los cuales forme parte. Una vez lo abandone, el grupo desaparecerá de la interfaz de grupos a los que pertenece.

Tareas:

- Borrar la relación de pertenencia del cliente al grupo en la base de datos.
- Crear el botón de salir del grupo en la interfaz del grupo.
- Actualizar la interfaz con la lista de grupos a los que pertenece.

Pruebas de Validación:

1. Se ha eliminado en la base de datos la relación de pertenencia entre el cliente y el grupo.
2. Si el usuario es el último miembro, el grupo es eliminado automáticamente por el sistema.

HU8: Como cliente quiero poder añadir miembros a un grupo para añadirlos en la toma de decisiones

Prioridad:	8
Estimación:	1 PH
RF Relacionados:	REQ11
Categoría:	Gestión de grupos

Descripción Detallada:

Un cliente podrá añadir miembros a los grupos a los que pertenezca.

Tareas:

- Se actualiza la interfaz del grupo con el número y nombre de los miembros.
- Se actualiza la interfaz de la lista de grupos al cliente.
- Se guarda la relación entre el cliente y el grupo en el sistema.
- Crear la interfaz de añadir nuevos miembros.

Pruebas de Validación:

1. Si se intenta añadir a un cliente que ya pertenece al grupo, se generará un mensaje de error.
2. Se guarda en la base de datos la asociación entre el nuevo miembro y el grupo.

HU9: Como cliente quiero poder eliminar miembros de un grupo para que dejen de formar parte del grupo

Prioridad:	12
Estimación:	1 PH
RF Relacionados:	REQ12
Categoría:	Gestión de grupos

Descripción Detallada:

Un miembro de un grupo podrá eliminar a otro miembro del mismo. Al usuario eliminado dejará de salirle el grupo en la lista de grupos.

Tareas:

- Borrar la relación de pertenencia del cliente al grupo en la base de datos.
- Crear el botón de eliminar al cliente del grupo en la interfaz del grupo.
- Actualizar la interfaz del cliente afectado con la lista de grupos a los que pertenece.
- Actualizar la interfaz con la lista de miembros del grupo.

Pruebas de Validación:

1. Se elimina en la base de datos la relación de pertenencia entre el cliente y el grupo.
2. Se actualiza en la interfaz la lista de miembros del grupo.
3. Se actualiza la interfaz del usuario eliminado.

HU10: Como cliente quiero poder recibir notificaciones respectivas a los grupos a los que pertenezca

Prioridad:	9
Estimación:	3 PH
RF Relacionados:	REQ15, REQ19, REQ20 y REQ21
Categoría:	Gestión de notificaciones

Descripción Detallada:

La aplicación notificará al usuario sobre los eventos relevantes en sus grupos: su incorporación o salida de un grupo, y cualquier actualización en el estado de las decisiones grupales.

Tareas:

- Implementar la interfaz de lectura de notificaciones.
- Guardar los datos de la notificación en la base de datos.
- Implementar el sistema de creación de notificaciones automáticas.

Pruebas de Validación:

1. Ante un cambio en los miembros del grupo se notificará al usuario afectado.
2. Todos los miembros del grupo serán notificados al cambiar el estado de una notificación.
3. Una notificación será borrada del sistema cuando el usuario al que va dirigida lo indique.

HU11: Como cliente quiero poder iniciar una votación para tomar una decisión

Prioridad:	13
Estimación:	5 PH
RF Relacionados:	REQ15, REQ19, REQ21 y REQ23
Categoría:	Gestión de votaciones

Descripción Detallada:

Un cliente podrá crear una votación individual o en un grupo del que forma parte. La votación individual se creará fuera de todos los grupos y solo podrá visualizarla y votar en ella el creador. Por otra parte, las votaciones creadas en un grupo serán públicas para todos los miembros del grupo. La votación contará con un título, una lista de opciones y el tipo de votación.

Tareas:

- Implementar la interfaz de creación de votación.
- Actualizar la interfaz del grupo con las votaciones activas.
- Guardar los datos de la votación en la base de datos.
- Implementar el sistema de extracción de información.

Pruebas de Validación:

1. Se guardan los datos de la votación en el sistema asociados al grupo del que forman parte o al cliente que la ha creado.
2. Se guarda junto con la votación el ID del usuario que la creó.
3. Se valida que la votación tenga al menos un título y dos opciones antes de crearla.
4. Se actualiza la interfaz de votaciones activas en el grupo en el que se haya creado.

HU12: Como cliente quiero asignarle un tiempo máximo a una votación para recibir los resultados en un tiempo limitado

Prioridad:	18
Estimación:	2 PH
RF Relacionados:	REQ18 y REQ20
Categoría:	Gestión de votaciones

Descripción Detallada:

El creador de una votación en un grupo podrá asignarle un tiempo máximo para que el resto de usuarios del grupo voten. Pasado ese tiempo, se cerrará automáticamente la votación y se anunciará el ganador.

Tareas:

- Añadir la opción de tiempo máximo en la interfaz de creación de votación.
- Guardar la hora de finalización en el sistema.

Pruebas de Validación:

1. Pasado el tiempo máximo, la votación se cierra automáticamente.
2. La hora de finalización se guarda en el sistema asociada a la votación.

HU13: Como cliente quiero poder editar una votación que haya creado para editar la decisión

Prioridad:	19
Estimación:	1 PH
RF Relacionados:	REQ15
Categoría:	Gestión de votaciones

Descripción Detallada:

Un creador de una votación podrá modificar cualquier dato relativo a ella antes de que cualquier usuario haya votado. Una vez que se haya registrado algún voto, solo podrá modificarse el tiempo máximo para la finalización de la votación.

Tareas:

- Crear la interfaz de edición de la votación.
- Cambiar los datos de la votación en la base de datos.
- Modificar la interfaz con los nuevos datos de la votación.

Pruebas de Validación:

1. La votación se modifica en la base de datos.
2. Se actualiza la interfaz con los nuevos datos de la votación.
3. Se respetan las restricciones de edición de acuerdo al número de votos registrados.
4. Solo se permite editar la votación al creador de la misma.

HU14: Como cliente quiero poder eliminar una votación para cancelar una decisión

Prioridad:	20
Estimación:	1 PH
RF Relacionados:	REQ16
Categoría:	Gestión de votaciones

Descripción Detallada:

Un creador de una votación podrá eliminarla antes de que finalice, borrando todos los datos relativos a ella.

Tareas:

- Crear en la interfaz el botón de borrado de votación.
- Eliminar los datos relativos a la votación en la base de datos.

Pruebas de Validación:

1. La votación y todas sus opciones y votos asociados se borran de la base de datos.
2. Solo se permite eliminar la votación el creador de la misma.

HU15: Como cliente quiero poder ver las votaciones activas en los grupos a los que pertenezco para poder votar en ellas

Prioridad:	22
Estimación:	2 PH
RF Relacionados:	REQ15
Categoría:	Gestión de votaciones

Descripción Detallada:

Un cliente podrá visualizar las votaciones activas de los grupos a los que pertenece.

Tareas:

- Mostrar las votaciones que no hayan finalizado en la interfaz del grupo.

Pruebas de Validación:

1. Se muestran todas las votaciones activas del grupo en cuestión.
2. No se muestran votaciones finalizadas o pertenecientes a otros grupos.

HU16: Como cliente quiero poder votar en las votaciones para tener derecho de elección

Prioridad:	25
Estimación:	2 PH
RF Relacionados:	REQ14 y REQ20
Categoría:	Gestión de votaciones

Descripción Detallada:

Un cliente perteneciente a un grupo podrá votar en las votaciones activas de los grupos a los que pertenece. Si lo hace, su voto se guardará y se tendrá en cuenta a la hora de elegir la opción ganadora.

Tareas:

- Implementar la interfaz de registro de voto.
- Guardar el voto en la base de datos.

Pruebas de Validación:

1. Un usuario solo podrá votar una vez por votación.
2. Solo podrán votar los usuarios pertenecientes al grupo.
3. Cuando se hayan registrado los votos de todos los miembros, se cerrará la votación automáticamente, aunque aún no haya pasado el tiempo máximo.
4. El voto será almacenado en la base de datos relacionando la opción votada, el cliente y la votación.

HU17: Como cliente quiero poder añadir opciones a una votación para poder votarlas posteriormente

Prioridad:	24
Estimación:	3 PH
RF Relacionados:	REQ15, REQ17 y REQ21
Categoría:	Gestión de votaciones

Descripción Detallada:

Si el creador de la votación lo permite los miembros de un grupo podrán sugerir opciones durante el proceso de creación. Esas opciones tras ser validadas por el creador o excederse el tiempo máximo de sugerencia serán añadidas como opciones que podrán votar cuando se publique la votación.

Tareas:

- Implementar la interfaz de creación de opciones.
- Guardar la opción en el sistema.
- Implementar el sistema de extracción de información.

Pruebas de Validación:

1. La opción se añade a la base de datos relacionada con la votación.
2. Solo los usuarios que son miembros de un grupo podrán sugerir opciones.
3. La interfaz de opciones de la votación se actualiza mostrando la nueva opción creada.

HU18: Como cliente quiero poder elegir el tipo de votación para adaptarlo a mis necesidades

Prioridad:	21
Estimación:	5 PH
RF Relacionados:	REQ19
Categoría:	Gestión de votaciones

Descripción Detallada:

El creador de una votación podrá elegir el tipo de elección que se llevará a cabo seleccionandola de una lista predeterminada.

Tareas:

- Crear en la interfaz de creación de votación el desplegable para elegir el tipo.
- El tipo seleccionado se guarda en la base de datos asociado a la votación.
- Implementar el tipo de elección por ruleta.
- Implementar el tipo de elección por voto único.
- Implementar el tipo de elección por voto múltiple.
- Implementar el tipo de elección por ranking.

Pruebas de Validación:

1. El tipo de elección se guarda en la base de datos.
2. Al comenzar la votación, el sistema de votos es el elegido.
3. El ganador es elegido siguiendo los parámetros establecidos por el tipo de votación seleccionada.

HU19: Como cliente quiero ver las decisiones pasadas para recordar las elecciones tomadas

Prioridad:	27
Estimación:	2 PH
RF Relacionados:	REQ5
Categoría:	Gestión de votaciones

Descripción Detallada:

Los clientes podrán acceder a un historial que muestre todas las votaciones de las que formaron parte.

Tareas:

- Crear la interfaz que muestre las votaciones finalizadas.

Pruebas de Validación:

1. Se muestran todas las votaciones finalizadas en las que participó el cliente.

HU20: Como cliente quiero ver la opción ganadora en una votación para conocer la decisión final

Prioridad:	28
Estimación:	2 PH
RF Relacionados:	REQ20
Categoría:	Gestión de votaciones

Descripción Detallada:

Los participantes de una votación, aunque no hayan ejercido su derecho al voto, podrán ver el resultado final de la elección.

Tareas:

- Guardar en la base de datos el número de votos asociado a una opción.
- Mostrar en la interfaz de historial la nueva votación finalizada.
- Avisar al usuario de la opción ganadora.

Pruebas de Validación:

1. La opción ganadora se guarda correctamente en el sistema asociada a la votación.
2. Solo se avisa de la opción ganadora a los participantes de la votación.

HU21: Como cliente quiero poder desempatar decisiones finalizadas

Prioridad:	31
Estimación:	1 PH
RF Relacionados:	REQ22
Categoría:	Gestión de votaciones

Descripción Detallada:

Si al finalizar una decisión existe un empate en el primer lugar, el creador tendrá la opción de desempatar. Para ello, podrá elegir entre aplicar un voto adicional exclusivo del creador o por medio de una ruleta aleatoria.

Tareas:

- Añadir el botón de desempate en el historial.
- Generar un menú intermedio donde seleccionar el modo de desempate.
- Añadir un voto extra a la opción ganadora.

Pruebas de Validación:

1. Cuando una decisión cuenta con un empate en el primer lugar, en el historial de la decisión le aparece al creador de la decisión el botón de desempate.
2. El botón de desempate redirige a una pantalla intermedia donde seleccionar el modo de desempate.
3. Al desempatar se guardán los nuevos votos en la base de datos y se actualiza la interfaz del historial.

HU22: Como cliente quiero poder seleccionar decisiones pre-determinadas para facilitar mi toma de decisiones

Prioridad:	29
Estimación:	4 PH
RF Relacionados:	REQ21
Categoría:	Gestión de votaciones

Descripción Detallada:

Los clientes podrán desde un grupo o desde su cuenta personal, crear una votación usando como plantilla las elecciones predeterminadas que ofrece la aplicación.

Tareas:

- Añadir elecciones predeterminadas en la base de datos.
- Añadir las elecciones predeterminadas en la interfaz de la aplicación.
- Implementar el sistema de extracción de información.

Pruebas de Validación:

1. Cada vez que alguien seleccione una decisión predeterminada se creará una nueva votación partiendo de esa plantilla.
2. Habrá al menos una decisión predeterminada en el sistema.

HU23: Como cliente quiero poder subir imágenes a la aplicación para usarlas en las opciones o como foto de perfil o grupo

Prioridad:	14
Estimación:	1 PH
RF Relacionados:	REQ4
Categoría:	Gestión de imágenes

Descripción Detallada:

La aplicación soportará la subida de imágenes, que podrán usarse como foto de perfil, foto de grupo o en las opciones.

Tareas:

- Guardar la imagen en el sistema.
- Implementar en las interfaces necesarias el botón para subir una imagen.

Pruebas de Validación:

1. La imagen se guarda en el sistema asociada al dato correspondiente.
2. La imagen se muestra en las interfaces.

HU24: Como administrador quiero poder crear otro administrador para que me ayude en las tareas de gestión

Prioridad:	2
Estimación:	2 PH
RF Relacionados:	REQ7
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá crear otro administrador del sistema. Para ello, tendrá que cambiar el rol asociado de otro usuario de cliente a administrador.

Tareas:

- Crear la interfaz de modificación de un usuario.
- Cambiar el rol del usuario en la base de datos.

Pruebas de Validación:

1. El rol del usuario se cambia en la base de datos.
2. El nuevo administrador accede al sistema y dispone de opciones de administración.

HU25: Como administrador quiero poder eliminar a otro administrador para revocar sus permisos de administración

Prioridad:	10
Estimación:	1 PH
RF Relacionados:	REQ8
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá modificar el rol a otro administrador y convertirlo en un usuario cliente.

Tareas:

- Crear la interfaz de modificación de un usuario.
- Cambiar el rol del usuario en la base de datos.

Pruebas de Validación:

1. El rol del usuario se cambia en la base de datos.
2. El antiguo administrador accede al sistema y solo dispone de las opciones de cliente.

HU26: Como administrador quiero poder modificar los datos de un usuario para solucionar problemas de gestión

Prioridad:	15
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá cambiar los datos de todos los usuarios del sistema.

Tareas:

- Crear la interfaz de edición de usuarios.
- Validar los datos introducidos.
- Modificar los datos en la base de datos.

Pruebas de Validación:

1. El sistema no permitirá cambiar el correo electrónico a uno ya existente en la base de datos.
2. El sistema validará que el correo electrónico introducido cumple el formato estándar. Siendo el formato estándar:
 - Contiene un único arroba (@), el cual no podrá ser ni el primer ni el último carácter.
 - Presenta un punto después del arroba (@).
 - Entre el arroba (@) y el punto debe aparecer al menos un carácter.
 - Después del último punto hay al menos dos caracteres.
 - No contiene espacios en blanco.
3. El nombre de usuario no podrá estar repetido en el sistema y no podrá contener el carácter arroba (@).
4. La contraseña introducida deberá contener al menos 8 caracteres, una letra mayúscula, una letra minúscula, un número y un carácter especial.
5. El campo de contraseña debe poder estar oculto mientras el usuario escribe.
6. El campo de contraseña y confirmación deben coincidir.
7. El usuario deberá confirmar su nuevo correo electrónico.
8. La contraseña debe guardarse en el sistema encriptada.
9. El sistema mostrará mensajes de error claros indicando que campos del usuario no han podido modificarse.

HU27: Como administrador quiero poder modificar datos de un grupo para solucionar problemas de gestión

Prioridad:	16
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá modificar los datos y lista de miembros asociados a un grupo.

Tareas:

- Crear la interfaz de edición de grupos.
- Validar los datos introducidos.
- Modificar los datos en la base de datos.

Pruebas de Validación:

1. Si se intenta añadir a un cliente que ya pertenece al grupo aparecerá un mensaje de error.
2. Se actualiza en la base de datos los cambios realizados en el grupo.
3. Si se elimina el último miembro que formaba parte del grupo se elimina automáticamente el grupo.

HU28: Como administrador quiero poder modificar datos de una votación para solucionar problemas de gestión

Prioridad:	23
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá modificar los datos asociados a una votación.

Tareas:

- Crear la interfaz de modificación de votaciones.
- Actualizar los datos en la base de datos.

Pruebas de Validación:

1. Se actualizan los datos en la base de datos.
2. Se actualiza la información en la interfaz del grupo o cliente correspondiente.

HU29: Como administrador quiero poder eliminar un usuario para solucionar problemas de gestión

Prioridad:	17
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

El administrador podrá eliminar la cuenta de un usuario del sistema, borrando todos sus datos.

Tareas:

- Añadir en la interfaz un botón de eliminación del usuario.
- Borrar los datos del usuario de los grupos y votaciones.

Pruebas de Validación:

1. Se eliminan los datos del usuario correspondiente en la base de datos.
2. Se elimina la cuenta de los grupos a los que pertenezca y las decisiones que haya creado.
3. Se actualizan las interfaces dejando de mostrar a ese usuario.

HU30: Como administrador quiero poder eliminar un grupo para solucionar problemas de gestión

Prioridad:	26
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá eliminar un grupo.

Tareas:

- Crear el botón de borrado del grupo.
- Borrar el grupo de la base de datos.

Pruebas de Validación:

1. Se elimina el grupo de la base de datos.
2. Se actualiza la interfaz dejando de aparecer ese grupo.

HU31: Como administrador quiero poder eliminar una votación para solucionar problemas de gestión

Prioridad:	30
Estimación:	1 PH
RF Relacionados:	REQ9
Categoría:	Gestión del administrador

Descripción Detallada:

Un administrador podrá eliminar una votación asociada a un grupo o a un cliente.

Tareas:

- Crear el botón de eliminar en la interfaz.
- Borrar los datos de la votación y los asociados a los votos de la base de datos.

Pruebas de Validación:

1. Se eliminan los datos de la votación y los asociados a los votos de la base de datos.
2. Se actualiza la interfaz dejando de aparecer la votación.

5.3. Diagrama de clases de análisis

En la Figura 5.1 se muestra un primer diagrama de clases que incluye las clases principales del sistema junto con sus atributos asociados.

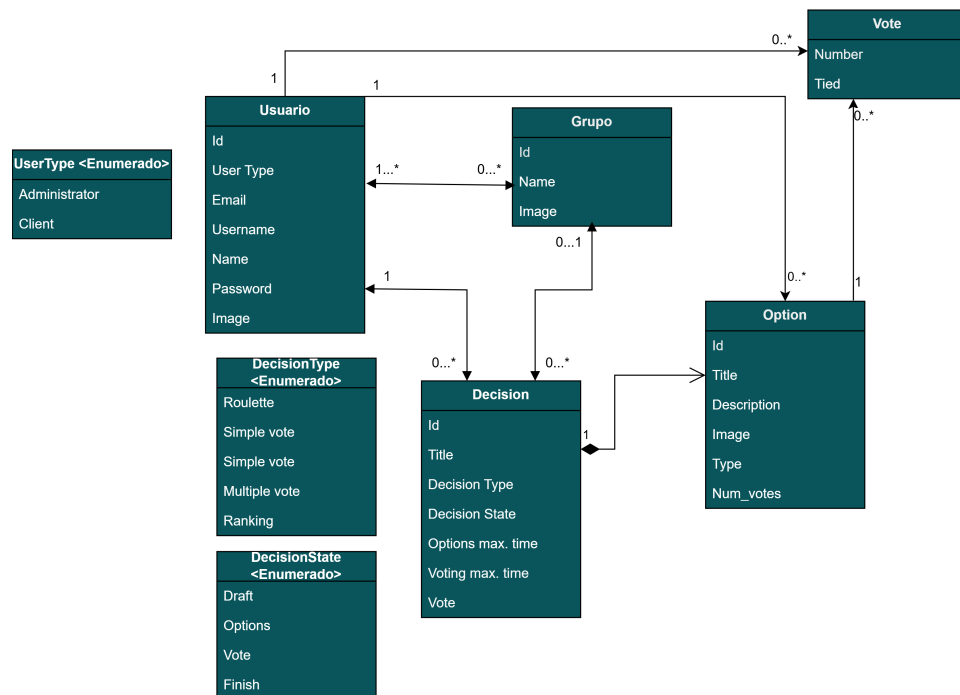


Figura 5.1: Diagrama de clases de análisis

Capítulo 6

Diseño

6.1. Arquitectura del sistema

El proyecto no se basa en un único estilo arquitectónico, como se observa en la Figura 6.1, sino en la combinación de varios enfoques complementarios. A nivel general, sigue una arquitectura cliente-servidor, donde la aplicación desarrollada en *Flutter* actúa como cliente y la plataforma *Supabase* proporciona los servicios de backend.

Aunque Supabase se percibe como una única plataforma desde el punto de vista del desarrollador, internamente el backend adopta un enfoque Backend as a Service (BaaS), delegando la gestión de distintos servicios especializados. Entre ellos se encuentran PostgreSQL para la persistencia de datos, Auth para la autenticación y gestión de usuarios, Storage para el almacenamiento de imágenes, Realtime para la sincronización de información en tiempo real y Edge Functions para la ejecución de lógica personalizada.

Las Edge Functions permiten encapsular operaciones que requieren ejecutarse en el servidor, como tareas administrativas o lógica de negocio que no debe residir en el cliente. Estas funciones pueden activarse de dos formas: mediante peticiones HTTPS realizadas por la aplicación cliente o mediante tareas programadas (**cron jobs**) configuradas en *Supabase*. En el segundo caso, las funciones se ejecutan automáticamente en intervalos definidos para realizar operaciones periódicas, como la comprobación de fechas de finalización de votaciones. Por este motivo, las Edge Functions siguen un modelo de ejecución serverless, en el que la infraestructura y el escalado son gestionados automáticamente por *Supabase*, sin necesidad de administrar servidores propios.

En el lado cliente se sigue una arquitectura a capas, diferenciando la capa de presentación, encargada de la interfaz de usuario, la capa de lógica de aplicación, basada en Riverpod, responsable de la gestión del estado de forma reactiva y de la implementación de las reglas de negocio, y la capa de acceso a datos, encargada de la comunicación con los servicios de *Supabase*. Riverpod permite desacoplar la lógica de la interfaz, facilitando una arquitectura reactiva en la que los cambios en el estado se propagan automáticamente a los componentes de la interfaz de usuario. Esta separación permite una clara división de responsabilidades y mejora la mantenibilidad y escalabilidad del código.

Finalmente, el sistema incorpora una arquitectura orientada a eventos mediante el servicio Realtime de *Supabase*. Gracias a este mecanismo, los clientes reciben notificaciones cuando se producen cambios relevantes en la base de datos, permitiendo actualizar automáticamente la información de grupos y votaciones sin necesidad de realizar consultas periódicas.

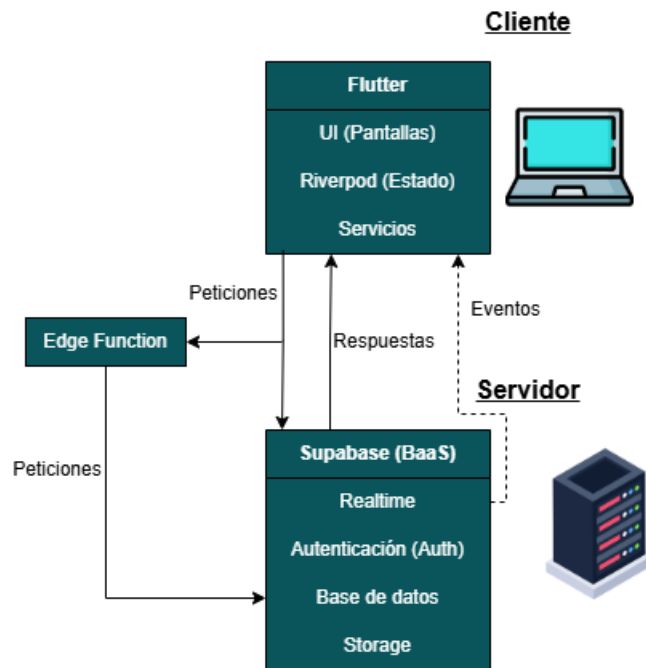


Figura 6.1: Diagrama de la arquitectura del sistema [7] [8]

6.2. API de la aplicación

La API sigue un diseño REST orientado a recursos. Las entidades principales del sistema (usuarios, grupos, decisiones, opciones y notificaciones) se exponen como endpoints, mientras que las relaciones intermedias (membresías a grupos, votos, última opción votada y notificaciones) se integran dentro de rutas jerárquicas que representan acciones del usuario. A continuación, se exponen los distintos endpoints.

Usuarios

Método	Ruta	Descripción
POST	/usuarios	Crear usuario
GET	/usuarios	Obtener lista de usuarios
GET	/usuarios/{id}	Obtener usuario
PUT	/usuarios/{id}	Actualizar usuario
DELETE	/usuarios/{id}	Eliminar usuario

Cuadro 6.1: API de usuarios

Notificaciones

Método	Ruta	Descripción
GET	/usuarios/{id}/notificaciones	Obtener notificaciones del usuario
DELETE	/notificaciones/{id}	Eliminar una notificación

Cuadro 6.2: API de notificaciones

Grupos

Método	Ruta	Descripción
POST	/grupos	Crear grupo
GET	/grupos	Obtener lista de grupos
GET	/grupos/{id}	Obtener grupo
PUT	/grupos/{id}	Actualizar grupo
DELETE	/grupos/{id}	Eliminar grupo

Cuadro 6.3: API de grupos

Miembros de grupos

Método	Ruta	Descripción
POST	/grupos/{id}/miembros	Añadir usuario al grupo
GET	/grupos/{id}/miembros	Obtener miembros del grupo
DELETE	/grupos/{id}/miembros/{userId}	Eliminar usuario del grupo

Cuadro 6.4: API de miembros de grupo

Decisiones

Método	Ruta	Descripción
POST	/decisiones	Crear decisión
GET	/decisiones/{id}	Obtener decisión
PUT	/decisiones/{id}	Actualizar decisión
DELETE	/decisiones/{id}	Eliminar decisión

Cuadro 6.5: API de decisiones

Opciones

Método	Ruta	Descripción
POST	/decisiones/{id}/opciones	Crear opción
GET	/decisiones/{id}/opciones	Obtener opciones de una decisión
GET	/opciones/{id}	Obtener opción
PUT	/opciones/{id}	Actualizar opción
DELETE	/opciones/{id}	Eliminar opción

Cuadro 6.6: API de opciones

Watch Decision

Método	Ruta	Descripción
POST	/decisiones/{id}/watch	Crear configuración watch_decision
GET	/decisiones/{id}/watch	Obtener configuración watch_decision
PUT	/decisiones/{id}/watch	Actualizar watch_decision
DELETE	/decisiones/{id}/watch	Eliminar watch_decision

Cuadro 6.7: API de watch_decision

Watch Position

Método	Ruta	Descripción
GET	/decisiones/{id}/watch/posicion	Obtener el progreso del usuario
PUT	/decisiones/{id}/watch/posicion	Actualizar el progreso del usuario

Cuadro 6.8: API de watch_position

Votos

Método	Ruta	Descripción
POST	/decisiones/{id}/votos/simple	Crear voto simple
GET	/decisiones/{id}/votos/simple	Obtener votos simples
POST	/decisiones/{id}/votos/ranking	Crear voto ranking
GET	/decisiones/{id}/votos/ranking	Obtener votos ranking
POST	/decisiones/{id}/votos/watch	Crear voto watch
GET	/decisiones/{id}/votos/watch	Obtener votos watch

Cuadro 6.9: API de votos

6.3. Diagrama de base de datos

En la Figura 6.2 podemos ver el diagrama entidad-relación de la base de datos. Como puede observarse, se ha optado por utilizar nomenclatura en inglés para la definición de las entidades y sus atributos, salvo en el nombre de las tablas *usuario*, *grupo* y *opcion*. Esto es debido a que en supabase las palabras *user*, *group* y *option* están reservadas y podrían ocasionar conflictos o problemas en la implementación.

El sistema estará formado por doce tablas: *auth.users*, *usuario*, *grupo*, *notifications*, *decision*, *option*, *simple_vote*, *ranking_vote*, *group_members*, *watch_decision*, *watch_decision_position* y *watch_vote*. La tabla *auth.user* es proporcionada por defecto por *Supabase* y se utiliza para gestionar de manera segura la información personal de los usuarios. Aunque esta tabla cuenta con un número elevado de atributos, en el diagrama únicamente se representan los que se usarán en el desarrollo.

Por su parte, las tablas *simple_vote*, *ranking_vote*, *watch_vote*, *watch_decision_position* y *group_members* actúan como tablas intermedias que permiten modelar relaciones de tipo muchos a muchos, garantizando la unicidad de las asociaciones. En concreto, *simple_vote* y *ranking_vote* relaciona los usuarios con las opciones de una decisión, *watch_vote*

y *watch_decision_position* los vinculan con las instancias de *watch_decision*, y *group_members* define la asociación entre usuarios y grupos.

En las relaciones existentes entre tablas, cabe destacar que la tabla *option* es una entidad debil de *decision*.

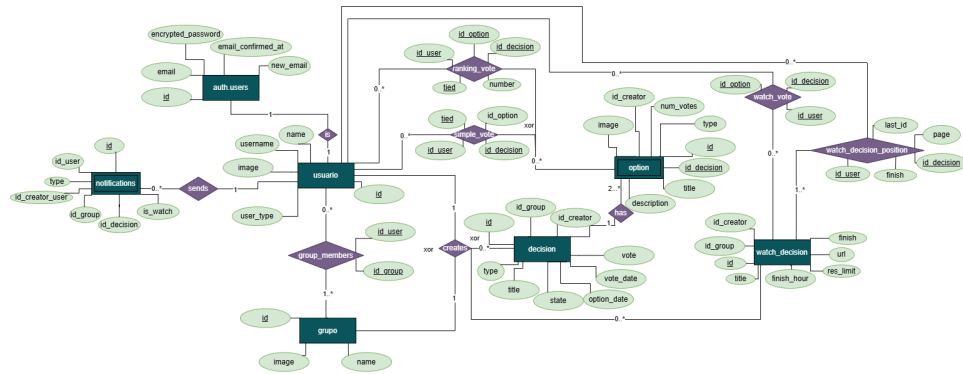


Figura 6.2: Diagrama de la base de datos

A continuación, se muestran en detalle los atributos de cada tabla, especificando los tipos de datos, claves primarias (PK), claves foráneas (FK), restricciones de unicidad (U), nulabilidad (N) y valores por defecto.

auth.users				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
email	TEXT	-	✓	×
encrypted_password	TEXT	-	×	×
email_confirmed_at	TIMESTAMP	-	×	✓
new_email	TEXT	-	×	✓
Claves				
PK	id			

Cuadro 6.10: Definición de la tabla *auth.users*

usuario				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
username	TEXT	-	✓	×
name	TEXT	-	×	×
image	TEXT	-	×	✓
user_type	user_type	'client'	×	×
Claves				
PK	id			
FK	id → auth.users(id)			

Cuadro 6.11: Definición de la tabla *usuario*

grupo				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
name	TEXT	-	×	×
image	TEXT	-	×	✓
Claves				
PK	id			

Cuadro 6.12: Definición de la tabla *grupo*

group_members				
Atributo	Tipo	Valor defecto	U	N
id_group	UUID	gen_random_uuid()	×	×
id_user	UUID	gen_random_uuid()	×	×
Claves				
PK	id_group, id_user			
FK	id_group → grupo(id)			
FK	id_user → auth.users(id)			

Cuadro 6.13: Definición de la tabla *group_members*

notifications				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
id_user	UUID	gen_random_uuid()	×	×
id_creator_user	UUID	gen_random_uuid()	×	✓
id_group	UUID	gen_random_uuid()	×	×
type	notification_type	-	×	×
id_decision	UUID	gen_random_uuid()	×	✓
is_watch	bool	false	×	×
Claves				
PK	id			
FK	id_group → grupo(id)			
FK	id_user → auth.users(id)			
FK	id_creator → auth.users(id)			

Cuadro 6.14: Definición de la tabla *notifications*

decision				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
id_creator	UUID	gen_random_uuid()	×	×
id_group	UUID	gen_random_uuid()	×	✓
title	TEXT	-	×	×
state	decision_state	-	×	×
options_date	TIMESTAMPTZ	NULL	×	✓
vote_date	TIMESTAMPTZ	NULL	×	✓
type	decision_type	vote	×	×
vote	bool	false	×	×
Claves				
PK	id			
FK	id_creator → auth.users(id)			
FK	id_group → grupo(id)			

Cuadro 6.15: Definición de la tabla *decision*

watch_decision				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
id_creator	UUID	gen_random_uuid()	×	×
id_group	UUID	gen_random_uuid()	×	✓
title	TEXT	-	×	×
finish_hour	TIMESTAMPTZ	NULL	×	✓
res_limit	INT	NULL	×	✓
url	TEXT	-	×	×
finish	bool	false	×	×
Claves				
PK	id			
FK	id_creator → auth.users(id)			
FK	id_group → grupo(id)			

Cuadro 6.16: Definición de la tabla *decision_watch*

option				
Atributo	Tipo	Valor defecto	U	N
id	UUID	gen_random_uuid()	✓	×
id_decision	UUID	gen_random_uuid()	×	×
id_creator	UUID	gen_random_uuid()	×	×
title	TEXT	-	×	×
description	TEXT	-	×	✓
image	TEXT	-	×	✓
type	option_type	standard	×	×
num_votes	INT	0	×	✓
Claves				
PK	id, id_decision			
FK	id_creator → auth.users(id)			
FK	id_decision → decision(id)			

Cuadro 6.17: Definición de la tabla *opcion*

simple_vote				
Atributo	Tipo	Valor defecto	U	N
id_user	UUID	gen_random_uuid()	×	×
id_option	UUID	gen_random_uuid()	×	×
id_decision	UUID	gen_random_uuid()	×	×
tied	bool	false	×	×
Claves				
PK	id_user, id_decision, tied			
FK	id_user → auth.users(id)			
FK	id_option → option(id)			
FK	id_decision → decision(id)			

Cuadro 6.18: Definición de la tabla *simple_vote*

ranking_vote				
Atributo	Tipo	Valor defecto	U	N
id_user	UUID	gen_random_uuid()	×	×
id_option	UUID	gen_random_uuid()	×	×
id_decision	UUID	gen_random_uuid()	×	×
tied	bool	false	×	×
number	INT	0	×	×
Claves				
PK	id_user, id_decision, id_option, tied			
FK	id_user → auth.users(id)			
FK	id_option → option(id)			
FK	id_decision → decision(id)			

Cuadro 6.19: Definición de la tabla *ranking_vote*

watch_vote				
Atributo	Tipo	Valor defecto	U	N
id_user	UUID	gen_random_uuid()	×	×
id_option	INT	0	×	×
id_decision	UUID	gen_random_uuid()	×	×
Claves				
PK	id_user, id_decision, id_option			
FK	id_user → auth.users(id)			
FK	id_decision → decision(id)			

Cuadro 6.20: Definición de la tabla *watch_vote*

watch_decision_position				
Atributo	Tipo	Valor defecto	U	N
id_user	UUID	gen_random_uuid()	×	×
id_decision	UUID	gen_random_uuid()	×	×
last_id	INT	0	×	×
page	INT	1	×	×
finish	bool	false	×	×
Claves				
PK	id_user, id_decision			
FK	id_user → auth.users(id)			
FK	id_decision → decision(id)			

Cuadro 6.21: Definición de la tabla *watch_decision_position*

Se puntualizan a continuación algunos atributos de las tablas:

En la tabla *auth_user* el atributo *new_email* será usado para hacer el cambio de email asociado a una cuenta. Por tanto, para mantener la unicidad, en *new_email* no podrá insertarse un valor que ya existe en *email*. En *new_email* sí podrá haber valores repetidos, ya que finalmente solo se trasladará a *email* en la primera cuenta que confirme el cambio.

6.4. Diagrama de clases

En la Figura 6.3 se muestran los modelos y servicios que mantienen la información que se muestra en la aplicación actualizada.

La función de la clase *NotificationDisplayModel* es ayudar en la representación visual de las notificaciones en la aplicación. Por su parte, *DecisionDraft* y *OptionDraft* son usados en el proceso de creación de una decisión con el propósito de ir almacenando temporalmente los datos entre pantallas hasta que son guardados en la base de datos. Asimismo, la finalidad de las clases *CheckUsername*, *CheckEmail* y *CheckPassword* es validar los datos de un usuario en su creación y edición, utilizando para ello un patrón plantilla.

Con el fin de aportar claridad visual se ha omitido la relación de asociación que mantienen todos los servicios con *SupabaseClient*, al ser utilizado como atributo para realizar operaciones con el backend.

6.5. Diagramas de secuencia

Se desarrollan ahora los principales métodos con los que cuenta la aplicación. En todos ellos, podemos observar la participación de los elementos *Frontend* que es la interfaz de usuario y la lógica de presentación, *Riverpod* que actúa como la capa de estado del *Frontend* y distintos módulos de *Supabase*. *Supabase Auth* maneja la sesión, *Supabase Database* almacena los datos y *Supabase Storage* se utiliza para almacenar las imágenes usadas en la aplicación.

En la Figura 6.4 podemos observar los métodos de inicio y cerrado de sesión. Para el inicio de sesión se hace uso de *Edge Function* como intermediario, con el objetivo de devolvernos el email asociado a un username, ya que *Supabase* solo permite iniciar sesión con email y contraseña. En caso de que todo el método transcurra correctamente, se devuelve la sesión. Por su parte, en el cerrado de sesión se establece la sesión a nula y se devuelve éxito.

La Figura 6.5 muestra el proceso de registro. En este proceso, se hace de dos módulos de *Supabase*. Por un lado, primero se intenta añadir el usuario en la tabla *auth.users* comprobando que no haya una cuenta existente con el email introducido. Posteriormente, mediante un trigger automático dentro de la base de datos, se crea el usuario en la tabla *usuario* comprobando que el username no esté repetido. Si ya existía ese username en el sistema, se

revierte todo el proceso.

Podemos observar en la Figura 6.6 como se crea un grupo y una opción. Ambos, cuentan con un proceso muy similar, pasando por el flujo *Frontend*, *Riverpod* y *Supabase Database* y devolviendo un error en caso de que haya algún problema. Sin embargo, el grupo cuenta con un paso intermedio para almacenar la imagen de perfil en *Supabase Storage*, devolviendo la url para que sea almacenada en los datos del grupo.

En la Figura 6.7 vemos la creación de una decisión normal y una decisión watch. La peculiaridad con respecto al resto de métodos de creación que podemos observar en la decisión normal es un bucle entre *Riverpod* y *Supabase Database* para la creación de las opciones asociadas a una decisión.

La Figura 6.8 muestra el proceso llevado a cabo para visualizar los datos de una decisión watch. En este proceso están presentes tanto Edge Function como la API de TMDB. En ella, primero es necesario pedir los datos de la decisión y la página e ID de la última película o serie que vimos y posteriormente pedir los datos a la API de TMDB mediante la Edge Function para mantener segura la API Key usada. Una vez que llegan los datos de los diez títulos a Riverpod, filtra el siguiente medio a visualizar y devuelve al frontend únicamente esos datos. Almacenando el resto para futuras consultas.

La Figura 6.8 ilustra el proceso para visualizar una *decision watch*. En este flujo intervienen tanto una Edge Function como la API de TMDB. En primer lugar, se solicitan los datos de la decisión, así como la página e ID del último título (película o serie) visualizado. A continuación, se consulta la API de TMDB a través de la Edge Function para proteger la clave de acceso (*API Key*). Una vez que Riverpod recibe los diez títulos correspondientes, usando el último ID visto se filtra el siguiente contenido a mostrar, devolviendo únicamente esa información al Frontend y almacenando el resto en caché para futuras consultas.

Por último, la Figura 6.9 muestra el proceso de voto. Lo especial de este método es que dispone de un trigger interno en *Supabase* para determinar si la combinación de usuario y grupo era el último que quedaba por registrar con respecto a ese decisión, y en caso de ser así cambia automáticamente el estado de la decisión a terminada.

6.6. Diseño de interfaces de usuario

Se exponen ahora los bocetos que representan el estado final de la aplicación.

La aplicación se encuentra disponible en modo claro y oscuro, escogíendose uno u otro según la configuración del dispositivo. Los bocetos se han desarrollado en tema claro, habiéndose adjuntado uno adicional en el login con tema oscuro para que se pueda apreciar la gama de colores a usar.

En la Figura 6.10 podemos observar las pantallas de inicio de sesión, registro, confirmación de email, cambio de contraseña y avisos legales. Desde la pantalla de inicio de sesión, se podrá acceder a cualquiera de las otras cuatro.

La pantalla de Login(Figura 6.10a) actúa como el eje central de este conjunto de pantallas, ya que desde ella es posible acceder al resto de funcionalidades relacionadas con la autenticación y la gestión de la cuenta.

Al seleccionar la opción “He olvidado mi contraseña”, se muestra un cuadro de diálogo en el que el usuario debe introducir la dirección de correo electrónico asociada a su cuenta. Tras solicitar la recuperación, se envía un correo electrónico que contiene un enlace de restablecimiento. Al acceder a dicho enlace, el usuario es redirigido a la pantalla de Recuperar Contraseña(Figura 6.10e), donde puede establecer una nueva contraseña. Además, una vez completado el cambio, el sistema envía un correo de notificación a la dirección asociada a la cuenta como medida de seguridad adicional.

Por otro lado, al pulsar el botón “Registrarse”, la aplicación redirige al usuario a la pantalla de Registro(Figura 6.10c), donde puede introducir los datos necesarios para crear una nueva cuenta en el sistema. Una vez finalizado el proceso de registro, es obligatorio verificar la dirección de correo electrónico antes de poder iniciar sesión. Esta verificación se realiza mediante un enlace enviado automáticamente a la cuenta de correo proporcionada durante el registro.

Finalmente, la pantalla de Aviso Legal(Figura 6.10f) sigue la misma estructura y diseño que las pantallas de Política de Privacidad y Términos y Condiciones, mostrando la información legal correspondiente de forma clara y accesible.

Las Figuras 6.11 y 6.12 muestran las pantallas relacionadas con la ges-

ción de grupos. La pantalla principal (Figura 6.11a) presenta un listado de grupos que puede filtrarse mediante el buscador. En el caso de un usuario estándar, únicamente se muestran los grupos a los que pertenece, mientras que los administradores pueden visualizar todos los grupos existentes en el sistema. Además, al pulsar el icono «+» es posible acceder a la pantalla de creación de grupos (Figura 6.11e). Una vez creado un nuevo grupo, este aparecerá automáticamente en el listado de la pantalla principal.

Al acceder a un grupo se muestra su pantalla principal (Figura 6.11b), donde se listan las decisiones que se encuentran en las fases de sugerencia de opciones o de votación. Cada una de estas decisiones podrá ser editada u eliminada únicamente por el creador de la decisión o usuarios con rol de administrador. Desde esta pantalla también es posible crear una nueva decisión mediante el icono «+». Esta acción redirige a la pantalla intermedia de creación (Figura 6.13), desde la que el usuario puede elegir entre crear una decisión estándar o solicitar recomendaciones de películas o series en función de una serie de filtros.

La barra de navegación superior del grupo permite acceder a distintas funcionalidades. Al seleccionar el segundo icono se muestra el listado de miembros del grupo (Figura 6.11c), desde donde es posible eliminar a un integrante pulsando el icono de la papelera situado junto a su nombre. Antes de completar la acción, la aplicación solicita una confirmación mediante un cuadro de diálogo (Figura 6.11d). Del mismo modo, si un usuario desea abandonar el grupo, puede hacerlo mediante el icono de la papelera situado en la cabecera de la pantalla, debiendo confirmar igualmente la operación.

Por último, el tercer icono de la barra de navegación da acceso al historial de decisiones del grupo (Figura 6.12a), donde se muestra un listado con todas las decisiones finalizadas. Al seleccionar una de ellas, se accede a una vista detallada que incluye el título, el tipo de decisión y las opciones ordenadas según el número de votos obtenidos. Dependiendo del resultado, la decisión puede haber finalizado con una opción ganadora (Figura 6.12b) o con un empate (Figura 6.12c). En este último caso, únicamente el creador de la decisión dispone del botón para resolver el empate. El desempate puede realizarse mediante un voto adicional del creador o a través de una ruleta aleatoria (Figura 6.12d). Asimismo, al pulsar sobre cualquiera de las opciones es posible consultar su información detallada (Figura 6.18b).

El modo individual, representado en la Figura 6.14, presenta una interfaz muy similar a la del modo de grupos, con el objetivo de mantener una experiencia de usuario homogénea en toda la aplicación. La principal diferencia

radica en que las decisiones pertenecen exclusivamente al usuario, por lo que desaparecen todas las funcionalidades relacionadas con la gestión de grupos y de sus miembros. Además, en la pantalla principal únicamente se muestran las decisiones que se encuentran en fase de votación, mientras que las decisiones finalizadas pueden consultarse desde el apartado de historial, cuya estructura y funcionamiento son equivalentes a los descritos para el modo de grupos.

La Figura 6.15 muestra la pantalla de notificaciones de la aplicación. Cada notificación puede eliminarse de forma individual mediante el icono «×». Su finalidad es mantener al usuario informado sobre los eventos más relevantes relacionados con los grupos a los que pertenece y con el estado de sus decisiones. Entre las notificaciones que pueden recibirse se encuentran las invitaciones para unirse a un grupo, la expulsión de un grupo, el inicio de la fase de sugerencia de opciones, el comienzo de la fase de votación y la finalización de una decisión.

La Figura 6.16 muestra la pantalla de edición del perfil de usuario. Desde ella es posible modificar los datos de la cuenta, incluyendo el nombre de usuario, el nombre, la dirección de correo electrónico y la contraseña. Asimismo, el usuario puede cerrar sesión, eliminar su cuenta mediante el icono de la papelera y consultar los documentos legales de la aplicación.

En caso de modificar la dirección de correo electrónico, la aplicación informa al usuario de que deberá confirmar el cambio mediante el enlace de verificación enviado a la nueva dirección de correo electrónico antes de que este se haga efectivo.

Las Figuras 6.17 muestran el proceso de creación y visualización de una decisión estándar. Pudiendo modificar su título, tipo, opciones asociadas y fechas límites para finalizar el estado de sugerencia de opciones y el estado de votación. Pulsando en el botón «+» se accede a la pantalla de creación de una opción. Pulsando en abrir opciones se creará la decisión empezando en el estado de sugerencia de opciones. Si por el contrario, pulsas en el botón de Guardar se creará en el estado de votación. Por su parte, la Figura 6.17c muestra la misma información, pero no permite modificar ningún dato de la decisión, salvo sus opciones asociadas, donde podrás añadir más opciones y se permitirá al creador de la decisión, al creador de la opción y a los administradores editar y borrar las opciones.

Las Figuras 6.17 muestran el proceso de creación y edición de una decisión estándar. Desde esta pantalla es posible configurar el título de la decisión,

seleccionar su tipo, gestionar las opciones asociadas y establecer las fechas límite tanto para la fase de sugerencia de opciones como para la fase de votación. Asimismo, al pulsar el botón «+» se accede a la pantalla de creación de una nueva opción.

Una vez configurada la decisión, el usuario puede pulsar el botón “Abrir opciones” para crearla e iniciar la fase de sugerencia de opciones. Alternativamente, si selecciona el botón “Guardar”, la decisión se crea directamente en la fase de votación, omitiendo la fase de sugerencia.

Por su parte, la Figura 6.17c muestra la pantalla de consulta de una decisión. En ella no es posible modificar la información general de la decisión, aunque sí gestionar las opciones asociadas. En concreto, los usuarios pueden añadir nuevas opciones y, en función de sus permisos, editarlas o eliminarlas. Estas acciones están disponibles para el creador de la decisión, el creador de la opción correspondiente y los administradores del sistema.

La Figura 6.18 muestra las pantallas de creación y consulta de una opción. Durante la creación de una opción (Figura 6.18a) el usuario puede definir un título y, de forma opcional si no genera datos, añadir una descripción y una imagen.

Además, la aplicación ofrece la posibilidad de completar automáticamente la información de la opción a partir de su título. Para ello, el usuario puede indicar si la opción corresponde a una película o a una serie, recuperándose automáticamente la información disponible para dicho contenido.

Si no se utiliza esta funcionalidad, la consulta de la opción presenta el formato mostrado en la Figura 6.18b, donde únicamente es posible visualizar la información introducida durante su creación. En cambio, cuando la información se obtiene automáticamente, la consulta adopta el formato ilustrado en las Figuras 6.20d y 6.20e, mostrando datos adicionales del contenido, como la imagen, la sinopsis, los géneros, la fecha de estreno y la valoración media.

El proceso de votación de una decisión estándar se representa en la Figura 6.19.

La Figura 6.19a muestra la interfaz correspondiente a las decisiones de voto único y voto múltiple. En ella se presenta el título de la decisión junto con el listado de opciones disponibles. Dependiendo del tipo de votación,

el usuario podrá seleccionar una única opción o varias mediante las casillas de selección (*checkbox*). Una vez realizada la selección, el voto se registra al pulsar el botón “Votar”.

Por su parte, la Figura 6.19c corresponde a la modalidad de votación por ranking. Aunque la interfaz es similar a la anterior, en este caso el usuario debe establecer un orden de preferencia entre las opciones disponibles, reordenándolas mediante una operación de arrastre desde la mayor hasta la menor prioridad.

Finalmente, la Figura 6.19b muestra la modalidad de decisión mediante ruleta. En este tipo de decisión, únicamente el creador puede iniciar el proceso pulsando el botón “Girar”. Tras la pulsación, la ruleta comienza a girar mostrando efectos visuales y sonoros, y al finalizar el giro se selecciona y anuncia automáticamente la opción ganadora.

La Figura 6.20 muestra el proceso de creación y votación de una decisión predefinida para la selección de películas o series. El proceso de creación, representado en las Figuras 6.20a y 6.20b, requiere definir un título para la decisión y seleccionar el tipo de contenido (película o serie). De forma opcional, el usuario puede configurar distintos filtros para personalizar la búsqueda de contenidos.

Entre los filtros disponibles se incluyen la plataforma de *streaming* en la que el contenido está disponible en España, los géneros, la valoración media, la duración, la fecha de estreno y el criterio de ordenación de los resultados. Los filtros de valoración, duración y fecha de estreno permiten definir un intervalo de valores, mientras que también es posible establecer un número máximo de resultados con el objetivo de agilizar el proceso de toma de decisiones. Asimismo, puede configurarse un tiempo límite para la fase de votación. Una vez completada la configuración, al pulsar el botón “Empezar” se crea la decisión en estado de votación.

La Figura 6.20c muestra la interfaz de votación de este tipo de decisiones. Para cada contenido propuesto, el usuario puede expresar su preferencia pulsando el botón del “Corazón” para votar a favor de su visualización o el botón con el símbolo “X” para votar en contra. Durante la votación se muestran el título de la decisión y la información principal de cada contenido, incluyendo el título, el póster, los géneros, la fecha de estreno, la valoración media y la sinopsis.

Si el usuario desea consultar información adicional, puede pulsar el botón “Ver más +”, que da acceso a las pantallas mostradas en las Figuras 6.20d y 6.20e. En ellas se presenta, siempre que esté disponible, información complementaria como el eslogan, las plataformas donde puede visualizarse el contenido, la duración, la fecha de estreno, el reparto principal, el equipo técnico, las compañías productoras, el presupuesto, la recaudación mundial, el tráiler y un listado de títulos similares.

Por último, la Figura 6.21 muestra las pantallas específicas del perfil de administrador. Como puede observarse, la barra de navegación inferior difiere de la disponible para un usuario estándar, sustituyendo los apartados de modo individual y notificaciones por un listado con todos los usuarios registrados en la aplicación (Figura 6.21c).

Desde dicho listado, al pulsar en un usuario, se accede a la pantalla de edición del usuario (Figuras 6.21d y 6.21e). Esta pantalla presenta las mismas funcionalidades que la edición del perfil de un usuario, incorporando además la posibilidad de modificar el tipo de usuario (usuario estándar o administrador).

Por otra parte, un administrador no puede eliminar su propia cuenta, por lo que la pantalla de perfil no muestra el icono de la papelera (Figura 6.21a). Asimismo, dado que los administradores no pertenecen a ningún grupo, el icono de la papelera situado en la cabecera de la pantalla de un grupo se utiliza para eliminar dicho grupo (Figura 6.21b).

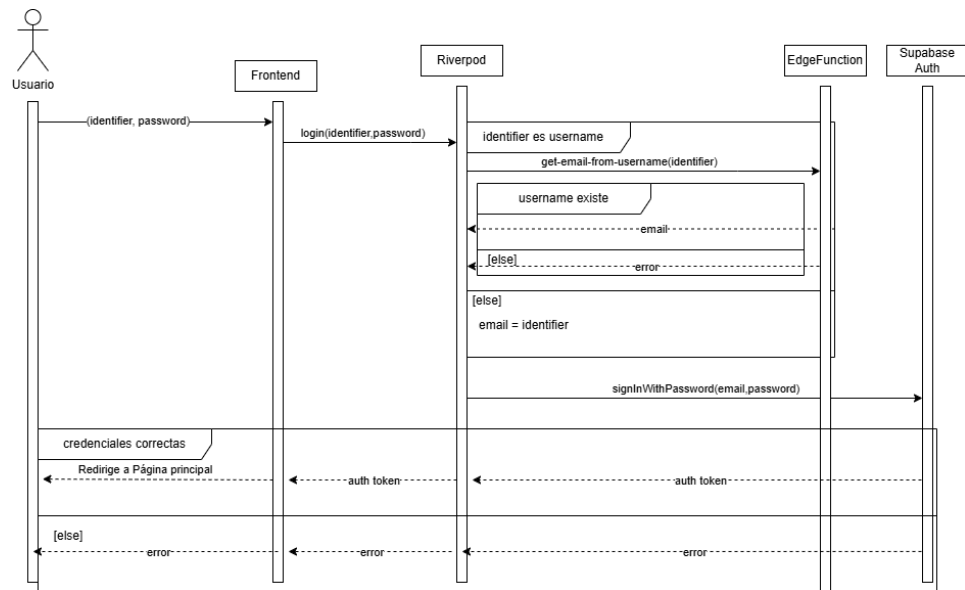
6.7. Diagrama de navegabilidad

En la siguiente Figura 6.22 se expone el diagrama que representa la navegabilidad dentro de la aplicación, indicando mediante las flechas la forma de pasar de una pantalla a otra.

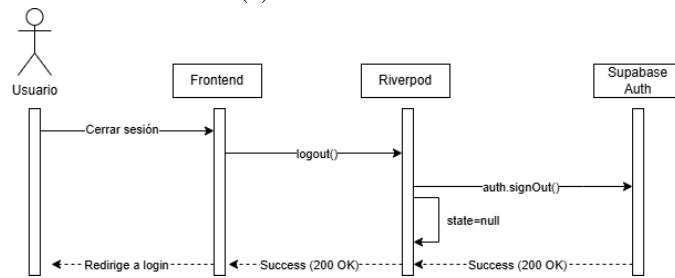
Las flechas grandes indican las posibles pantallas que pueden abrirse al iniciar la aplicación. De tener una sesión abierta, se iniciará en la *HomePage*, si no en *Login*.

Asimismo, desde cualquier pantalla que no sea *Login*, *Registro*, *Políticas* o *Recuperar contraseña* se podrá navegar a las distintas pantallas representadas en violeta. Se han ahorrado estas líneas, para aportar claridad.

Dependiendo del tipo de usuario, se podrá acceder a distintas pantallas violetas. Un usuario normal no podrá acceder a la pantalla *Lista de usuarios*; por su parte, un administrador no dispondrá de las pantallas *Modo individual* y *Notificaciones*. Asimismo, únicamente los creadores de la una decisión del historial podrán acceder a la pantalla de desempate.



(a) Inicio de sesión



(b) Cierre de sesión

Figura 6.4: Diagramas de secuencia de sesión

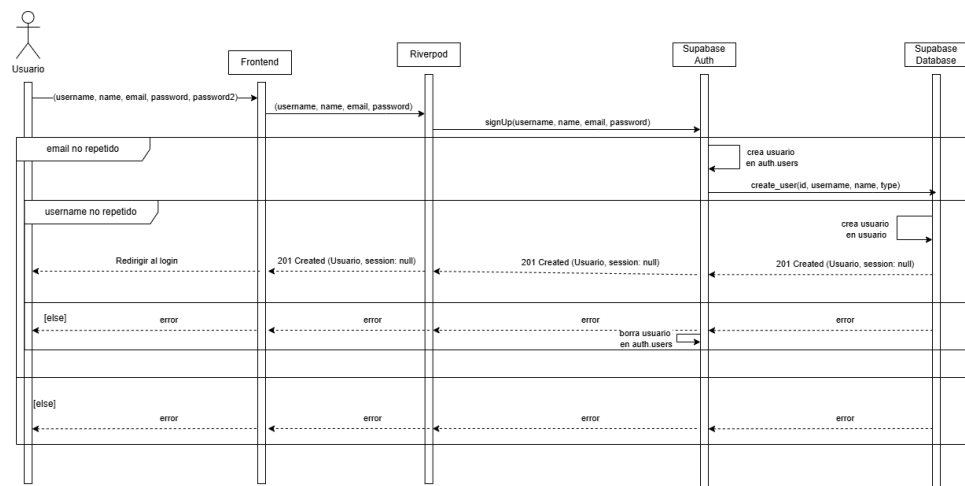
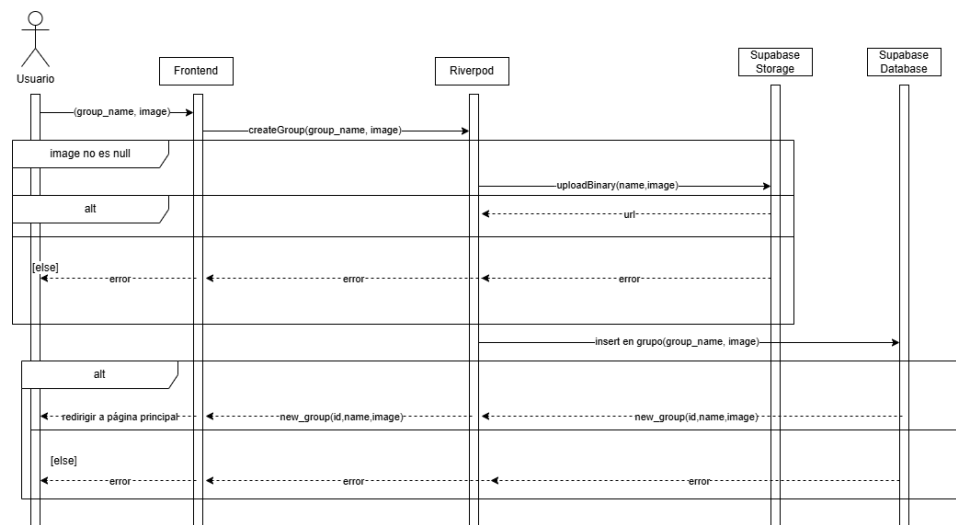
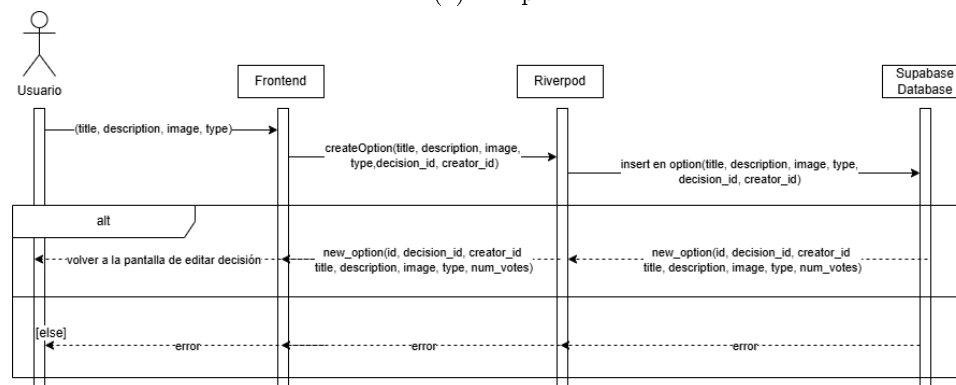


Figura 6.5: Diagrama de secuencia de registro

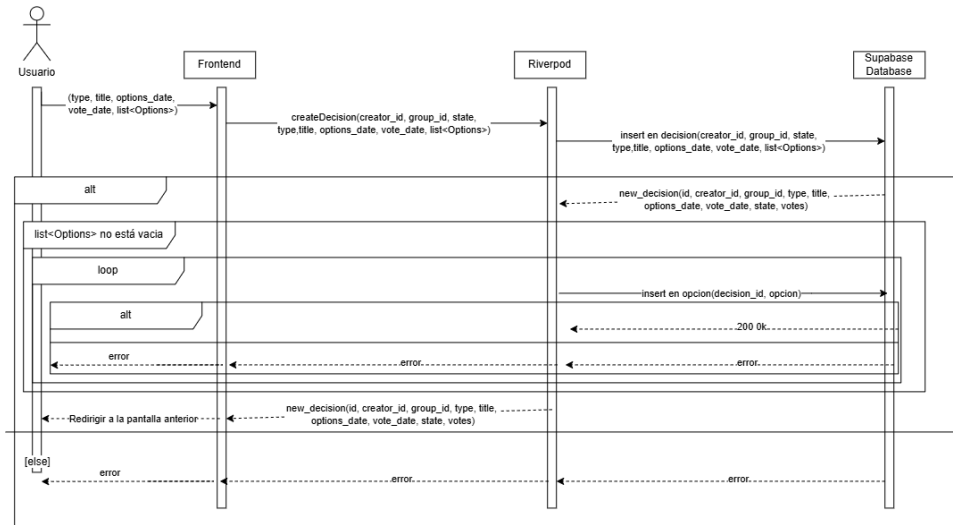


(a) Grupo

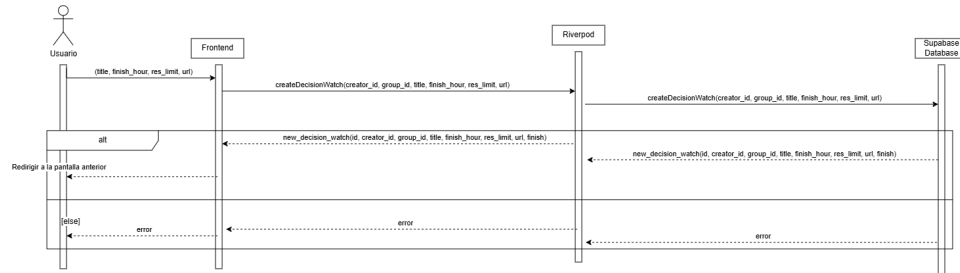


(b) Opción

Figura 6.6: Diagramas de secuencia de creacion



(a) Normal



(b) Watch

Figura 6.7: Diagramas de secuencia de creación de decisión

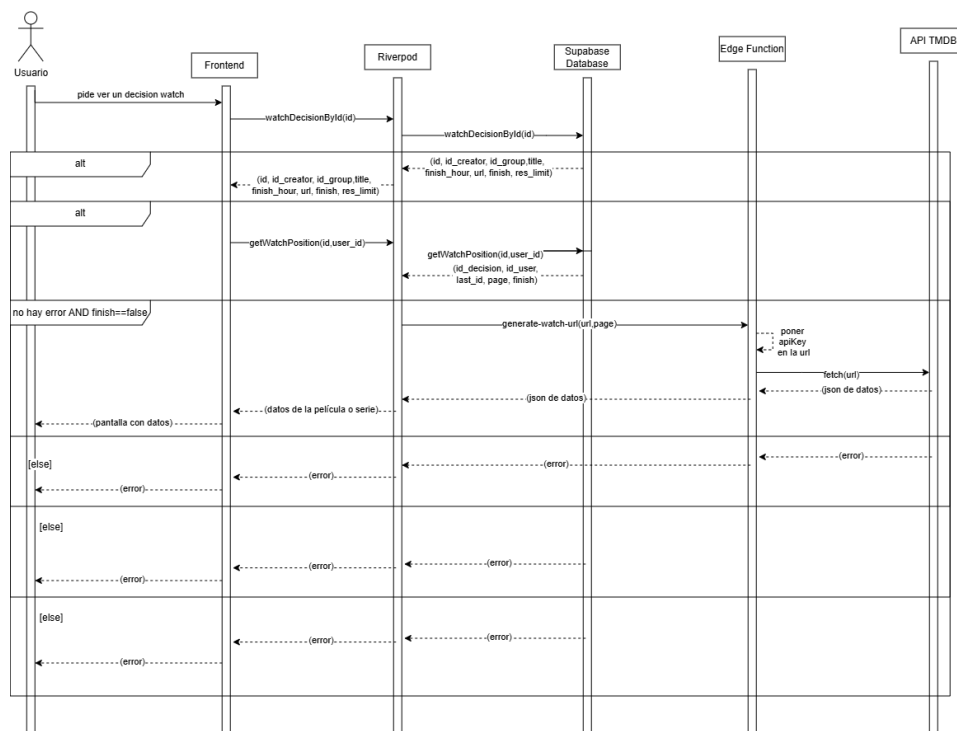


Figura 6.8: Diagramas de secuencia de visualización de una decision watch

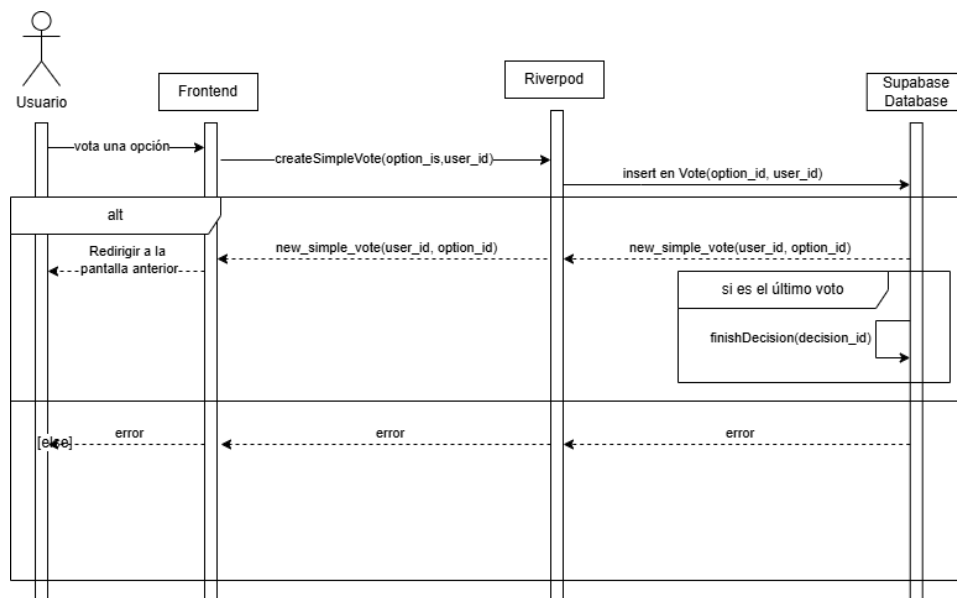


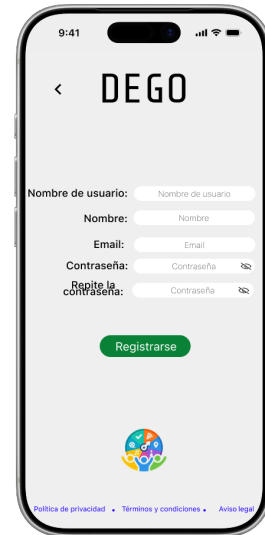
Figura 6.9: Diagramas de secuencia de voto



(a) Login



(b) Login modo oscuro



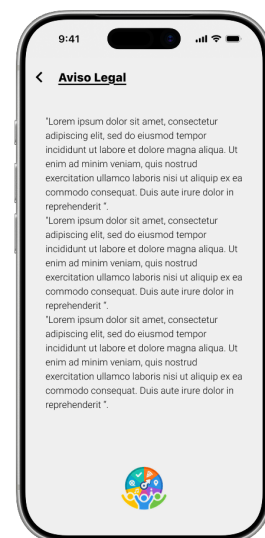
(c) Registro



(d) Confirmar email

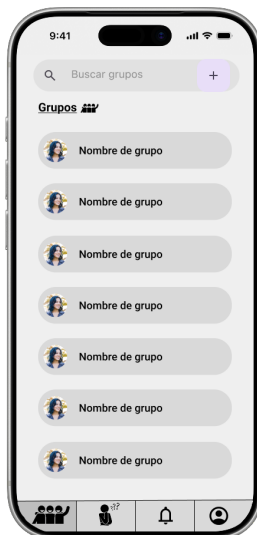


(e) Recuperar contraseña



(f) Aviso Legal

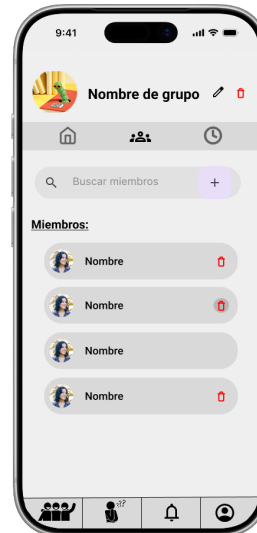
Figura 6.10: Bocetos de las pantallas iniciales



(a) Página principal



(b) Pantalla principal de un grupo



(c) Miembros de un grupo



(d) Eliminar un miembro



(e) Crear un grupo

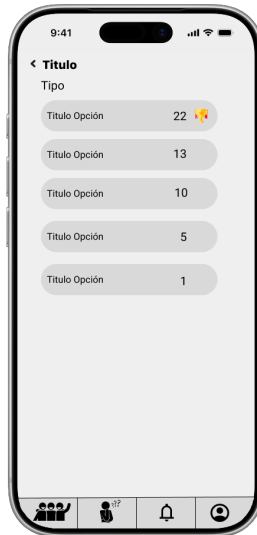


(f) Salirse de un grupo

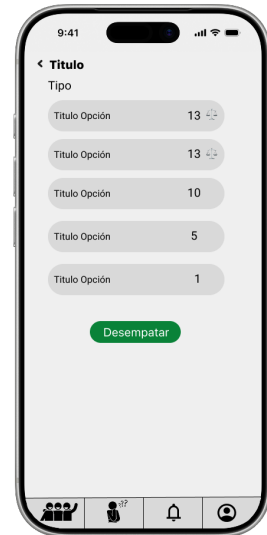
Figura 6.11: Bocetos de las pantallas de grupos



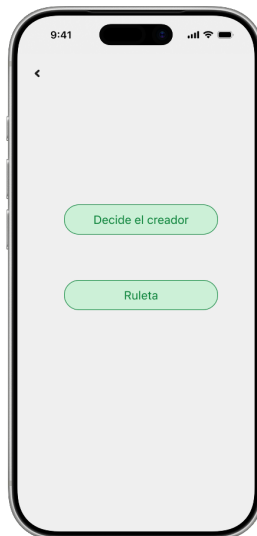
(a) Historial de un grupo



(b) Historial de una decisión con un ganador



(c) Historial de una decisión con un empate



(d) Pantalla intermedia para desempatar

Figura 6.12: Bocetos del historial de un grupo



Figura 6.13: Boceto de la pantalla inicial para crear una decisión



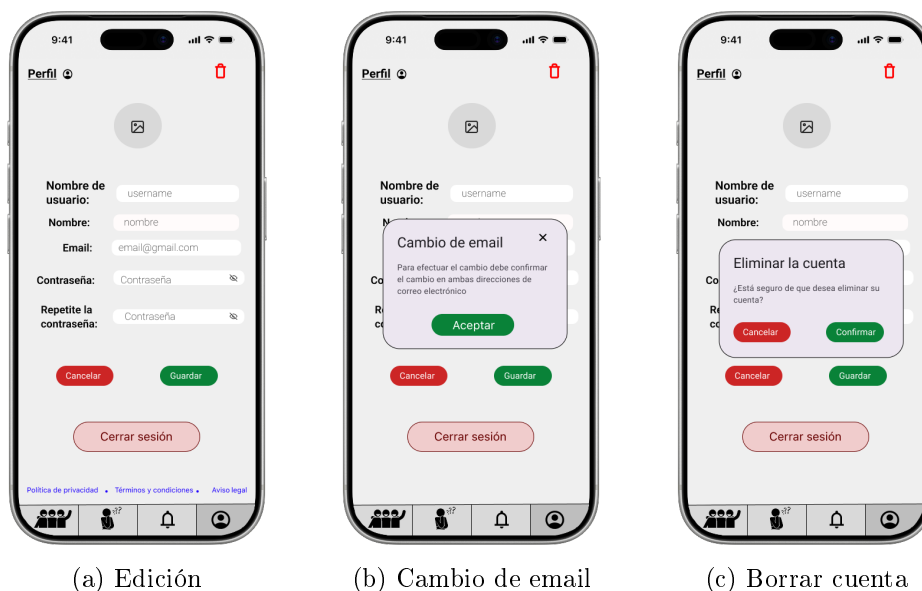
(a) Página principal

(b) Historial

Figura 6.14: Bocetos del modo individual



Figura 6.15: Bocetos de las notificaciones



(a) Edición

(b) Cambio de email

(c) Borrar cuenta

Figura 6.16: Bocetos del perfil



Figura 6.17: Creación y consulta de una decisión

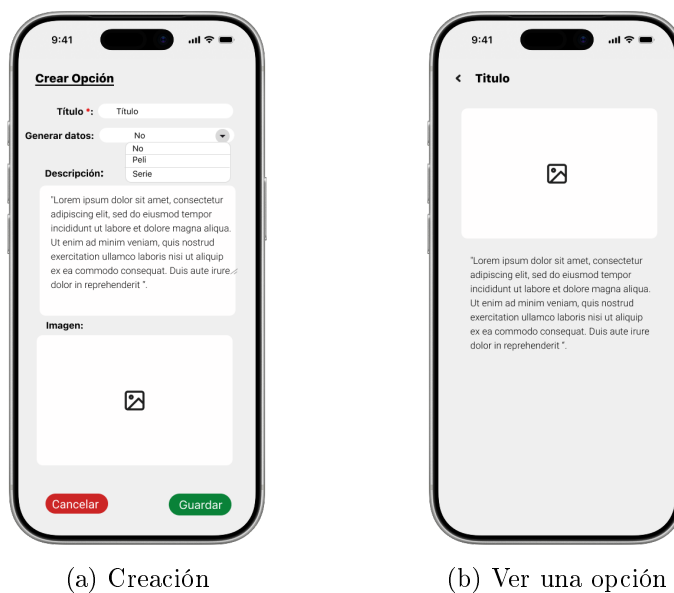


Figura 6.18: Creación y consulta de una opción



Figura 6.19: Votar

9:41

¿Qué ver?

Título:

Tipo: Película Serie

Plataforma: Nombre Nombre Nombre Nombre

Género: Nombre Nombre Nombre Nombre Nombre

Puntuación:

Duración película:

Estreno:

(a) Crearla

9:41

Puntuación:

Duración película:

Estreno:

Orden: Mejor valoradas Más populares Más taquilleras Más recientes Mejor valoradas Más vistas

Máximo de resultados: ☐

Máximo:

Tiempo máximo:

Cancelar Empezar

(b) Crearla

9:41

< Título decision

Título

Géneros: Nombre Nombre Nombre

Fecha de estreno: 12-11-2025

Puntuación: 7.2 (12563 votos)

Sinopsis: Texto relativo a la descripción. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Lorem ipsum dolor sit amet, consectetur adipiscing elit.

Ver más +

✕ ❤️

(c) Votar

9:41

< Título decision

Título

Eslogan

Disponible en: Nombre Nombre Nombre

Duración: Duración: 2h 30min Fecha de estreno: 12-11-2025

Reparto principal: Nombre, Nombre, Nombre, Nombre, Nombre, Nombre, Nombre

Equipo técnico: Dirección: Nombre Guion: Nombre

Compañías Productoras: Nombre - Nombre

Finanzas: Presupuesto: x € Recaudación Mundial: x €

Trailer:

(d) Ver detalles

9:41

Equipo técnico: Dirección: Nombre Guion: Nombre

Compañías Productoras: Nombre - Nombre

Finanzas: Presupuesto: x € Recaudación Mundial: x €

Trailer:

Títulos similares:

Nombre Nombre Nombre North

(e) Ver detalles

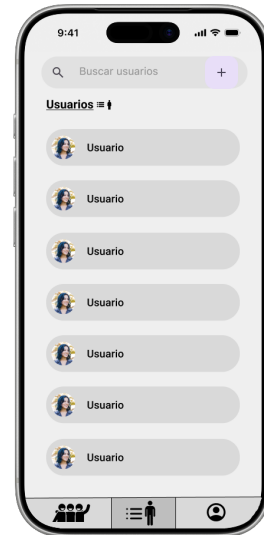
Figura 6.20: Bocetos de decisión predeterminada



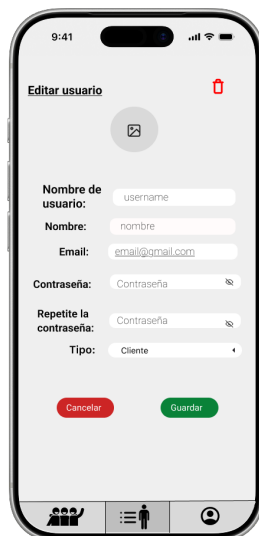
(a) Perfil



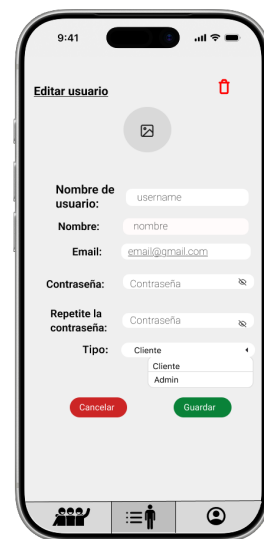
(b) Eliminar un grupo



(c) Lista de usuarios



(d) Editar un perfil



(e) Cambiar el tipo de un perfil

Figura 6.21: Bocetos de las pantallas de administración

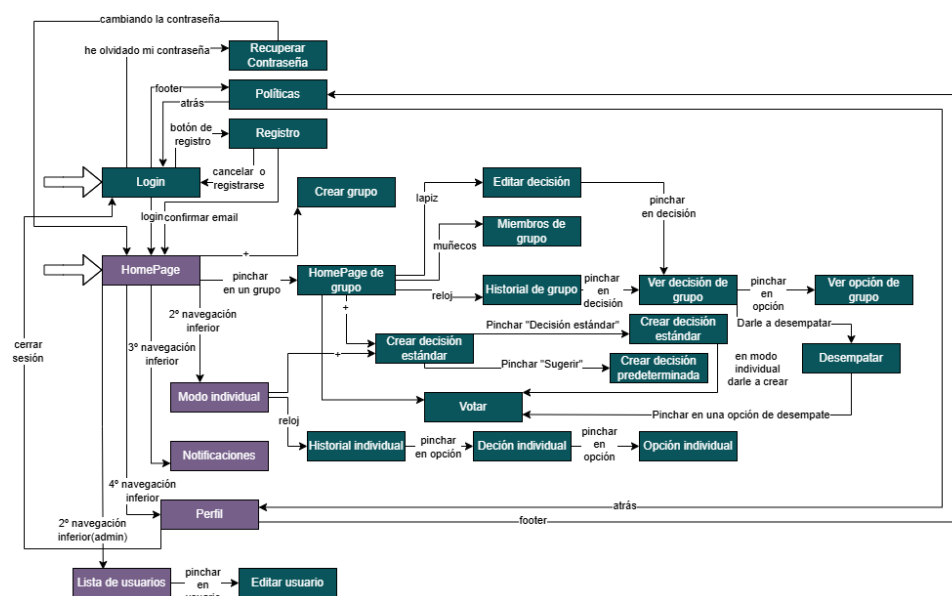


Figura 6.22: Diagrama de la navegabilidad del sistema

Capítulo 7

Implementación

7.1. Herramientas

En esta sección se describen las distintas herramientas empleadas durante el desarrollo del proyecto, indicando la función que desempeña cada una de ellas en las distintas fases de creación e implementación de la aplicación.

- **Visual Studio Code:** Entorno de desarrollo integrado (*IDE*) utilizado para la implementación del Frontend de la aplicación.
- **Android Studio:** Herramienta empleada para la ejecución y prueba de la aplicación en emuladores y dispositivos Android, permitiendo visualizar los cambios realizados durante el desarrollo.
- **Supabase:** Plataforma Backend as a Service (*BaaS*) utilizada como backend de la aplicación, proporcionando servicios de base de datos, autenticación y almacenamiento. Su uso se extendió durante todo el ciclo de vida del desarrollo. Como se ilustra en las Figuras 7.1 y 7.2, la plataforma facilitó la visualización de la estructura y el contenido de las distintas tablas. Además, tal como muestra la Figura 7.3, permitió definir directamente desde su interfaz web tanto las tablas como los *triggers* y las funciones de la base de datos. Por último, resultó una herramienta clave para la monitorización y detección de errores en las *edge functions*, disparadores y en el sistema de almacenamiento de imágenes, optimizando significativamente el flujo de trabajo.
- **Jira:** Herramienta de gestión de proyectos utilizada para la planificación y organización de los distintos *sprints*, lo que facilitó la distribución y el seguimiento de las tareas a lo largo de todo el desarrollo. Inicialmente, permitió dividir las historias de usuario en bloques de trabajo de acuerdo con sus puntos de historia asociados. Posteriormente, se

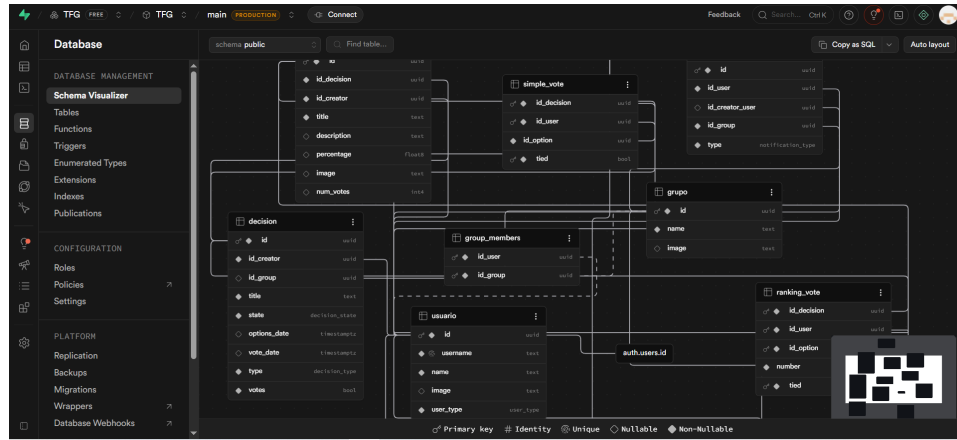


Figura 7.1: Estructura de las tablas de base de datos con Supabase

id	id_creator	id_group	title	text	state	decision_state
08e0750-d327-4268-82b6-832ad5b3d131	a954e500-8240-4b07-ba20-6c27...	05e0e71d-8c4b-4154-a770-703ab...	asesssas		vote	
15ee7445-2251-4c7f-8777-737bba3c98b	a954e500-8240-4b07-ba20-6c27...	eb844a87-0097-448c-8bfe-9eb5f...	Prueba		finish	
2397f86-d679-4021-a70b-fc46a4da40	a954e500-8240-4b07-ba20-6c27...	eb844a87-0097-448c-8bfe-9eb5f...			options	
2a3963d9-b0c4-4879-b197-20b40dccc1b	a954e500-8240-4b07-ba20-6c27...	1082f3e-53ba-400b-bfbc-dad19...	Empate multiple		finish	
3102f31-7305-4a46-b0d-99b0f623cd0	a954e500-8240-4b07-ba20-6c27...	66cd7a38-3675-45ae-893b-fb4b6...	ranking		vote	
4908b0c0-057b-4919-8d04-316c682a582	a4e4eac3-45c0-4fa1-b4a6-fa4738...	eb844a87-0097-448c-8bfe-9eb5f...	driftids		options	
4bbabef7-70ad-46d1-b623-751edf62e92	a4e4eac3-45c0-4fa1-b4a6-fa4738...	eb844a87-0097-448c-8bfe-9eb5f...	ruleta		finish	
4d58f29f-a9f5-47a5-b378-0e40d3c3e17e	a954e500-8240-4b07-ba20-6c27...	1082f3e-53ba-400b-bfbc-dad19...	prueba ranking 2		finish	
5fbc380f-ec9f-47d4-9cf1-392c804cf4d4	a4e4eac3-45c0-4fa1-b4a6-fa4738...	eb844a87-0097-448c-8bfe-9eb5f...	pan		options	
53ca9df-1450-4270-9ca5-a92f8f81cf1	a954e500-8240-4b07-ba20-6c27...	eb844a87-0097-448c-8bfe-9eb5f...	Votación para que se calle padre		finish	
759791e-0363-44a7-8151-487b8f0107c	a4e4eac3-45c0-4fa1-b4a6-fa4738...	8ade6619-f4df-4f0c-9a5b-f4e531...	tema		finish	
90fcc4db-1f6c-4837-b4d9-60b49ac316fb	a954e500-8240-4b07-ba20-6c27...	1082f3e-53ba-400b-bfbc-dad19...	Empate ranking		finish	
a44388ba-3c74-44ea-a5ea-04d7132d2501	a954e500-8240-4b07-ba20-6c27...	66cd7a38-3675-45ae-893b-fb4b6...	edades		finish	
ab3481b-972c-49ab-853e-5ebd1cc3ecae	a954e500-8240-4b07-ba20-6c27...	eb844a87-0097-448c-8bfe-9eb5f...	Prueba de emote		finish	

Figura 7.2: Contenido de las tablas de base de datos con Supabase

empleó como indicador del progreso global de la aplicación (ver Figura 7.4), ayudando a estimar el cumplimiento de los plazos. Asimismo, tal como se muestra en la Figura 7.5, fue de gran utilidad para visualizar el estado de las tareas dentro del *sprint* activo, optimizando así la organización del trabajo diario.

- **Github:** Plataforma de control de versiones empleada para alojar el código fuente del proyecto y gestionar su evolución mediante un repositorio.
- **Google Drive:** Servicio de almacenamiento en la nube utilizado en la generación de los distintos diagramas representados en el presente documento.
- **Gmail:** Servicio de correo electrónico utilizado para el envío de men-

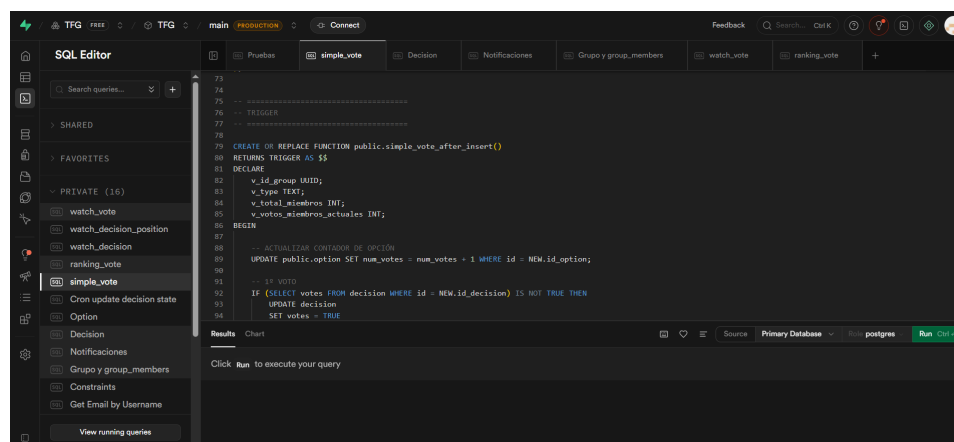


Figura 7.3: Creación de la base de datos con Supabase

sajes de verificación de las cuentas registradas en la aplicación y para la confirmación de los cambios de contraseña.

- **Overleaf:** Plataforma basada en $\text{\LaTeX}$ utilizada para la redacción y edición de la presente memoria.
- **Figma:** Herramienta de diseño empleada en la elaboración de los diagramas de interfaz de usuario incluidos en el documento.

7.2. Explicación de la implementación

A lo largo de este apartado se explicarán los aspectos más importantes desarrollados durante la implementación de la aplicación.

7.2.1. Decisiones

El proceso más importante e interesante de la aplicación es la toma de decisiones. Para la creación de una decisión se utilizan dos modelos auxiliares, *DecisionDraft* y *OptionDraft*, que contienen los mismos campos que los modelos *Decision* (*id*, *id_group*, *id_creator*, *title*, *state*, *options_date*, *vote_date*, *type*, *votes*) y *Option* (*id*, *id_decision*, *id_creator*, *title*, *description*, *type*, *image*, *num_votes*) respectivamente, pero sin los identificadores y variables de estado asociadas. Estos modelos tienen como finalidad almacenar los datos temporalmente entre pantallas. En *DecisionDraft* se almacena además la lista de *OptionDraft* creadas. Una vez finalizado el diseño de la decisión, el usuario puede optar por dos caminos: crear la decisión directamente para ponerla en votación, o iniciar el proceso de sugerencia de opciones. Dependiendo de la elección, tal como se muestra en la Figura 6.11b, la decisión

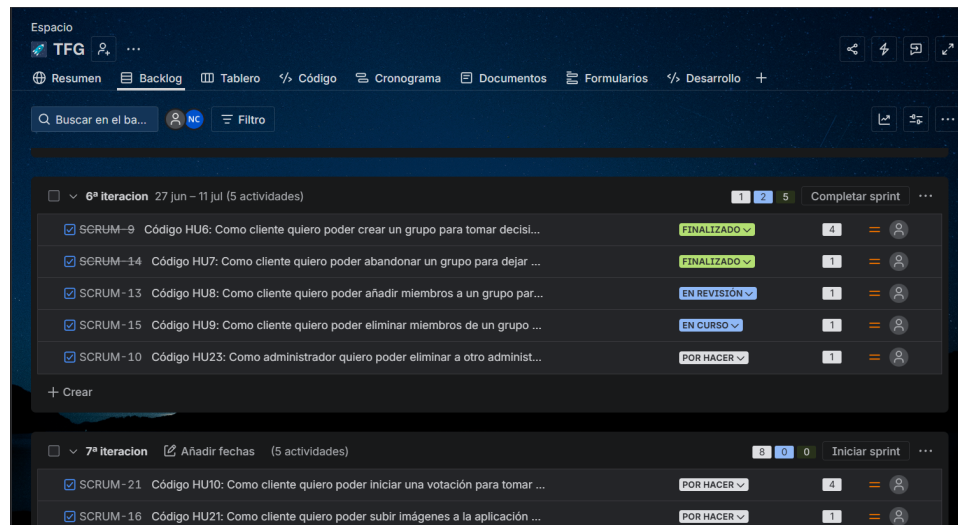


Figura 7.4: Gestión de los sprints con Jira

aparecerá en un apartado u otro de la pantalla.

Una decisión atraviesa 4 estados a lo largo de su ciclo de vida, tal como se observa en la Figura 7.6. Esos estados son:

1. **Draft:** Indica la fase de creación de la decisión.
2. **Option:** Estado opcional que permite a los miembros de un grupo sugerir alternativas para la decisión.
3. **Vote:** Habilita la votación de las distintas opciones disponibles. Si la decisión pertenece a un grupo, todos sus miembros pueden votar; sin embargo, en las decisiones de tipo ruleta, solo el creador tiene permitido emitir el voto. En este estado, la interfaz de votación se adaptará según el tipo de decisión seleccionado.
4. **Finish:** Es el estado final e inmutable de la decisión, en el cual ya no es posible editarla ni registrar nuevos votos.

Las transiciones posibles entre estos estados son las siguientes:

- **Draft** → **Option:** Se activa automáticamente una vez definidos el título y el tipo de la decisión, habilitando a los miembros del grupo proponer opciones.
- **Draft** → **Vote:** Para realizar esta transición, es necesario haber definido el título, el tipo y, al menos, dos opciones.

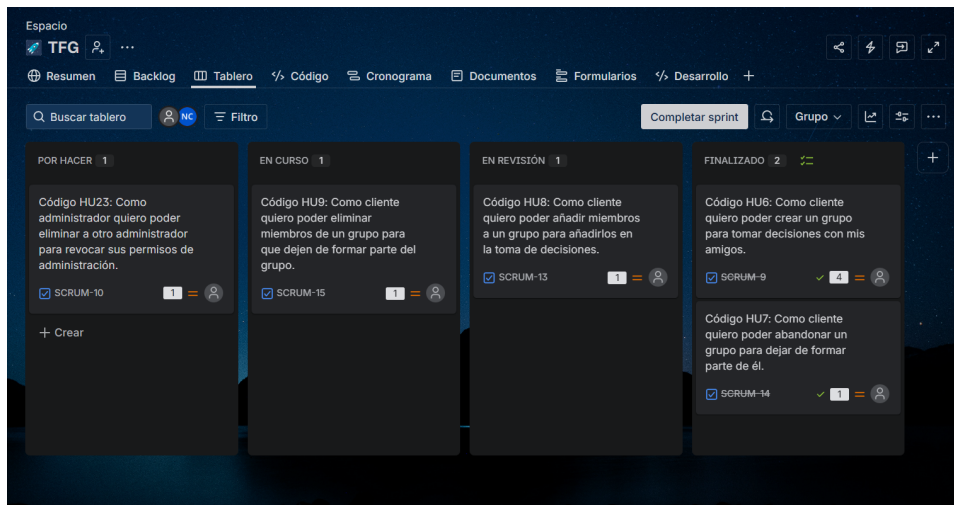


Figura 7.5: Gestión de las historias de usuario con Jira

- **Option** → **Vote**: Esta transición puede ejecutarse de dos formas: de manera manual, mediante un botón accionado por el creador de la decisión, o de forma automática a través de una *Edge Function* -que permite encapsular operaciones que requieren ejecutarse en el servidor- la cual se ejecuta una vez por minuto y comprueba si la hora de finalización del estado actual es menor o igual que la hora actual, actualizando el estado de la decisión cuando se cumple dicha condición.
- **Vote** → **Finish**: Puede ocurrir de tres maneras: manualmente por el creador, automáticamente mediante la *Edge Function*, o de forma automática al completarse los votos de todos los participantes. Esta última vía se gestiona mediante un **trigger** que se dispara con cada nueva votación. Por otro lado, las decisiones de tipo ruleta tienen un flujo de cierre distinto: dado que el creador genera un único voto aleatorio, la decisión finaliza automáticamente justo después de registrarse dicha acción.

Una vez que una decisión alcanza el estado *Finish*, si el proceso de votación concluye con un empate entre las opciones más votadas, el creador de la decisión podrá resolverlo mediante un mecanismo de desempate. Para ello, dispondrá de dos alternativas: emitir un voto adicional a favor de una de las opciones empatadas o utilizar una ruleta de selección aleatoria. En ambos casos, la opción seleccionada recibirá un voto adicional, resolviendo el empate y determinando el resultado final de la decisión.

Para garantizar una experiencia de usuario fluida, interactiva y colaborativa, el sistema requiere que cualquier modificación en los datos se refleje instantáneamente en todos los clientes conectados. Para lograr esta sincro-

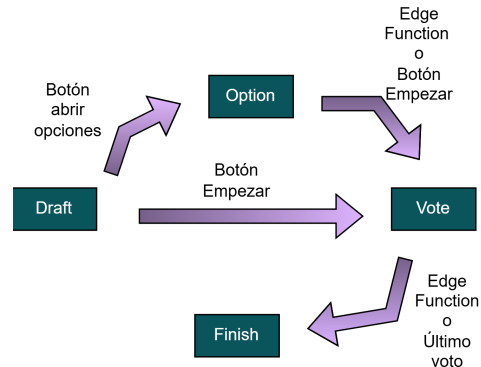


Figura 7.6: Ciclo de vida de una decisión

nización en tiempo real con un acoplamiento mínimo, se implementó una arquitectura reactiva (ver Figura 7.7) basada en tres pilares: Supabase Realtime, Dart Streams y Riverpod.

1. Captura de Eventos en la Base de Datos (Supabase Realtime):
El flujo comienza en el backend con la activación de la funcionalidad **Realtime** en las tablas críticas de *Supabase* (basada en el sistema de replicación lógica de PostgreSQL). Al habilitar esta opción, la base de datos empieza a escuchar activamente los eventos de inserción (*INSERT*), actualización (*UPDATE*) y eliminación (*DELETE*). *Supabase* expone estos cambios a través de un canal de *WebSockets*, transformando cualquier mutación en la base de datos en un evento de red inmediato.
2. Consumo y Canalización del Flujo (Dart Streams):
En el lado de la aplicación, el SDK de *Supabase* para *Flutter* se suscribe a dicho canal de *WebSockets*. Esta suscripción se gestiona mediante un **Stream** de Dart, el cual actúa como un conducto o tubería asíncrona de datos continua. En lugar de realizar peticiones HTTPS tradicionales de tipo pull (donde la aplicación solicita información periódicamente), el **Stream** opera bajo un modelo push: permanece abierto y en espera, reaccionando de manera pasiva y emitiendo un nuevo elemento cada vez que el servidor notifica un cambio.
3. Gestión de Estado y Reactividad en la UI (Riverpod):
Para evitar que la lógica de red se mezcle con la interfaz de usuario, se introduce **Riverpod** como gestor de estado y pieza articuladora. Se define un proveedor de tipo **StreamProvider**, el cual se encarga de escuchar el **Stream** proveniente de *Supabase*. Las ventajas de delegar este control a **Riverpod** son clave:

- **Desacoplamiento:** La interfaz de usuario no conoce los detalles de la conexión a la base de datos; solo consume el estado expuesto por el proveedor.
- **Gestión de Ciclo de Vida:** Riverpod controla automáticamente la apertura y el cierre de las conexiones a los canales en tiempo real, destruyendo el **Stream** cuando la pantalla se destruye para evitar fugas de memoria (*memory leaks*).
- **Manejo de Estados Asíncronos:** Permite a las vistas reaccionar de forma nativa a tres estados bien definidos: carga (*loading*), error (*error*) o datos listos (*data*), actualizando los componentes visuales de la UI mediante su método `ref.watch()` sin necesidad de redibujar la pantalla completa de forma manual.

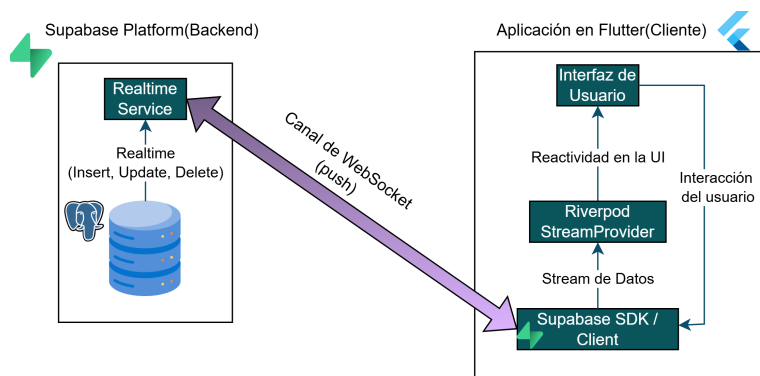


Figura 7.7: Arquitectura reactiva de la aplicación

7.2.2. Decisiones de películas y series (`watch_decision`)

Este tipo de decisiones constituye una variante de las decisiones estándar y se divide en dos vertientes principales: la búsqueda directa de un título específico y la generación de sugerencias personalizadas basadas en filtros.

Al ser una extensión de las decisiones normales, comparten características comunes, pero con un flujo simplificado. Internamente denominadas `watch_decision`, constan únicamente de los estados *vote* y *finish*. Además, permiten configurar un tiempo máximo de expiración para la votación. El sistema de votación funciona de manera idéntica al estándar, almacenando el identificador (ID) de la película o serie seleccionada. Asimismo, se incluye un `trigger` en la base de datos que detecta cuándo un usuario ha finalizado su votación; en el momento en que todos los participantes completan el proceso, este mecanismo cambia automáticamente el estado de la decisión a *finish*.

Para la gestión de sugerencias, los filtros seleccionados por el usuario (tipo de contenido, plataforma de *streaming* disponible en España, puntuación, fecha de estreno, orden de visualización y límite de resultados) se procesan y almacenan en la base de datos estructurados en el formato de la URL que se utilizará para la consulta. Un ejemplo de URL es `/discover/movie?endpoint=movie&language=es-ES&sort_by=popularity.desc&watch_region=ES&with_watch_providers=337&with_genres=35%7C10402&with_runtime.gte=60&with_runtime.lte=150&primary_release_date.gte=2007-01-01&primary_release_date.lte=2026-12-31&vote_average.gte=7.0&vote_average.lte=10.0`. Esta sería la URL correspondiente a una búsqueda de películas disponibles en Disney+, cuyo género sean comedia o musical, con una puntuación entre 7 y 10, una duración entre 60 y 150 minutos y estrenada entre el 2007 y el 2026, devolviendo los resultados ordenados por popularidad.

Integración con la API de TMDB

La obtención de información sobre el catálogo audiovisual se realiza mediante la API de TMDB (*The Movie Database*) con un proceso como el que se puede observar en la Figura 7.8. Dado que las peticiones requieren el uso de una clave privada (*API Key*), todos los métodos de consulta se ejecutan de forma segura desde el lado del servidor a través de una *Edge Function*, evitando así la exposición de credenciales en el cliente.

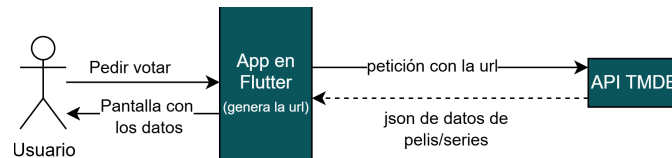


Figura 7.8: Respuesta de la API TMDB

La API de TMDb proporciona información detallada sobre películas (*movie*), series (*tv*) y profesionales de la industria (*person*), como actores, directores y productores. Permite realizar búsquedas específicas por ID o nombre (*search*), así como explorar contenido mediante un potente motor de descubrimiento avanzado (*discover*). Entre sus filtros disponibles se incluyen:

- Lanzamiento e Idioma: Fecha de estreno, tipo de lanzamiento, idioma original y región.
- Comunidad: Puntuación media y número mínimo de votos.
- Contenido: Inclusión o exclusión de géneros, compañías productoras y etiquetas temáticas (keywords).

- Talento: Reparto de actores y miembros del equipo técnico.
- Streaming: Disponibilidad en plataformas y tipo de monetización (suscripción, alquiler o compra).
- Clasificación: Criterios de ordenamiento personalizados.

Además, mediante la autenticación de usuarios, la API permite interactuar directamente con la plataforma para gestionar listas personalizadas y puntuar títulos.

El sistema interactúa con la API en cuatro puntos clave del flujo de la aplicación:

Sugerencias: Mediante el endpoint *discover* realiza una petición parametrizada con los filtros del usuario y devuelve los resultados paginados (20 títulos por página). Con el objetivo de ofrecer una experiencia de usuario fluida sin recargas entre opciones, el sistema solicita de forma proactiva la siguiente página cuando al usuario le restan solo 5 opciones por visualizar. Para permitir la reanudación de la sesión en cualquier momento, se almacena en la base de datos la página actual y el ID del último título evaluado.

Detalles: Solicita información extendida de un título específico a partir de su ID. Esta información se corresponde al eslogan, título, fecha, duración, estado, actores, directores, productores, plataformas en las que está disponible, presupuesto, recaudación, compañías productoras, trailer, y producciones similares.

Historial: Recupera las opciones votadas en una decisión, agrupadas por ID y ordenadas por número de votos. Realiza una petición masiva para obtener los datos básicos de cada opción. Si el usuario desea profundizar, puede acceder al módulo de *Detalles*.

Buscador: Permite al usuario interactuar desde una decisión normal. Al introducir un título marcado como contenido audiovisual, consulta la API con *search* y autocompleta la descripción de la opción con los metadatos obtenidos.

Su uso es gestionado mediante Edge Functions. Las cuales reciben los parámetros para la búsqueda como el ID o el título, usándolos en la construcción de la url junto con la API KEY. Con la API KEY, hacen una petición *fetch* a la base de datos, recibiendo un *json* como el de la Figura 7.9 con los resultados.

```

page: 1,
- results: [
  - {
    adult: false,
    backdrop_path: "/q90ts28BY3rkIQhrytzm1m2u.jpg",
    genre_ids: [
      12,
      16,
      10751,
      14
    ],
    id: 62177,
    title: "Tadomaha!",
    original_language: "en",
    original_title: "Brave",
    overview: "Merida, la indómita hija del Rey Fergus y de la Reina Elinor, es una habilidosa arquera que decide romper con una antigua costumbre, que es sagrada para los señores de la tierra: el gigantesco Lord MacGuffin, el malhumorado Lord Macintosh y el sarcástico Lord Disswall. Los señores de Merida desencadenan el caos y la furia en el reino. Además, pide ayuda a una sabia anciana que le concede un deseo muy desafortunado. La muchacha tendrá que afrontar grandes peligros antes de aprender qué es la auténtica valentía.",
    popularity: 15.6291,
    poster_path: "/c1rcu400huG0P3oK0Q7A21wep.jpg",
    release_date: "2012-06-21",
    as_fscore: false,
    video: false,
    vote_average: 7.832,
    vote_count: 14346
  },
  - {
    adult: false,
    backdrop_path: "/8e1fdu9Qe2A8xgtD45546XXx.jpg",
    genre_ids: [
      28,
      32,
      878
    ],
    id: 822219,
    title: "Capitán América: Brave New World",
    original_language: "en",
    original_title: "Captain America: Brave New World",
    overview: "Tras reunirse con el recién elegido presidente de los EE. UU., Thaddeus Ross, Sam se encuentra en medio de un incidente internacional. Debe descubrir el motivo que se esconde tras un extraño complot global. ¿Será de nuevo un enemigo el que se oculta en la sombra?",
  }
]

```

Figura 7.9: Respuesta de la API TMDb

7.2.3. Seguridad y Control de Acceso

La seguridad de la plataforma se ha diseñado bajo un enfoque multicapa, garantizando la integridad de los datos, la confidencialidad de la información sensible y el cumplimiento del principio de menor privilegio, tanto en el backend como en el cliente.

Autenticación y Verificación de Identidad

Como primer mecanismo de seguridad, el sistema delega la gestión de la autenticación en Supabase Auth, un servicio desacoplado basado en la arquitectura GoTrue, el protocolo OAuth 2.0 y el uso de JSON Web Tokens (JWT). Para reducir el riesgo de registros automatizados y verificar la identidad del usuario, se implementa un proceso obligatorio de validación de la dirección de correo electrónico mediante un canal externo (out-of-band).

Durante el proceso de registro (sign-up), Supabase Auth crea la cuenta de usuario con el correo electrónico pendiente de verificación. De forma simultánea, genera un token de un solo uso con un tiempo de validez limitado, el cual se incorpora a un enlace seguro enviado mediante correo electrónico. Este mecanismo se emplea tanto para la confirmación de nuevas cuentas como para el restablecimiento de contraseñas. La entrega de dichos correos se realiza mediante el protocolo SMTP utilizando un servidor de retransmisión de correo (SMTP Relay).

Para posibilitar el envío de estos correos electrónicos, Supabase Auth

se configura con un servidor SMTP externo. En este proyecto se ha utilizado el servicio SMTP de Gmail, registrando en el panel de configuración de Supabase la dirección de correo utilizada para el envío y una contraseña de aplicación (App Password) generada específicamente para este propósito. Esta contraseña, distinta de la credencial principal de la cuenta de Google, permite que Supabase se autentique de forma segura frente al servidor SMTP sin exponer la contraseña real del usuario.

Una vez configurado el servidor SMTP, Supabase actúa como cliente SMTP. Cuando se produce un evento que requiere el envío de un mensaje (como el registro de un nuevo usuario o una solicitud de recuperación de contraseña), Supabase genera automáticamente el contenido del correo, incorpora el enlace seguro con el token correspondiente y establece una conexión autenticada con el servidor SMTP de Gmail. Tras autenticar correctamente a Supabase mediante las credenciales SMTP configuradas, el servidor acepta el mensaje y se encarga de su distribución hasta el proveedor de correo del destinatario, siguiendo el funcionamiento habitual del protocolo SMTP. De este modo, la aplicación Flutter no interviene en el proceso de envío y nunca gestiona directamente las credenciales del servicio de correo. Este enfoque permite desacoplar la gestión del correo electrónico de la aplicación cliente, centralizando el proceso en Supabase y evitando que las credenciales SMTP queden expuestas en el código fuente de la aplicación móvil.

Cuando el usuario accede al enlace de verificación desde la aplicación desarrollada en Flutter, Supabase Auth recibe el token incluido en dicho enlace y lo valida mediante su servicio de autenticación. El backend verifica que dicho token sea válido, que no haya expirado, que corresponda al usuario asociado y que no haya sido utilizado previamente. Si todas las comprobaciones son satisfactorias, la cuenta se marca como verificada y Supabase crea una nueva sesión autenticada, emitiendo un Access Token y un Refresh Token basados en el estándar JWT. Todo este proceso queda representado en la Figura 7.10

Una vez autenticado, el SDK de Supabase mantiene la sesión gestionando el almacenamiento local de los tokens de autenticación en el dispositivo. Ambos tokens se generan de forma independiente y cumplen funciones diferentes.

El Access Token es un JWT firmado digitalmente por Supabase que contiene información sobre la identidad del usuario y los permisos asociados a la sesión. Posee una vida útil limitada y se incluye automáticamente en las peticiones realizadas a los servicios protegidos. En cada petición protegida, Supabase verifica la firma del token y extrae sus claims (declaraciones o atri-

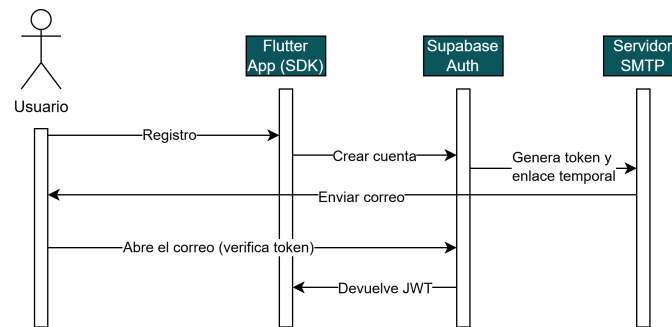


Figura 7.10: Proceso de verificación del correo electrónico

butos del token) para identificar al usuario autenticado y aplicar las políticas de seguridad definidas mediante Row Level Security (RLS), sin necesidad de volver a autenticar al usuario mediante sus credenciales. Cuando este expira, el cliente utiliza el Refresh Token para solicitar de forma transparente un nuevo Access Token. Durante este proceso, Supabase también genera un nuevo Refresh Token, invalidando el anterior e implementando un mecanismo de rotación de tokens que incrementa la seguridad frente a posibles ataques de reutilización. Mientras la sesión permanezca vigente y no sea invalidada, este proceso se realiza automáticamente, evitando que el usuario tenga que volver a introducir sus credenciales.

Seguridad en Capa de Base de Datos y Control de Acceso

Para proteger la persistencia de los datos, todas las funciones y consultas ejecutadas en la base de datos realizan una estricta validación de los permisos del cliente de forma jerárquica. Este control de acceso se articula mediante tres criterios no excluyentes, pudiendo solaparse varios de ellos para autorizar una única operación:

- **Control de Acceso Basado en Roles (RBAC):** Se evalúa el rol asignado al usuario (cliente o administrador) dentro del sistema.
- **Validación de Pertenencia (Contexto de Grupo):** Se verifica si el usuario pertenece activamente al grupo o entidad sobre la que pretende realizar modificaciones.
- **Políticas de Propietario (Ownership):** Se comprueba de manera unívoca que las acciones que alteren datos críticos o privados correspondan estrictamente a la cuenta del usuario autenticado.

Aislamiento de Lógica Sensible mediante Edge Functions

En escenarios donde las acciones requieren privilegios elevados o exponen vectores de ataque, se despliegan *Edge Functions* para aislar la lógica del cliente. Un caso crítico de esta arquitectura ocurre durante el inicio de sesión: dado que *Supabase* restringe de forma nativa la autenticación al uso exclusivo de *email* y contraseña, y la aplicación permite el acceso mediante nombre de usuario, se desarrolló una *Edge Function* que realiza la conversión interna del nombre de usuario al correo electrónico asociado.

El uso de *Edge Functions* para este propósito, en lugar de funciones SQL tradicionales expuestas, aporta las siguientes ventajas competitivas en seguridad:

- **Prevención de Phishing y Extracción de Datos:** Evita que un atacante pueda realizar consultas masivas (*scraping*) de correos electrónicos mediante fuerza bruta sobre los nombres de usuario.
- **Encapsulamiento y Ofuscación:** Oculta los detalles de implementación interna y restringe el acceso directo al esquema protegido `auth.users` de PostgreSQL.
- **Escalabilidad y Defensas Avanzadas:** Permite interconectar el flujo con lógica de terceros en el futuro o implementar capas adicionales de infraestructura como políticas de límite de peticiones (*rate-limiting*) y protección anti-bots.

Seguridad en el Frontend y Roles de Administración

En la capa de aplicación desarrollada en *Flutter*, la interfaz de usuario adapta dinámicamente las funcionalidades disponibles en función del estado de autenticación y del rol del usuario. De este modo, únicamente se muestran o habilitan aquellas acciones para las que el usuario dispone de permisos. Por ejemplo, la edición de una decisión está restringida a su creador y a los usuarios con rol de administrador (Figura 7.11a). De forma análoga, la gestión de las opciones asociadas a una decisión puede ser realizada por el creador de la opción, el creador de la decisión o un administrador (Figura 7.11b). Estas comprobaciones se implementan comparando el identificador del usuario autenticado con el identificador del creador del recurso o verificando el rol asignado al usuario, como se muestra en el siguiente fragmento de código: `"if (votDec.id_creator == currentUserId || currentUserType == 'admin')"`



Figura 7.11: Privilegios de gestión

Finalmente, la aplicación delega la gobernanza total del sistema al rol de **Administrador**. Este perfil cuenta con facultades de moderación global sobre usuarios, grupos y decisiones. Como salvaguarda crítica del sistema, la lógica de negocio estipula que solo un usuario con rol de administrador posee la autorización de modificar o escalar los roles del resto de los usuarios, previniendo ataques de escalada de privilegios.

7.2.4. Gestión de grupos y usuarios

La gestión de usuarios y grupos constituye uno de los pilares fundamentales de la aplicación, ya que son imprescindibles para cumplir el objetivo principal, la toma de decisiones en grupo. Para garantizar la consistencia de la información, esta funcionalidad combina la lógica implementada en Flutter con las restricciones y políticas definidas en Supabase.

Gestión de grupos

La creación de grupos puede ser realizada por cualquier usuario autenticado mediante una inserción en la tabla *grupo*. Tras registrar el nuevo grupo, un **trigger** en la base de datos inserta automáticamente un registro en la

tabla *group_members* que vincula el grupo con su creador, asignándole la condición de miembro.

Dentro de un grupo, los miembros pueden realizar acciones como invitar o eliminar a otros integrantes, abandonar el grupo o modificar la información básica del mismo (nombre e imagen). Todas estas operaciones están sujetas a un proceso de verificación previo: la base de datos valida que la acción provenga del usuario autenticado (mediante Supabase Auth) y que dicho usuario pertenezca al grupo (consultando la tabla *group_members*).

En el caso particular de cada gestión:

- **Invitación de miembros:** Requiere ingresar el nombre de usuario del nuevo miembro. El frontend comprueba que el usuario exista y que no pertenezca previamente al grupo. De cumplirse ambas condiciones, se obtiene su identificador único (ID) y se le envía una notificación de invitación.
- **Eliminación de miembros:** Se obtiene el ID del usuario a eliminar, se le notifica la acción y se borra la relación correspondiente en la tabla *group_members*.
- **Abandono del grupo:** Sigue un flujo similar a la eliminación, con la diferencia de que el registro eliminado en *group_members* corresponde al del propio usuario autenticado, redirigiéndolo posteriormente a la pantalla de inicio de la aplicación y omitiendo el envío de notificaciones.

La eliminación explícita de un grupo está reservada únicamente a los administradores del sistema. No obstante, si el último integrante abandona el grupo, un **trigger** en Supabase ejecuta la eliminación automática del grupo para evitar grupos vacíos. Para asegurar el correcto funcionamiento de este mecanismo, cada vez que se borra un registro en *group_members* se verifica la existencia de otros miembros en el mismo grupo; si el conteo resulta en cero, se elimina la entrada en la tabla *grupo*. Esta comprobación automática garantiza la limpieza del sistema tanto si un usuario abandona el grupo como si un administrador elimina progresivamente a sus miembros.

Por otra parte, si un administrador elimina directamente una entrada en la tabla *grupo*, un borrado en cascada (**ON DELETE CASCADE**) configurado en Supabase elimina automáticamente todos los datos vinculados (miembros, notificaciones, decisiones y votos).

En cuanto a la privacidad y visualización, los usuarios solo pueden acceder a los grupos a los que pertenecen, mientras que los administradores

pueden consultar la totalidad de los grupos. Esta restricción está garantizada mediante políticas de seguridad a nivel de fila (RLS - Row Level Security).

Finalmente, la vista del grupo expone las decisiones activas, la lista de miembros y el historial de decisiones finalizadas. Esta información es filtrada por políticas RLS en las tablas correspondientes y organizada dinámicamente según el estado de las decisiones en la interfaz mediante *Riverpod*.

Gestión de usuarios

Los usuarios pueden registrarse en la aplicación proporcionando un nombre de usuario, nombre, dirección de correo electrónico y contraseña. El registro se realiza mediante el servicio de autenticación de Supabase Auth, que crea un nuevo usuario en la tabla *auth.users*. Tras esta operación, un **trigger** implementado en la base de datos inserta automáticamente un nuevo registro en la tabla usuario con la información adicional de la aplicación (nombre de usuario, nombre, imagen y tipo de usuario). La unicidad del nombre de usuario se garantiza mediante una restricción **UNIQUE** sobre dicho atributo. En caso de incumplirse esta restricción, la transacción se revierte y el registro no llega a completarse. Para finalizar el proceso de registro, el usuario debe verificar su dirección de correo electrónico mediante el mecanismo descrito en el apartado anterior. Una vez que se ha verificado, Supabase rellena el campo *email_confirmed_at* de la tabla *auth.users*, permitiendo a partir de ese momento el acceso a la aplicación.

Una vez autenticados, los usuarios pueden modificar su información personal. Estas operaciones están protegidas mediante políticas Row Level Security (RLS), que únicamente permiten modificar el registro correspondiente al usuario autenticado.

El rol asignado a un usuario únicamente puede ser modificado por un administrador. Antes de ejecutar la operación, el sistema consulta el rol asociado al usuario autenticado y verifica que dispone de privilegios de administración. Como medida de seguridad adicional, un administrador no puede modificar su propio rol, evitando así la pérdida accidental de dichos privilegios.

La eliminación de una cuenta se realiza mediante una Edge Function de Supabase. Esta función utiliza la API de administración de Supabase Auth para eliminar el usuario de la tabla *auth.users*. Como consecuencia, las restricciones **ON DELETE CASCADE** definidas en la base de datos eliminan automáticamente los registros asociados en las tablas *usuario*, *group_members*,

notifications y *watch_decision_position*, garantizando la eliminación completa de la cuenta.

Sincronización en tiempo real

La aplicación utiliza la funcionalidad *Realtime* de Supabase, tal como vimos en la Figura 7.7, para mantener sincronizada la información compartida entre todos los miembros de un grupo. Mediante suscripciones a los cambios producidos en la base de datos, cualquier modificación realizada por un usuario (creación o eliminación de grupos, cambios en los miembros, las decisiones o en las votaciones) se refleja automáticamente en el resto de clientes conectados, sin necesidad de realizar recargas manuales de la información.

Esta estrategia mejora la experiencia de usuario y garantiza que todos los miembros visualicen el mismo estado del grupo prácticamente en tiempo real.

7.2.5. Notificaciones

La aplicación cuenta con un sistema de notificaciones automatizadas que se activan ante eventos clave: la gestión de usuarios en grupos (invitaciones y expulsiones) y el ciclo de vida de las decisiones grupales (creación, actualización y finalización).

Existen cinco tipos de notificaciones distribuidas en dos categorías principales (gestión de grupos y gestión de decisiones):

- **invite_group:** Notifica a un usuario que ha sido invitado a unirse a un grupo, permitiéndole aceptar o denegar dicha invitación.
- **kick_group:** Informa a un usuario que ha sido eliminado de un grupo.
- **create_option:** Avisa sobre la creación de una nueva toma de decisión, la cual inicia en la fase de propuesta de opciones.
- **create_vote:** Notifica a los miembros que una decisión ha cambiado a la fase de votación.
- **end_vote:** Informa al usuario sobre el cierre de una votación.

Como se ilustra en la Figura 7.12, existe una dependencia temporal entre ellas, ya que ciertos flujos requieren que una notificación preceda a otra. Las

flechas violetas expresan obligatoriedad, mientras que las verdes opcionalidad. El paso de `create_option` a `create_vote` es opcional, ya que el estado *option* en las decisiones es opcional.

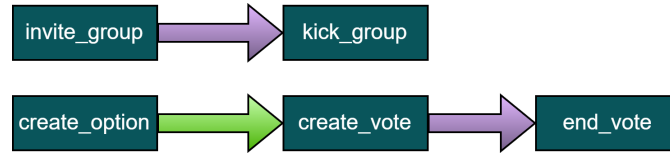


Figura 7.12: Dependencia entre las notificaciones

La generación de estas alertas se realiza mediante dos mecanismos diferenciados:

1. **Desde la aplicación:** Flujo directo en el backend cuando un usuario invita o elimina a otro de un grupo.
2. **Mediante disparadores (*triggers*) en la base de datos:** Automatismos que se ejecutan al insertar o modificar registros en las tablas asociadas a las decisiones.

Por último, el diseño de la interfaz (Figura 7.13) adapta el mensaje informativo según el tipo de alerta. Además, se incluye un componente visual de tipo **badge** en la aplicación para reflejar en tiempo real el número de notificaciones pendientes del usuario.

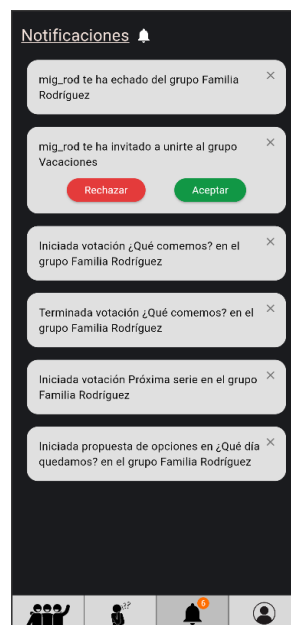


Figura 7.13: Interfaz de las notificaciones

Capítulo 8

Pruebas

Con el objetivo de verificar el correcto funcionamiento del sistema, se ha llevado a cabo una serie de pruebas agrupadas en tres categorías distintas.

8.1. Pruebas unitarias

Las pruebas unitarias han sido llevadas a cabo cubriendo distintos aspectos de la aplicación:

- **Comprobar el email:** Se emplea en el registro y edición de un usuario para verificar la validez del formato del correo electrónico. Se ha verificado:
 1. Que no esté vacío
 2. Que contenga el carácter '@'
 3. Que no contenga el carácter '@' al principio
 4. Que no contenga el carácter '@' al final
 5. Que no contenga más de un carácter '@'
 6. Que incluya al menos un punto ('.')
 7. Que existan caracteres entre el '@' y el punto
 8. Que un formato de correo electrónico correcto no genere ningún error
- **Comprobar el nombre de usuario:** Utilizado en el registro y en la edición del perfil de los usuarios. Consiste en verificar que el campo no esté vacío, no contenga el carácter '@' y que un nombre de usuario válido sea aceptado sin errores.

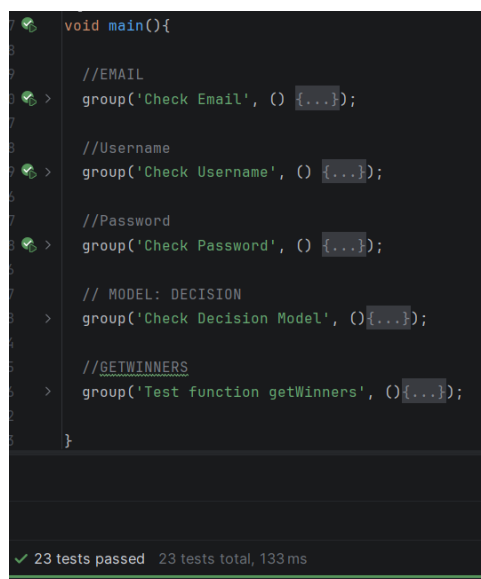
- **Comprobar la contraseña:** También se emplea en los flujos de registro y edición de usuario. Para considerarse válida, debe cumplir los siguientes requisitos:
 1. Que no esté vacía.
 2. Que posea una longitud mínima de 8 caracteres
 3. Que contenga al menos una letra minúscula
 4. Que contenga al menos una letra mayúscula
 5. Que contenga al menos un dígito numérico
 6. Que disponga de al menos un carácter especial (por ejemplo: ‘_’, ‘;’, ‘.’, ‘¿’, etc.)
 7. Que una contraseña válida no genere ningún error.
- **Modelo de decisión:** Se ha verificado el correcto comportamiento del modelo lógico más complejo del sistema, comprobando:
 1. Que el mapeo genere un objeto correcto
 2. Que se asigne el valor por defecto en caso de omitirse un parámetro obligatorio
 3. Que se asigne el valor *null* si no se proporciona un parámetro opcional
 4. Que el proceso de desmapeo se realice de forma correcta
- **La función que devuelve las opciones ganadoras:** Se ha comprobado que, dada una serie de opciones de votación, la función devuelva correctamente aquellas que contienen un número de votos igual que el valor ganador.

Como se puede apreciar en la Figura 8.1, todas las pruebas unitarias se han ejecutado con éxito.

8.2. Pruebas de widget

Las pruebas de widget tienen como objetivo validar el correcto funcionamiento de la interfaz y la lógica visual de distintas pantallas del sistema. Tal como se observa en la Figura 8.2, todos los tests han resultado satisfactorios.

- **Inicio de sesión** El inicio de sesión cuenta con un aspecto como el que podemos ver en la Figura 8.3a. Para validar esta pantalla, se han diseñado pruebas para verificar:
 - La presencia de los elementos principales de la interfaz:



```
void main(){  
    //EMAIL  
    group('Check Email', O {...});  
    //Username  
    group('Check Username', O {...});  
    //Password  
    group('Check Password', O {...});  
    // MODEL: DECISION  
    group('Check Decision Model', O {...});  
    //GETWINNERS  
    group('Test function getWinners', O {...});  
}
```

✓ 23 tests passed 23 tests total, 133 ms

Figura 8.1: Resultado de los tests unitarios

1. El campo de texto para el usuario
 2. El campo de texto para la contraseña
 3. El título de la aplicación “DEGO”
- Que al intentar iniciar sesión con los campos vacíos, el sistema devuelva dos mensajes de error indicando 'Campo obligatorio'
 - La correcta entrada de texto en los campos de usuario y contraseña
 - Que el icono del ojo alterne correctamente la visibilidad de la contraseña
 - Que el botón “He olvidado mi contraseña” despliegue un diálogo de recuperación que contenga los botones de “Enviar” y “Cancelar”
- **Registro** Esta pantalla, cuyo diseño se muestra en la Figura 8.3b, cuenta con un conjunto de pruebas análogo al del inicio de sesión. En este caso se verifica:
- Que la pantalla renderice los elementos principales:
 1. El campo para el nombre de usuario
 2. El campo para el nombre completo
 3. El campo para el correo electrónico
 4. El campo para la contraseña
 5. El campo para confirmar la contraseña
 6. El botón de registro

```

//LOGIN SCREEN
group('Login screen', () {});

//REGISTER
group('Register screen', () {});

//SIMPLE VOTE
group('Simple Vote', () {});

//CREATE DECISION
group('Create decision', () {});

✓ 23 tests passed 23 tests total, 4 sec 279 ms
C:\flutter\bin\flutter.bat --no-color test
Testing started at 11:46 ...

```

Figura 8.2: Resultado de los tests de widgets

7. La flecha para volver a la pantalla anterior
- Que al intentar registrarse con los campos vacíos se muestren 5 errores de “Campo obligatorio”
 - La posibilidad de escribir en cada uno de los campos de texto
 - Que el icono del primer ojo modifique la visibilidad de la contraseña
 - Que el icono del segundo ojo modifique la visibilidad de la confirmación de la contraseña
 - Que se lance un mensaje de error si la contraseña y su confirmación no coinciden
- **Voto simple** La pantalla de votación simple, ilustrada en la Figura 8.3c, ha sido evaluada mediante los siguientes casos de prueba:
 - Visualización de los componentes clave:
 1. El título de la decisión
 2. El listado de opciones seleccionables
 3. El botón de cancelación
 4. El botón para confirmar el voto
 - Capacidad para seleccionar una opción de la lista
 - Exclusividad en la selección (garantizar que solo se pueda marcar una opción a la vez)

- Mostrar un mensaje de error al intentar votar sin haber seleccionado ninguna opción
- **Crear una decisión** La creación de una decisión estándar, cuyo aspecto es mostrado en la Figura 8.3d, ha sido validada mediante las siguientes pruebas:
 - Dispone de los elementos principales:
 1. El título informativo de la pantalla
 2. El título de la decisión
 3. El desplegable con los tipos de decisiones
 4. El botón para añadir opciones
 5. El botón para sugerir opciones
 6. La fecha máxima para sugerir opciones
 7. La fecha máxima para votar
 8. El botón de cancelación
 9. El botón de creación de la decisión
 - La obligatoriedad de poner un título
 - La posibilidad de escribir un título
 - La capacidad de cambiar el tipo de decisión
 - La obligatoriedad de disponer de al menos dos opciones para poder empezar la votación
 - Las opciones asociadas a la decisión son mostradas
 - Borrar una opción actualiza el listado de opciones asociadas
 - Las decisiones de tipo ruleta no disponen de fecha máxima para sugerir opciones

8.3. Pruebas de sistema

Las pruebas de sistema implican simular el flujo completo de extremo a extremo (*end-to-end*) que realizaría un usuario real en la aplicación. Para evaluar este comportamiento bajo un estándar común, se han definido una serie de flujos de operaciones(véase la Figura 8.4):

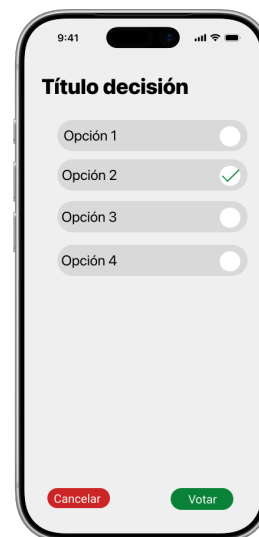
1. Registro de un nuevo usuario
2. Inicio de sesión, acceso a un grupo y creación de una decisión de visualización
3. Inicio de sesión, acceso a un grupo específico y emisión de un voto en una decisión activa



(a) Aspecto del Login



(b) Registro



(c) Aspecto Votación simple



(d) Crear una decisión

Figura 8.3: Aspecto de las pantallas usadas en las pruebas de widget

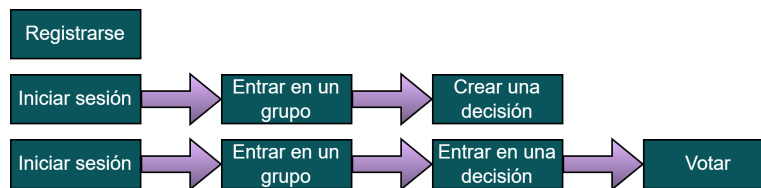


Figura 8.4: Flujo de las pruebas de sistemas

Como se constata en la Figura 8.5, todos los tests de integración del sistema se completaron sin complicaciones, demostrando la robustez y el correcto comportamiento de la aplicación en un escenario de uso real.

```

PS C:\Users\USUARIO\Documents\Universidad\dego> flutter test .\integration_test\system_test_register.dart
00:00 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_register.dart
00:59 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_register.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
01:00 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_register.dart
01:01 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_register.dart
01:09 +0: Register
supabase.supabase_flutter: INFO: ***** Supabase init completed *****
01:19 +1: All tests passed!
  
```

(a) Registro

```

PS C:\Users\USUARIO\Documents\Universidad\dego> flutter test .\integration_test\system_test_watch.dart
00:00 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_watch.dart
00:49 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_watch.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
00:50 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_watch.dart
00:52 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test_watch.dart
01:02 +0: Create decision watch
supabase.supabase_flutter: INFO: ***** Supabase init completed *****
01:23 +1: All tests passed!
  
```

(b) Crear decisión de visualización

```

PS C:\Users\USUARIO\Documents\Universidad\dego> flutter test integration_test/system_test.dart
00:00 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test.dart
00:48 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test.dart
✓ Built build/app/outputs/flutter-apk/app-debug.apk
00:49 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test.dart
00:51 +0: loading C:/Users/USUARIO/Documents/Universidad/dego/integration_test/system_test.dart
00:58 +0: User work stream
supabase.supabase_flutter: INFO: ***** Supabase init completed *****
01:13 +1: All tests passed!
  
```

(c) Votación

Figura 8.5: Resultado de los tests de sistema

Capítulo 9

Conclusiones

Tras la finalización del presente Trabajo Fin de Grado puede concluirse que los objetivos inicialmente planteados han sido alcanzados de forma satisfactoria. El resultado obtenido es una aplicación funcional que permite facilitar la toma de decisiones en grupo mediante diferentes metodologías de votación, proporcionando además una experiencia de uso intuitiva, segura y adaptable a distintos escenarios.

En relación con los objetivos definidos al inicio del proyecto, todos ellos han sido cumplidos:

- **Gestionar perfiles, grupos y decisiones:** La aplicación permite crear, modificar y eliminar perfiles de usuario, grupos y decisiones. Estas funcionalidades están disponibles de acuerdo con los permisos asignados a administradores y usuarios estándar.
- **Integrar múltiples metodologías de toma de decisiones:** Se han implementado distintos sistemas de votación, incluyendo voto único, voto múltiple, ranking y ruleta, permitiendo adaptar el proceso de decisión a diferentes necesidades.
- **Permitir consultar el historial de decisiones:** Los usuarios pueden consultar las decisiones finalizadas, visualizando las opciones ordenadas según el número de votos obtenidos y el resultado final de cada una de ellas.
- **Ofrecer opciones de personalización:** La aplicación adapta automáticamente su apariencia al tema claro u oscuro configurado en el dispositivo del usuario, mejorando la experiencia de uso.
- **Garantizar la seguridad y privacidad de los datos:** Se han implementado diversas medidas de protección tanto en el cliente como

en el servidor, incluyendo autenticación mediante JWT, políticas *Row Level Security* (RLS), Edge Functions y controles de acceso basados en permisos.

- **Proporcionar categorías de decisión por defecto:** Se han desarrollado decisiones especializadas para la selección de películas y series, integrando información obtenida mediante la API de TMDB y permitiendo aplicar diferentes filtros.
- **Incorporar herramientas de búsqueda y filtrado:** La aplicación dispone de buscadores que facilitan la localización de grupos y usuarios, agilizando la navegación por el sistema.
- **Optimizar el proceso de toma de decisiones:** Las pruebas realizadas con usuarios permitieron comprobar que la aplicación facilita alcanzar consensos de manera más rápida y reduce las discusiones derivadas de la elección de una alternativa.

Desde un punto de vista personal, el desarrollo de este proyecto ha supuesto un reto con una experiencia de gran valor formativo. Durante su realización se han adquirido conocimientos relacionados con el desarrollo de aplicaciones multiplataforma mediante Flutter, el diseño e integración de servicios backend utilizando Supabase, la aplicación de mecanismos de seguridad en sistemas distribuidos y la planificación y gestión de un proyecto software de tamaño medio. Asimismo, el trabajo ha permitido consolidar buenas prácticas de ingeniería del software, tanto en el diseño de la arquitectura como en la implementación, documentación y validación del sistema desarrollado.

En conjunto, el proyecto ha permitido desarrollar una solución completa que cumple los objetivos inicialmente planteados y que constituye una base sólida sobre la que continuar incorporando nuevas funcionalidades y mejoras.

9.1. Líneas de trabajo futuro

Aunque la aplicación desarrollada cumple los objetivos establecidos, existen diversas líneas de trabajo que permitirían ampliar sus capacidades y mejorar la experiencia de uso. Estas posibles mejoras pueden agruparse en tres categorías principales:

- **Mejoras de interfaz.** Se podrían incorporar mayores opciones de personalización visual, permitiendo al usuario seleccionar manualmente distintos temas de color, independientemente de la configuración

del dispositivo. Asimismo, sería posible añadir soporte para múltiples idiomas. Desde el inicio del desarrollo, todos los textos de la aplicación se han gestionado mediante un sistema de internacionalización, lo que facilitaría la incorporación de nuevos idiomas. Del mismo modo, podría ofrecerse el contenido de películas y series adaptado a diferentes regiones o países.

- **Mejoras funcionales.** Entre las posibles ampliaciones se encuentra la incorporación de nuevas modalidades de votación, como decisiones multicriterio, lanzamiento de moneda, generación de números aleatorios u otros mecanismos de selección. También sería posible ofrecer una mayor personalización en las decisiones existentes como, por ejemplo, permitiendo modificar los porcentajes asociados a las opciones de la ruleta. Además, podrían añadirse nuevas categorías de decisiones por defecto, como la selección de restaurantes cercanos mediante filtros de ubicación, precio, tipo de cocina o valoración de los usuarios.
- **Mejoras de seguridad.** Con el objetivo de reforzar la protección de la plataforma, podrían incorporarse mecanismos de limitación de intentos de acceso (*rate limiting*), sistemas de detección de bots en el proceso de autenticación, autenticación en dos factores (2FA) y comprobaciones frente a bases de datos de contraseñas comprometidas para evitar el uso de credenciales inseguras.

Finalmente, el desarrollo de este proyecto demuestra la viabilidad de una plataforma orientada a facilitar la toma de decisiones en grupo mediante herramientas digitales. La arquitectura implementada permite mantener una solución escalable y fácilmente extensible, lo que facilita la incorporación de nuevas funcionalidades sin comprometer el diseño actual del sistema.

Bibliografía

- [1] Flutter. Flutter brand guidelines. <https://flutter.dev/brand>, 2026. Accedido el 17 de febrero de 2026.
- [2] Supabase Inc. Supabase logo. <https://supabase.com/brand-assets>, 2026. Accedido el 4 de marzo de 2026.
- [3] The PostgreSQL Global Development Group. Postgresql logo. <https://www.postgresql.org/about/press/presskit16/base/>, 2026. Accedido el 23 de febrero de 2026.
- [4] Flaticon. Icono de interfaz de usuario. https://www.flaticon.com/free-icon/prototype_4661375, 2026. Accedido el 12 de marzo de 2026.
- [5] Flaticon. Icono de escudo. https://www.flaticon.com/free-icon/shield_8777581, 2026. Accedido el 12 de marzo de 2026.
- [6] Flaticon. Icono de base de datos. https://www.flaticon.com/free-icon/database_9850774, 2026. Accedido el 12 de marzo de 2026.
- [7] Flaticon. Icono de cliente. https://www.flaticon.es/icono-gratis/ordenador-portatil_689355, 2026. Accedido el 8 de abril de 2026.
- [8] Flaticon. Icono de servidor. https://www.flaticon.es/icono-gratis/almacenamiento-del-servidor_6056783, 2026. Accedido el 8 de abril de 2026.
- [9] Fever. Fever - descubre los mejores eventos en tu ciudad. <https://feverup.com/es>, 2026. Accedido el 10 de febrero de 2026.
- [10] TheFork. Thefork - reserva en los mejores restaurantes de tu ciudad. <https://www.thefork.es/>, 2026. Accedido el 10 de febrero de 2026.
- [11] 2NITE. 2nite - nightlife in your pocket. <https://www.2nite.app/>, 2026. Accedido el 10 de febrero de 2026.
- [12] Daccord. Daccord - group decisions, made easy. <https://daccordapp.com/>, 2026. Accedido el 10 de febrero de 2026.

- [13] Tiny Decisions. Tiny decisions - decisions made easy. <https://tinydecisions.app/>, 2026. Accedido el 10 de febrero de 2026.
- [14] Redwerk. Mejores herramientas de desarrollo móvil multiplataforma. <https://redwerk.es/blog/mejores-herramientas-de-desarrollo-movil-multiplataforma/>, 2024. Accedido el 29 de enero de 2026.
- [15] Google. Flutter - build apps for any screen. <https://flutter.dev/>, 2026. Accedido el 14 de febrero de 2026.
- [16] Meta Platforms, Inc. React native - learn once, write anywhere. <https://reactnative.dev/>, 2026. Accedido el 14 de febrero de 2026.
- [17] Google Developers. Android sdk - build for android. <https://developer.android.com/>, 2026. Accedido el 14 de febrero de 2026.
- [18] Apple Inc. ios sdk - apple developer documentation. <https://developer.apple.com/>, 2026. Accedido el 14 de febrero de 2026.
- [19] Ionic Framework. Ionic - the mobile sdk for the web. <https://ionicframework.com/>, 2026. Accedido el 14 de febrero de 2026.
- [20] Microsoft. .net multi-platform app ui (.net maui). <https://dotnet.microsoft.com/es-es/apps/maui>, 2026. Accedido el 14 de febrero de 2026.
- [21] Unity Technologies. Unity - ir a crear. <https://unity.com/es>, 2026. Accedido el 14 de febrero de 2026.
- [22] NativeScript Community. Nativescript - write native with typescript. <https://nativescript.org/>, 2026. Accedido el 14 de febrero de 2026.
- [23] Django Software Foundation. Django - the web framework for perfectionists with deadlines. <https://www.djangoproject.com/>, 2026. Accedido el 15 de febrero de 2026.
- [24] Pallets Projects. Flask. <https://flask.palletsprojects.com/>, 2026. Accedido el 15 de febrero de 2026.
- [25] Sebastián Ramírez. Fastapi - high performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com/>, 2026. Accedido el 15 de febrero de 2026.
- [26] OpenJS Foundation. Express - fast, unopinionated, minimalist web framework for node.js. <https://expressjs.com/>, 2026. Accedido el 15 de febrero de 2026.

- [27] Kamil Myśliwiec. Nest - a progressive node.js framework for building efficient, reliable and scalable server-side applications. <https://nestjs.com/>, 2026. Accedido el 15 de febrero de 2026.
- [28] Broadcom (VMware). Spring boot. <https://spring.io/projects/spring-boot>, 2026. Accedido el 15 de febrero de 2026.
- [29] Taylor Otwell. Laravel - build and ship software with tools crafted for productivity. <https://laravel.com/>, 2026. Accedido el 15 de febrero de 2026.
- [30] Microsoft. Asp.net core - free, cross-platform, open source, a framework for building web apps and services. <https://dotnet.microsoft.com/en-us/apps/aspnet>, 2026. Accedido el 15 de febrero de 2026.
- [31] Ruby on Rails Community. Ruby on rails - a web-app framework that includes everything needed to create database-backed web applications. <https://rubyonrails.org/>, 2026. Accedido el 15 de febrero de 2026.
- [32] Supabase Inc. Supabase - build in a weekend, scale to millions. <https://supabase.com/>, 2026. Accedido el 4 de marzo de 2026.
- [33] The PostgreSQL Global Development Group. Postgresql - the world's most advanced open source relational database. <https://www.postgresql.org/>, 2026. Accedido el 15 de febrero de 2026.
- [34] Oracle Corporation. Mysql. <https://www.mysql.com/>, 2026. Accedido el 15 de febrero de 2026.
- [35] MariaDB Foundation. Mariadb server: the innovative open source database. <https://mariadb.org/>, 2026. Accedido el 15 de febrero de 2026.
- [36] SQLite Development Team. Sqlite. <https://sqlite.org/>, 2026. Accedido el 15 de febrero de 2026.
- [37] MongoDB, Inc. Mongoddb - the developer data platform. <https://www.mongodb.com/>, 2026. Accedido el 15 de febrero de 2026.
- [38] Google. Firebase - google's mobile and web app development platform. <https://firebase.google.com/>, 2026. Accedido el 15 de febrero de 2026.
- [39] Google. Firebase realtime database. <https://firebase.google.com/docs/database>, 2026. Accedido el 15 de febrero de 2026.
- [40] Redis Ltd. Redis - the real-time data platform. <https://redis.io/>, 2026. Accedido el 15 de febrero de 2026.

- [41] Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo de 27 de abril de 2016 (Reglamento General de Protección de Datos). Reglamento general de protección de datos (rgpd). <https://www.boe.es/buscar/doc.php?id=DOUE-L-2016-80807>, 2016. Accedido el 11 de marzo de 2026.
- [42] Jefatura del Estado. Ley orgánica 3/2018 de protección de datos personales y garantía de los derechos digitales (lopdgdd). <https://www.boe.es/eli/es/lo/2018/12/05/3/con>, 2018. Accedido el 11 de marzo de 2026.